

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARRERA DE INFORMATICA



TESIS DE GRADO
"DIGITALIZACIÓN INTELIGENTE DE RECIBOS
MANUSCRITOS PARA PEQUEÑOS COMERCIANTES
MEDIANTE UNA APLICACIÓN MÓVIL BASADA EN
DEEP LEARNING"

Para optar al Título de Licenciatura en Informática

MENCIÓN: ingeniería de sistemas informáticos

POR: WHILLY EDGAR AMORAGA MAMANI

TUTOR: DR. JUAN MARCOS MIRANDA NINA PH.D.

La Paz - Bolivia

2024

DEDICATORIA

A mi mamá Alejandra quien me brindo siempre su amor incondicional y un apoyo constante.

A mi padre Bernabé que me inculco la voluntad de seguir y aprender de los errores, son la guía de mi proceso de formación.

A mis hermanas María, Ana y Sofia que siempre estuvieron conmigo a lo largo de mi vida, me han ayudado y motivado en la vida.

Y especialmente a Dios a pesar de mis errores y fallas, siempre ha estado conmigo.

AGRADECIMIENTOS

A Dios por cuidarme, apoyarme y nunca abandonarme.

A mi familia que siempre estuvo apoyándome y alentándome, mi madre, mis hermanas y mi padre, gracias por todo.

A quienes forman parte de mi vida, a las que agradezco su amistad, compañía, paciencia, conocimiento y apoyo mi gratitud y aprecio.

A mi tutor Dr. Juan Marcos Miranda Nina Ph.D. por la confianza, tiempo, conocimiento y paciencia en todo el proceso mi gratitud enorme y mis mejores deseos.

A mis docentes que no perdieron la voluntad de transmitir sus conocimientos.

RESUMEN

En los últimos años el crecimiento de los contribuyentes del régimen tributario simplificado que acogen a los pequeños comerciantes enfrenta la dificultad de no contar con herramientas digitales debido a las características específicas en este régimen, un capital reducido y están exentos de emitir facturas fiscales recurriendo a recibos, notas de venta manuscritos como control de ventas y extensión de recibo físico informal al cliente. Este estudio propone el diseño e implementación de una aplicación móvil basada en Deep Learning se usaron YOLOv5 y CRNN para entrenamiento y digitalización de recibos manuscritos: detecta áreas clave, reconoce valores, genera comprobantes digitales exportables, y almacena datos en SQLite para el control y análisis de ventas básico. Empleando la metodología ágil Mobile-D, se entrenaron los modelos con recibos recolectados de los pequeños comerciantes de las diferentes áreas de la ciudad de La Paz, logrando una de precisión del 80% en la digitalización. Los resultados evidencian que el rendimiento de las redes neuronales depende críticamente del volumen de datos y ajustes de entrenamiento, validando la app como herramienta viable para la inclusión digital, optimizando las labores diarias y extensión de comprobantes digitales a clientes.

Palabras clave: régimen tributario simplificado, pequeños comerciantes, digitalización de recibos manuscritos, Deep Learning, aplicación móvil, yolov5, CRNN, inclusión digital, SQLite, control de ventas.

ABSTRACT

In recent years, the growth of taxpayers under the simplified tax regime, which includes small merchants, has faced the challenge of lacking digital tools due to the specific characteristics of this regime, such as limited capital and exemption from issuing fiscal invoices. As a result, they rely on handwritten receipts and sales notes as informal records and physical proof of transactions for customers. This study proposes the design and implementation of a mobile application based on Deep Learning. YOLOv5 and CRNN were used for training and digitization of handwritten receipts: the system detects key areas, recognizes values, generates exportable digital receipts, and stores data in SQLite for basic sales control and analysis. Using the agile Mobile-D methodology, the models were trained with receipts collected from small merchants in various areas of the city of La Paz, achieving an 80% accuracy in digitization. The results show that the performance of neural networks critically depends on the volume of data and training adjustments, validating the app as a viable tool for digital inclusion, streamlining daily tasks, and issuing digital receipts to customers.

Keywords: simplified tax regime, small merchants, handwritten receipt digitization, deep learning, mobile application, YOLOv5, CRNN, digital inclusion, SQLite, sales control.

INDICE

CAPITULO I	1
1 MARCO REFERENCIAL	1
1.1 INTRODUCCIÓN	1
1.2 ANTECEDENTES	3
1.2.1 Antecedentes internacionales	3
1.1.1 Antecedentes nacionales	4
1.3 PLANTEAMIENTO DEL PROBLEMA	5
1.3.1 Problema General	5
1.3.2 Problemas Específicos	7
1.4 OBJETIVOS	7
1.4.1 Objetivo General	7
1.4.2 Objetivo Específico	7
1.5 HIPÓTESIS	8
1.6 OPERACIÓN DE VARIABLES	8
1.7 JUSTIFICACIÓN	9
1.7.1 Justificación Social	9
1.7.2 Justificación Económica	9
1.7.3 Justificación Académica	9
1.8 LIMITES Y ALCANCES	10
1.8.1 Alcance	10
1.8.2 Limites	10
1.9 METODOLOGIAS	11

1.9.1	<i>Metodología De Desarrollo.</i>	11
1.9.2	<i>Metodología De La Investigación Científica.</i>	12
CAPITULO II		13
2	MARCO TEORICO	13
2.1	INTRODUCCION	13
2.2	REGIMEN TRIBUTARIO SIMPLIFICADO RTS	14
2.2.1	<i>Actividades comprendidas en el RTS.</i>	14
2.3	INTELIGENCIA ARTIFICIAL (IA)	15
2.4	MACHINE LEARNING	17
2.5	DEEP LEARNING	18
2.5.1	<i>Redes Neuronales.</i>	19
2.5.2	<i>Funciones De Activación.</i>	21
2.5.3	<i>Redes Neuronales Convolucionales (CNN).</i>	27
2.5.3.1	Capa Convolucional (Convolutional Layer)	28
2.5.3.2	Capa De Agrupación O Pooling Layer	31
2.5.3.3	Capa Totalmente Conectada o Fully-Connected Layer (FC).	32
2.5.3.4	Capa Softmax o Softmax layer	35
2.5.3.5	Aplicación De La Redes Neuronales Convolucionales.	36
2.5.4	<i>Redes Neuronales Recurrentes (RNN).</i>	37
2.5.4.1	Redes Neuronales Recurrentes Estándar.	39
2.5.4.2	Redes Neuronales Recurrentes Bidireccionales (BRNN).	40
2.5.4.3	Memoria A Corto Plazo (LSTM).	40
2.5.4.4	Unidades Recurrentes Bloqueadas (GRU).	42

2.5.4.5	Codificador-Decodificador RNN.	43
2.5.5	Yolov5.	43
2.5.6	Tesseract OCR.	45
2.6	METODOLOGIA MOBILE-D.....	46
2.7	TECNOLOGIAS.....	49
CAPITULO III.....		55
3	MARCO APLICATIVO.....	55
3.1	INTRODUCCION	55
3.2	FASE DE EXPLORACION.....	55
3.2.1	<i>Establecimiento De Grupo De Interés.....</i>	<i>55</i>
3.2.2	<i>Requisitos Iniciales.</i>	<i>56</i>
3.2.3	<i>Requerimientos Funcionales.....</i>	<i>57</i>
3.2.4	<i>Requerimientos No Funcionales.....</i>	<i>58</i>
3.2.5	<i>Definición Del Alcance.....</i>	<i>58</i>
3.2.5.1	<i>Limitaciones.</i>	<i>59</i>
3.2.5.2	<i>Alcances.</i>	<i>59</i>
3.2.6	<i>Establecimiento De Tecnologías.....</i>	<i>59</i>
3.3	FASE DE INICIACIÓN.....	60
3.3.1	<i>Configuración Del Ambiente De Desarrollo.</i>	<i>60</i>
3.3.1.1	<i>Preparación Del Ambiente.</i>	<i>60</i>
3.3.2	<i>Planificación De Fases.....</i>	<i>63</i>
3.3.3	<i>Diseño De Aplicación.</i>	<i>65</i>
3.4	FASE DE PRODUCCION	66

3.4.1	<i>Iteración 1</i>	67
3.4.1.1	Captura De Datos.	67
3.4.1.2	Códigos Python Para Tratar Las Capturas.	69
3.4.2	<i>Iteración 2</i>	72
3.4.2.1	Entrenamiento De Detección De Áreas Con Yolov5.....	72
3.4.3	<i>Iteración 3</i>	75
3.4.3.1	Códigos Python Para Subsecciones Con Yolov5.....	75
3.4.3.2	Entrenamiento De Detección En Subáreas.....	79
3.4.4	<i>Iteración 4</i>	80
3.4.4.1	Dataset Para El Entrenamiento De La CRNN.	80
3.4.4.2	Entrenamiento De La CRNN.	82
3.4.5	<i>Iteración 5</i>	86
3.4.5.1	Corrección Mediante Diccionario De Palabras En Python.	86
3.4.5.2	Estructura De Notas Digitales Mediante Detección De Posiciones.	87
3.4.6	<i>Iteración 6</i>	88
3.4.6.1	Conversión De Modelos Yolov5 a TFLite para móviles.	88
3.4.6.2	Conversión De Modelos CRNN a PyTorch Mobile.....	91
3.5	FASE DE ESTABILIZACION	95
3.5.1	<i>Iteración 7</i>	95
3.5.1.1	Diseño frontend de la aplicación.....	95
3.5.1.2	Implementación Funcional e Integración de Componentes.	97
3.6	FASE DE PRUEBAS	108

3.6.1	<i>Validación de Requisitos.....</i>	108
3.6.2	<i>Pruebas de Funcionamiento</i>	109
3.6.3	<i>Prueba de Integración</i>	109
3.6.4	<i>Despliegue del instalador de la aplicación.....</i>	109
CAPITULO IV		110
4	EVALUACION DE RESULTADOS.....	110
4.1	METODOLOGIAS DE EVALUACION.....	110
4.1.1	<i>Selección De Estudio.</i>	110
4.1.2	<i>Descripción De La Aplicación Móvil.....</i>	110
4.1.3	<i>Proceso de Prueba.....</i>	111
4.1.3.1	Preparación de Datos.....	111
4.1.3.2	Cálculo de Chi Cuadrado de Bondad de Ajuste.....	114
4.1.3.3	Cálculo del Coeficiente Kappa de Cohen.....	118
CAPITULO V.....		125
5	CONCLUSIONES Y RECOMENDACIONES	125
5.1	CONCLUSIONES.....	125
5.2	RECOMENDACIONES.....	126
6	BIBLIOGRAFÍA.....	127
ANEXOS.....		131

INDICE DE FIGURAS

Figura 1 <i>Introducción a redes neuronales</i>	19
Figura 2 <i>Estructura simple de una red neuronal</i>	20
Figura 3 <i>Función de activación</i>	21
Figura 4 <i>Función de paso binario</i>	22
Figura 5 <i>Función de activación lineal</i>	23
Figura 6 <i>Función de activación Sigmoid/Logistic</i>	24
Figura 7 <i>Función Tanh</i>	25
Figura 8 <i>Función ReLu</i>	26
Figura 9 <i>Funcionamiento de una red neuronal convolucional</i>	29
Figura 10 <i>Operación de convolución</i>	30
Figura 11 <i>Capa de agrupación</i>	31
Figura 12 <i>Capa totalmente conectada y procesos de capas anteriores</i>	33
Figura 13 <i>Neurona recurrente</i>	37
Figura 14 <i>YoloV5 grafica de rendimiento</i>	45
Figura 15 <i>Metodología Mobile-D</i>	47
Figura 16 <i>Logo VsCode</i>	50
Figura 17 <i>Logo Colab</i>	50
Figura 18 <i>Logo Android Studio</i>	51
Figura 19 <i>Logo Python</i>	51
Figura 20 <i>Logo YoloV5</i>	52
Figura 21 <i>Logo TensorFlow</i>	52
Figura 22 <i>PyTorch</i>	53

Figura 23 Logo Kotlin.....	53
Figura 24 Logo GitHub.....	54
Figura 25 Diseño de aplicación.....	66
Figura 26 Notas de venta, recibos de venta	67
Figura 27 Notas de venta capturadas	68
Figura 28 Notas de venta generación de bundingbox para el dataset.....	69
Figura 29 boundingbox csv, áreas para dataset	70
Figura 30 Script conversión áreas para boundingbox.....	71
Figura 31 formato de dataset de las notas de ventas.....	72
Figura 32 Instalación YOLOV5 y requisitos	73
Figura 33 Valores de Entrenamiento	74
Figura 34 Detección con YoloV5 de nota de venta	74
Figura 35 Pesos de entrenamiento de YoloV5 áreas.....	75
Figura 36 Extraer Áreas Detectadas.....	76
Figura 37 Notas de venta detección de celdas del área detalle	77
Figura 38 Áreas de detalle en csv capturados	78
Figura 39 Formato de Dataset de Detalle	79
Figura 40 Detección de YoloV5 celdas detalle	79
Figura 41 Pesos de entrenamiento de área detalle	80
Figura 42 Captura de datos para el CRNN.....	81
Figura 43 Archivo Txt con datos capturados.....	81
Figura 44 Formato de dataset para entrenamiento CRNN.....	82
Figura 45 Instalar Librerías e Importaciones.....	83

Figura 46 Configuración inicial para el entrenamiento CRNN.....	84
Figura 47 Class CRNN arquitectura del modelo CRNN.....	85
Figura 48 Prueba de detección de entrenamiento de palabras en celda	85
Figura 49 Pesos de entrenamiento de la CRNN.....	86
Figura 50 Conversión Pt a SaveModel de TFLite.....	89
Figura 51 Conversión SaveModel TFLite a TFLite float16.....	90
Figura 52 Detección con TFLite Float16.....	90
Figura 53 Visualización de Detección y Confianza con TFLITE.....	90
Figura 54 Configuración de entrada dinámica a una estática	93
Figura 55 Verificación de conversión.....	94
Figura 56 Pruebas Frontend Pantalla Inicial.....	96
Figura 57 Frontend pantallas prueba	97
Figura 58 Fragmento View	98
Figura 59 Clase para filtros.....	99
Figura 60 Izquierda original, Derecha con Filtros.....	99
Figura 61 Clase de redimensión y Rotación	100
Figura 62 Clase Detector.kt	101
Figura 63 Clase DetectorD.kt	102
Figura 64 CLase CRNNModelUtils.kt.....	103
Figura 65 Clase DatabaseHelper.kt.....	105
Figura 66 ModelView.....	106
Figura 67 Deteccion y Recorte.....	107
Figura 68 Captura de aplicación de detección CRNN.....	107

Figura 69 <i>Instalador APK</i>	109
Figura 70 <i>Matriz de confusión</i>	111
Figura 71 <i>Gráfico de distribución de error de chi cuadrado por carácter.</i>	117

INDICE DE TABLAS

Tabla 1 <i>Capital por Categorías RTS</i>	15
Tabla 2 <i>Diseño Mobile-D</i>	63
Tabla 3 <i>Problemas en adaptación CRNN a PT PyTorch</i>	92
Tabla 4 <i>Validación de Requisitos</i>	108
Tabla 5 <i>Tabla generada atreves de la matriz de confusión</i>	112
Tabla 6 <i>Tabla de cálculos para chi cuadrado</i>	114
Tabla 7 <i>Tabla de la Matriz de Confusión Valores Para Cálculos Kappa Cohen</i>	118
Tabla 8 <i>Tabla de cálculos de P_e</i>	121
Tabla 9 <i>Interpretación del valor de Kappa</i>	123

CAPITULO I

1 MARCO REFERENCIAL

1.1 INTRODUCCIÓN

En la actualidad, la transformación digital avanza rápidamente en todos los sectores, incluyendo el comercio. Sin embargo, en países como Bolivia, muchos pequeños comerciantes, especialmente aquellos que pertenecen al Régimen Tributario Simplificado (RTS), continúan utilizando métodos tradicionales para registrar sus ventas, como el uso de notas de venta manuscritas. Esta práctica limita su capacidad de organización, seguimiento de ingresos y acceso a servicios financieros o tecnológicos.

En Bolivia el Padrón Nacional de Contribuyentes Biométrico Digital (PBD) en la gestión 2021 reporta una cantidad final de 442.133 contribuyentes activos y habilitados, siendo el 24% perteneciente al Régimen Tributario Simplificado, correspondiente a comerciantes minoristas, artesanos y vivanderos. Boletín Económico Ingresos Tributarios, 2021, Bolivia, (economiyfinanzas.gob.bo portal).

Este régimen facilita las operaciones de pequeños negocios al eximirlos de la obligación de emisión de facturas fiscales. Sin embargo, también presenta limitaciones significativas en la gestión administrativa, el control y análisis de ventas. Muchos de estos contribuyentes suelen depender de registros manuales o notas de venta en talonarios, lo que dificulta la elaboración de balances contables, análisis de ventas y la planificación estratégica, limitando la transparencia operativa y dificultando el acceso a servicios financieros. Además, los clientes no cuentan con comprobantes digitales, lo que limita la transparencia en las transacciones.

Ante esta problemática, se propone el desarrollo de una aplicación móvil Android, capaz de digitalizar notas de venta manuscritas mediante el uso de visión por computadora y redes neuronales, específicamente una combinación de modelos YOLO¹ y CRNN². El objetivo principal es transformar automáticamente una imagen de una nota de venta en un recibo digital estructurado, que permita al comerciante registrar, consultar, extender sus ventas de manera rápida, sin necesidad de conocimientos técnicos avanzados ni conexión a internet. Este proyecto busca no solo modernizar procesos, sino también fomentar la inclusión tecnológica al promover el desarrollo en un sector esencial en la economía boliviana.

A lo largo de este documento, los términos “recibo”, “nota de venta” y “proforma” se emplean de forma intercambiable. En el contexto del RTS³ en Bolivia, estos documentos no siempre corresponden a facturas fiscales, pero cumplen la función de comprobante de venta, registro y respaldo de una transacción. Por razones de concisión, se optó por usar el término “recibo” en el título del trabajo, aunque la mayor parte de los datos y ejemplos provienen de notas de venta manuscritas utilizadas en comercios informales.

La primera parte del trabajo utilizará la metodología Mobile-D⁴ y consistirá en la recopilación de notas de venta de diferentes negocios para construir un dataset⁵ amplio. Luego, este dataset se usará para entrenar modelos de detección de áreas de interés en las notas

¹ Yolo es un modelo de visión artificial para la detección de objetos.

² CRNN Convolutional Recurrent Neural Networks es la combinación de dos tipos de redes neuronales, CNN (convolutional neural network) y RNN (Recurrent neural networks).

³ RTS siglas de Régimen Tributario Simplificado

⁴ Mobile-D es una metodología de desarrollo de software ágil, específicamente diseñada para el desarrollo de aplicaciones móviles.

⁵ El dataset es un conjunto de datos estructurados que se utiliza para entrenar, validar y probar una red neuronal.

manuscritas. Posteriormente, se entrenará una red CRNN para el reconocimiento de texto manuscrito. Finalmente, se procederá al diseño e implementación de la aplicación móvil que integrará ambos procesos.

1.2 ANTECEDENTES

1.2.1 Antecedentes internacionales.

Análisis en modelos de atención, tesis de grado, Escuela Técnica Superior de Ingeniería Universidad de Sevilla.

El proyecto presenta una solución basada en modelos de atención (Bahdanau, Loung, Monotonic). De esta manera se propone que la dependencia de la transcripción realizada de los caracteres anteriores y posteriores al que ocupa en cada momento puede aportar claridad y exactitud a la transcripción (García González, 2020).

Aplicación de conversión de texto manuscrito. Tesis de maestría, Pontificia Universidad Católica Del Perú.

Se diseña una aplicación que hace uso de HTR⁶ y redes neuronales convolucionales, para la detección y reconocimiento de texto manuscrito históricos peruanos utilizando modelos mixtos, se utilizó diferentes modelos previamente entrenados y se modificaron para detectar los manuscritos y se aplicó el que mejor resultado tuviera en la predicción y se aplicó al resto de los manuscritos peruanos históricos (Cruz, 2024) .

⁶ HTR siglas de Handwriting Text Recognition que significan reconocimiento de texto manuscrito.

1.1.1 Antecedentes nacionales.

Aplicación de traducción chino a español, Tesis De Grado, Universidad Mayor De San Andrés.

Se diseña una aplicación en ordenar que utiliza autómatas finitos determinísticos que hacen uso de OCR⁷ para la traducción de chino al español (Garcia, 2015).

El reconocimiento óptico de caracteres (OCR) una herramienta clave para la digitalización, según IBM⁸, “El software OCR puede aprovechar la inteligencia artificial (IA⁹) para implementar métodos más avanzados de reconocimiento inteligente de caracteres (ICR¹⁰) para identificar idiomas o escritura a mano” (IBM International Business Machines Corporation, s.f.).

En 1974, Ray Kurzweil fundó Kurzweil Computer Products, Inc., cuyo producto OCR omnicanal podía reconocer texto impreso en prácticamente cualquier fuente (IBM International Business Machines Corporation, s.f.). Con el tiempo, esta tecnología experimentó un avance exponencial, extendiéndose hacia nuevas áreas como el reconocimiento de texto manuscrito (HTR).

⁷ OCR (Optical Character Recognition) es una tecnología que convierte imágenes de texto, como documentos escaneados o fotografías, en texto digital editable y que puede ser buscado.

⁸ IBM que significa International Business Machines, es una empresa multinacional de tecnología y consultoría.

⁹ IA siglas de inteligencia artificial

¹⁰ ICR (optical character recognition), reconocimiento inteligente de caracteres es una versión avanzada de reconocimiento de escritura a mano

Utilizando HTR (handwritten text recognize) reconocimiento de texto manuscrito, se aprovechó el uso de las redes neuronales convolucionales (CNN) para el reconocimiento de dígitos manuscritos. Desarrollando LeNet-5, un modelo que revolucionó el reconocimiento de caracteres. Utilizaron el conjunto de datos MNIST como base para sus investigaciones (LeCun, Bottou, Bengio, & Haffner, 1998).

Avances más posteriores realizados en su trabajo, Bluche demostró que las redes neuronales profundas podían superar significativamente los métodos tradicionales de reconocimiento de texto manuscrito, reduciendo las tasas de error de manera considerable (Bluche, 2015).

Estos avances no solo se han limitado a avances académicos, sino que también se han ampliado en diversas áreas, educación, logística, servicios administrativos, comercio, entre otros.

El reconocimiento de texto manuscrito, sigue presentando desafíos, la gran variedad de estilos de escritura, el grosor del trazo y el papel usado, así también el ruido en las imágenes, dificultando las tareas de reconocimiento. Sin embargo, modelos más recientes y el aprendizaje de transferencia, han abierto nuevas posibilidades ante estos desafíos, permitiendo entrenar sistemas más robustos y precisos, con la posibilidad de adaptarse a los diferentes tipos de escritura.

1.3 PLANTEAMIENTO DEL PROBLEMA

1.3.1 Problema General.

Los pequeños negocios acogidos al Régimen Tributario Simplificado (RTS) representan el 24% del total de contribuyentes activos en Bolivia. Aunque este régimen les otorga beneficios

significativos, como estar exentos de la emisión de facturas, también representan limitaciones importantes que afectan su gestión administrativa y financiera. Al no contar con herramientas tecnológicas accesibles y adecuadas para sus necesidades, estos negocios se enfrentan a una dependencia casi total de registros manuales como recibos o notas de venta. Esta situación incrementa la probabilidad de errores contables, dificulta el análisis de ventas y limita la generación balances financieros precisos.

La ausencia de un sistema contable automatizado no solo afecta la eficiencia operativa y tiempo, sino también la transparencia de sus transacciones. Los clientes no reciben comprobantes digitales, lo que impacta negativamente en la confianza y profesionalización de estos pequeños comerciantes. Además, esta carencia de registros formales dificulta el acceso a servicios financieros, ya que las entidades bancarias suelen requerir reportes de ventas confiables como parte de la evaluación para el crédito solicitado. Al no poder acceder a servicios financieros formales, muchos pequeños negocios se ven forzados a recurrir a prestamistas informales, exponiéndolos a tasas de interés elevadas y condiciones poco favorables. En consecuencia, los comerciantes del RTS, con recursos limitados, enfrentan barreras significativas para acceder a financiamiento, comprometiendo así su capacidad para crecer y desarrollarse en un entorno competitivo.

La falta de digitalización en sus operaciones no solo limita su sostenibilidad a largo plazo, sino que también los excluye de los beneficios que conlleva la adopción de tecnologías modernas, como el análisis de datos. Este problema no solo afecta a los comerciantes, sino que también tiene un impacto en la economía local, al frenar el desarrollo de un sector esencial en la actividad económica del país.

1.3.2 Problemas Específicos.

- No se puede realizar un análisis de ventas rápido.
- No es posible realizar balances contables del negocio sin usar documentos a mano.
- No existe comprobantes digitales para el cliente.
- No existe comprobantes, ni datos de ventas para solicitar financiamiento.
- Falta de indicadores de ventas por etapas o fechas.

1.4 OBJETIVOS.

1.4.1 Objetivo General.

Diseñar una aplicación móvil ligera, distribuida bajo una licencia GPL¹¹, basada en Deep Learning¹² con tecnología de reconocimiento de texto manuscrito (Handwritten Text Recognition, HTR), para los contribuyentes acogidos al régimen RTS, que permita la digitalización de recibos manuscritos, que sirva para un control de ventas y la generación de recibos digitales para el cliente. Lo que facilitaría un análisis en sus ventas y coadyuve en la toma de decisiones futuras.

1.4.2 Objetivo Específico.

- Diseñar de un modelo de reconocimiento de texto manuscrito (HTR¹³).

¹¹ GPL (GNU Licencia Pública General) es una licencia de software libre.

¹² Deep Learning significa aprendizaje profundo, es un subconjunto del Machine Learning que utiliza redes neuronales artificiales con múltiples capas.

¹³ HTR siglas de Handwriting Text Recognition que significan reconocimiento de texto manuscrito.

- Diseñar de funcionalidad para la generación de recibos digitales.
- Recolectar data set de texto manuscrito para el entrenamiento de la red neuronal.
- Transferir el entrenamiento de la red a una para ser utilizada en el móvil.
- Almacenar los datos en una base de datos para ser utilizadas en análisis de venta y control.
- Desarrollar de la aplicación móvil amigable e intuitiva.

1.5 HIPÓTESIS

Se plantea que el desarrollo e implementación de una aplicación móvil basada en técnicas de Deep Learning y reconocimiento de texto manuscrito (HTR), aplicada al contexto de los contribuyentes del Régimen Tributario Simplificado, será capaz de digitalizar notas de venta manuscritas con una precisión superior al 70%, generando recibos digitales estructurados que pueden ser extendidos a los clientes cuando se requiera, y permitiendo el almacenamiento de datos para facilitar un análisis básico de ventas.

1.6 OPERACIÓN DE VARIABLES

Variable dependiente: Aplicación móvil, resultante del proceso de diseño e implementación.

Variable independiente: Proceso de detección y digitalización de notas manuscritas e implementación de una aplicación de reconocimiento de texto manuscrito.

Variable interviniente: Rendimiento y Algoritmos de Deep Learning utilizados para el reconocimiento de texto manuscrito.

1.7 JUSTIFICACIÓN.

1.7.1 Justificación Social.

Con el desarrollo de la aplicación móvil con licencia GPL garantiza que se fomentará la inclusión tecnológica, promoviendo análisis en sus ventas y facilitando en el acceso a financiamiento. Al permitir el acceso abierto, pequeños comerciantes y comunidades con recursos limitados pueden beneficiarse de la herramienta sin preocuparse por costos de licencias privativas, fomentando una inclusión accesible a nuevas tecnologías.

1.7.2 Justificación Económica.

La viabilidad económica se ha convertido en un factor predominante en el desarrollo de todo trabajo de investigación.

Al facilitar un control más eficiente de ventas y análisis, puede mejorar la gestión financiera de los pequeños negocios que pertenecen al régimen simplificado (RTS), esto incrementa la productividad, reduce las pérdidas por errores contables, favoreciendo a una mejor toma de decisiones, estabilidad en estos negocios y mayor acceso a créditos formales y una mejora de sostenibilidad económica.

1.7.3 Justificación Académica.

El proyecto contribuye al conocimiento en áreas como las redes neuronales, reconocimiento de texto manuscrito y aplicaciones móviles. Aportando una investigación aplicada. El uso de una licencia GPL en el desarrollo de esta aplicación tiene como objetivo fomentar y permitir a otros desarrolladores e investigadores puedan utilizar, modificar y

redistribuir el dataset y código de manera libre, esto permite a instituciones académicas, sectores públicos adoptar y adaptar esta tecnología sin costos adicionales.

1.8 LIMITES Y ALCANCES

1.8.1 Alcance.

La presente investigación contempla los siguientes alcances:

- Diseñar una aplicación móvil basado en Deep learning y HTR.
- Diseñar e implementar una tecnología innovadora para comerciantes minoristas que pertenecen al régimen simplificado de Bolivia.
- Utilizar los recursos en Deep Learning, para la digitalización de recibos manuscritos.
- Generar recibos digitales.
- Almacenamiento de datos para análisis de ventas.
- Conversión de redes neuronales entrenadas previamente para garantizar eficiencia en dispositivos móviles.

1.8.2 Limites.

- La implementación está orientada únicamente al contexto boliviano, específicamente la paz Bolivia para negociantes en RTS.
- La aplicación está limitada a reconocer textos manuscritos hechos en recibos o notas de ventas.
- La aplicación se limita a escritura clara y legible. Escrituras excesivamente desordenadas o ilegibles pueden no ser reconocidas con precisión.

- La eficacia de la aplicación dependerá de la calidad de la cámara de celular.
- La aplicación funcione offline, la sincronización y exportación de datos no podrían estar presentes.
- El almacenamiento de datos está limitado a la capacidad del dispositivo móvil.
- La eficiencia del reconocimiento de texto puede variar dependiendo de factores externos: ambiente, iluminación, color de texto y el tamaño de trazo en la escritura.
- La implementación puede enfrentar resistencia por parte de comerciantes que prefieren métodos tradicionales.

1.9 METODOLOGIAS

La metodología de esta tesis se enfoca en el diseño, desarrollo y validación técnica de una aplicación móvil para la digitalización de recibos escritos a mano, utilizando técnicas de Deep Learning y HTR. Se empleará el enfoque aplicado y experimental, acompañado de la metodología ágil Mobile-D para guiar el proceso iterativo de desarrollo de la aplicación.

1.9.1 Metodología De Desarrollo.

Para el desarrollo del presente trabajo, se utilizará la metodología de desarrollo Mobile-D, es una metodología ágil, ya que es la más adecuada cuando se trata de plataformas móviles. Además, que permite responder rápidamente a los cambios que se puedan producir durante la etapa de desarrollo del proyecto, permitiendo la reducción de tiempos de producción (Amaya Balaguera, 2013).

El desarrollo de la aplicación seguirá principios de software libre, publicando todo el código fuente bajo una licencia GPL. Esto asegura que cualquier persona pueda examinar,

modificar y distribuir el software, promoviendo la transparencia y la posibilidad de mejoras continuas por parte de la comunidad.

1.9.2 Metodología De La Investigación Científica.

La metodología científica, proporciona un conjunto de pasos y reglas lógicas por medio de las cuales es posible realizar el planteamiento de problemas y formular una hipótesis.

- **Observación:** Consiste en examinar atentamente los hechos y fenómenos que tiene lugar en la naturaleza y que pueden ser percibidos por los sentidos. La observación debe ser cuidadosa, exhaustiva y exacta.
- **Identificación de problema:** Después de la observación se plantea el cómo y porque del fenómeno observado, para la identificación del problema que se va estudiar.
- **Hipótesis:** Formular una hipótesis consiste en elaborar una explicación provisional de los hechos observados y de sus posibles causas. Es otras palabras es la solución preliminar ante el problema planteado, es una declaración que puede ser falsa o verdadera.
- **Experimentación:** Se realizan pruebas modificando la variable que intervienen en el proceso y comprobara si se cumple la hipótesis.
- **Resultados:** El análisis de los datos experimentales permite comprobar si la hipótesis era correcta y dar una explicación científica al hecho o fenómeno observado.

CAPITULO II

2 MARCO TEORICO

2.1 INTRODUCCION

En este capítulo presente se usarán los conceptos, métodos, herramientas que proporcionan las tecnologías actuales.

Las tecnologías móviles son consideradas un sector de crecimiento constante, lo que ha impulsado grandes innovaciones tecnológicas centradas en la telefonía móvil.

Hoy en día, El teléfono móvil, es un complemento cotidiano e indispensable por el amplio espectro de usos: escuchar música, enviar y recibir mensajes, llamadas, navegar en internet, entre otros.

A medida que avanza las tecnologías estas avanzan, estas se adaptan para cubrir muchos aspectos de la vida cotidiana, incluyendo diferentes áreas, tratando de integrar una parte de la sociedad que, por sus condiciones económicas, no siempre tienen tecnologías, un claro ejemplo de ellos es el RTS¹⁴, que podrían beneficiarse del uso de herramientas digitales adaptadas a sus necesidades, contribuyendo significativamente a la inclusión tecnológica, facilitando sus operaciones y promoviendo una gestión más eficiente de sus negocios.

¹⁴ RTS siglas de Régimen Tributario Simplificado.

2.2 REGIMEN TRIBUTARIO SIMPLIFICADO RTS

El Régimen Tributario Simplificado (RTS), fue incorporado a la norma tributaria mediante el Decreto Supremo N°21521 de 13 de febrero de 1987, La última modificación fue mediante el Decreto Supremo N°3698 en fecha 25 de octubre del 2018. Característica capital mínimo a 60000 Bs. como se ve en la Tabla 1, y están exentos de emitir facturas fiscales.

2.2.1 Actividades comprendidas en el RTS

Comerciantes minoristas, son comerciantes minoristas, a los efectos de este régimen, las personas naturales que desarrollan actividades de compra venta de mercaderías o prestación de servicios en mercados públicos, ferias, kioscos, pequeñas tiendas y puestos ubicados en la vía pública y que cumplan con los requisitos del numeral 1 del Artículo 3 del Decreto Supremo N° 24484.

Artesanos, para los efectos de este régimen son artesanos, las personas naturales que ejercen un arte u oficio manual, trabajando por cuenta propia en sus talleres o a domicilio, percibiendo por su trabajo una remuneración de terceros en calidad de clientes y que cumplan los requisitos del numeral 2 del artículo 3 del Decreto Supremo N° 24484.

Vivanderos, para los efectos de este régimen son vivanderos las personas naturales que expenden comidas, bebidas y alimentos en kioscos y pequeños locales y que cumplan los requisitos del numeral 1 del artículo 3 del Decreto Supremo N° 24484.

Tabla 1*Capital por Categorías RTS*

CATEGORÍA	CAPITAL		PAGO BIMESTRAL
1	12.001	15.000	47
2	15.001	18.700	90
3	18.701	23.500	147
4	23.501	29.500	158
5	29.501	37.000	200
6	37.001	60.000	350

2.3 INTELIGENCIA ARTIFICIAL (IA)

La inteligencia artificial tiene por objeto que los ordenadores hagan la misma clase de cosas que puede hacer la mente (Boden, 2017).

Tipos de inteligencia artificial:

Según su funcionalidad o capacidad

- Sistema de IA reactiva (Reactive AI): sistemas que reaccionan a estímulos del entorno, pero no son capaces de recordad ni usar experiencias pasadas para tomar decisiones, ejemplo: Deep Blue, computadora que gano al campeón de ajedrez Garry Kasparov 1997.
- IA con memoria limitada (Limited Memory AI): sistemas que usan datos históricos para tomar decisiones, ejemplo: vehículos autónomos.
- Teoría de la mente (Theory of mind AI): tipo de IA que busca comprender emociones, intenciones y pensamientos de los seres humanos, aun en desarrollo.

- IA autoconsciente (self-aware AI): tipo de IA hipotético que tendría el mismo nivel de conciencia humana, con emociones y estados mentales.

Según su nivel de autonomía

- IA débil (Narrow AI o Weak AI): diseñada para realizar tareas específicas y no tiene conciencia ni capacidad para generalizar conocimientos. Ejemplo: asistentes virtuales como Siri, motores de recomendaciones como Netflix.
- IA fuerte (General AI): es una IA capaz de realizar tareas intelectuales que un ser humano puede hacer, aun no existente.
- Superinteligencia: IA que supera la inteligencia humana en todos los aspectos, incluyendo creatividad, resolución de problemas y toma de decisiones, aun siendo un concepto teórico.

Según su enfoque técnico

- Machine learning (ML): sistemas que aprenden de datos sin ser programados explícitamente, incluyen técnicas como el aprendizaje supervisado, no supervisado y por refuerzo.
- Deep learning: subconjunto del machine learning que utiliza redes neuronales artificiales con multicapa, llamada redes neuronales profundas.
- Natural language processing (NLP): enfocado en la interacción entre humanos y máquinas a través del lenguaje.
- Computer visión: permite a las máquinas interpretar y entender imágenes y videos.

2.4 MACHINE LEARNING

Aprendizaje automático (machine learning), la máquina se ve en los datos de entrada y las respuestas correspondientes, y descubra cuáles son las reglas debe ser. Un sistema de aprendizaje automático está capacitado en lugar de explícitamente programado. Se presenta muchos ejemplos relevantes para una tarea, y encuentra una estructura estadística en estos ejemplos que eventualmente permite que el sistema cree reglas para automatizar la tarea (Chollet, 2021).

El aprendizaje automático es una subparte de la inteligencia artificial que analizan grandes cantidades de datos estructurados e identifican patrones y correlacionan datos.

El aprendizaje automático se divide en tres categorías principales:

- Aprendizaje supervisado. - se define por su uso de conjuntos de datos etiquetados para entrenar algoritmos que clasifiquen datos o predigan resultados con precisión. A medida que se introducen datos de entrada en el modelo, éste ajusta sus ponderaciones hasta que se ha ajustado adecuadamente. Esto ocurre como parte del proceso de validación cruzada para garantizar que el modelo evite el sobreajuste (IBM International Business Machines Corporation, s.f.).
- Aprendizaje no supervisado. - también conocido como machine learning no supervisado, utiliza algoritmos de machine learning para analizar y agrupar conjuntos de datos no etiquetados (subconjuntos denominados clústeres). Estos algoritmos descubren patrones ocultos o agrupaciones de datos sin necesidad de intervención humana. La capacidad de este método para descubrir similitudes y

diferencias en la información lo hace ideal para el análisis exploratorio de datos, las estrategias de venta cruzada, la segmentación de clientes y el reconocimiento de imágenes y patrones. También se utiliza para reducir el número de características de un modelo mediante el proceso de reducción de la dimensionalidad. El análisis de componentes principales (PCA) y la descomposición en valores singulares (DVE) son dos métodos habituales para ello. Otros algoritmos utilizados en el aprendizaje no supervisado son las redes neuronales, el k-medias y los métodos de agrupación probabilística (IBM International Business Machines Corporation, s.f.).

- Aprendizaje por refuerzo. - es un modelo de machine learning similar al aprendizaje supervisado, pero el algoritmo no se entrena con datos de ejemplo. Este modelo aprende sobre la marcha mediante el método de ensayo y error (IBM International Business Machines Corporation, s.f.).

2.5 DEEP LEARNING

El deep learning es un subconjunto del machine learning que utiliza redes neuronales multicapa, llamadas redes neuronales profundas, para simular el complejo poder de toma de decisiones del cerebro humano (IBM International Business Machines Corporation, s.f.).

Para diferenciar entre el Deep Learning y el machine Learning es la estructura de la arquitectura de red neuronal subyacente. Los modelos tradicionales de machine Learning “no profundos (Deep Learning)” utilizan redes neuronales simples con una o dos capas computacionales. Mientras que los modelos de aprendizaje supervisado requieren datos de entrada estructurados y etiquetados para obtener resultados precisos, los modelos de deep

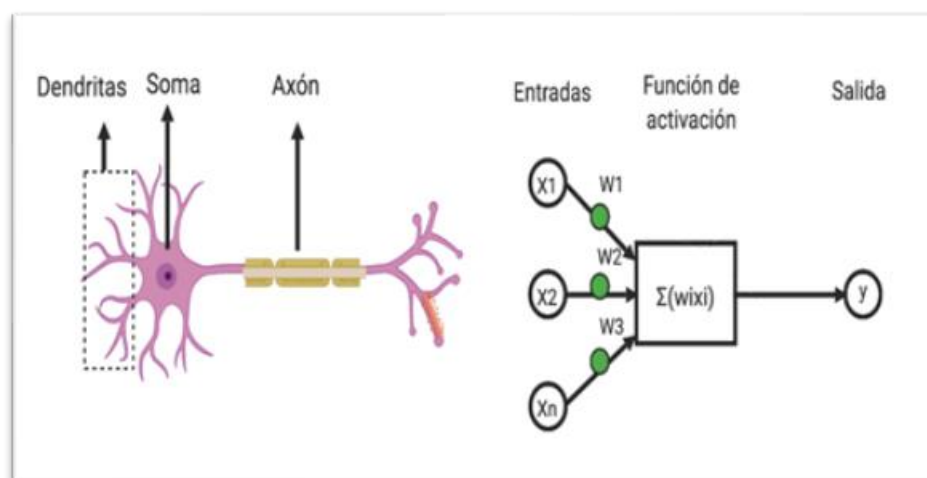
learning pueden utilizar el aprendizaje no supervisado. Con el aprendizaje no supervisado, los modelos pueden extraer las características, los rasgos y las relaciones que necesitan para obtener resultados precisos a partir de datos brutos y no estructurados. Además, estos modelos pueden incluso evaluar y refinar sus resultados para aumentar la precisión.

2.5.1 Redes Neuronales.

¿Qué es una neurona artificial? Dentro del campo de la Inteligencia Artificial son métodos de aprendizaje automático cuya finalidad es imitar los procesos biológicos de las redes neuronales de los organismos vivos (Hernández Orallo, Ramírez Quintana, & Ferri Ramírez, 2004).

Figura 1

Introducción a redes neuronales



Nota. – Comparación de una neurona biológica y red neuronal, adaptado de Neurona vs.

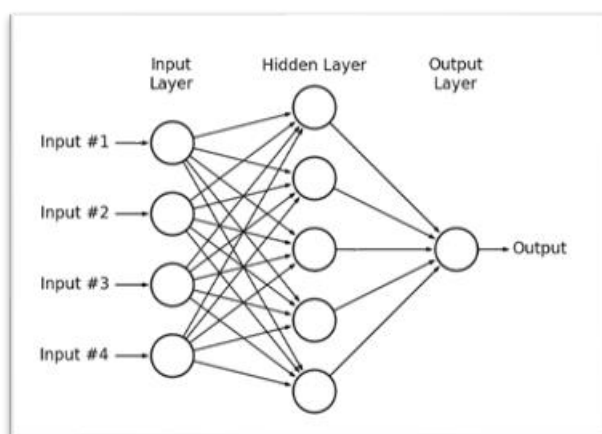
Perceptrón [fotografía], por Ulises García, 2019. Future Lab.

(<https://futurelab.mx/redes%20neuronales/inteligencia%20artificial/2019/06/25/intro-a-redes-neuronales-pt-1/>).

Hay tres partes normalmente en una red neuronal Figura 1: una capa de entrada, con unidades que representan los campos de entrada; una o varias capas ocultas; y una capa de salida, con una unidad o unidades que representa el campo o los campos de destino. Las unidades se conectan con fuerzas de conexión variables (o ponderaciones). Los datos de entrada se presentan en la primera capa, y los valores se propagan desde cada neurona hasta cada neurona de la capa siguiente. al final, se envía un resultado desde la capa de salida como se ve en Figura 2.

Figura 2

Estructura simple de una red neuronal



Nota. – Perceptrón multicapa (MPL), adaptado de CNN a Python [imagen], por Luciano Strika, 2019, kdnuggets (<https://www.kdnuggets.com/2019/07/convolutional-neural-networks-python-tutorial-tensorflow-keras.html>).

Como se ve en la Figura 1, puede formularse matemáticamente a la neurona básica de la siguiente manera: se cuenta con n entradas x_i , en cada entrada es multiplicada por un peso w_i , si existen n entradas se contará con n pesos, para finalmente calcular la suma total, se muestra en la ecuación (1).

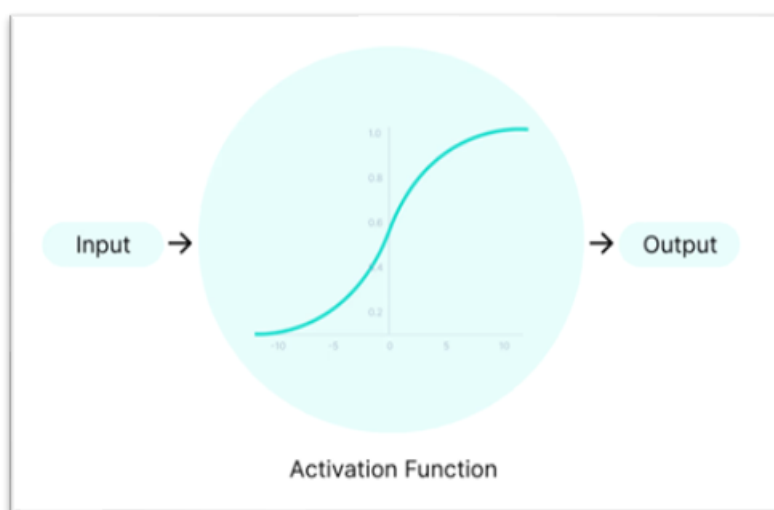
$$suma\ total = \sum_{i=0}^n w_i * x \quad (1)$$

2.5.2 Funciones De Activación.

La función de activación es aquella que define la salida de una neurona como se ve en la Figura 3, si existe un estado de activación o no, es por ello que existen una variedad de funciones con distintas finalidades, a continuación, se presentan las funciones que son utilizadas con mayor frecuencia en los diversos problemas (Baheti, 2021).

Figura 3

Función de activación



Nota. – Referencia de una función de activación. Adaptado de Activation Functions in Neural Networks [Imagen], por Pragati Baheti, 2021, v7 labs (<https://www.v7labs.com/blog/neural-networks-activation-functions>).

- Función de escalón o Función umbral de paso binario. - la función de escalón, es una de las funciones de activación más simples utilizadas en redes neuronales. Su

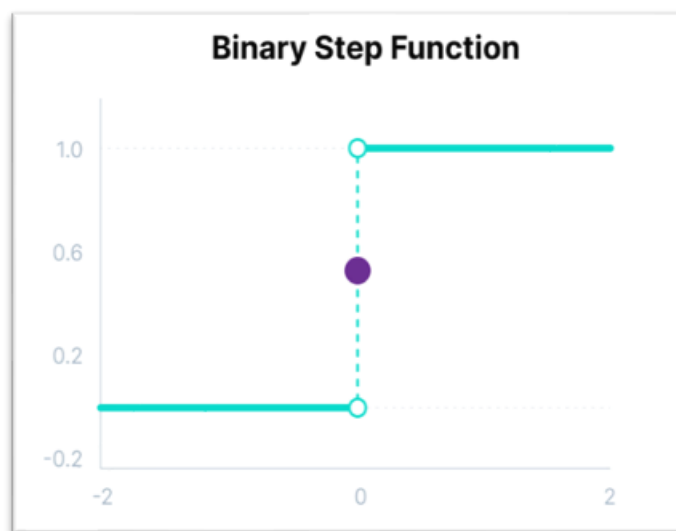
propósito principal es transformar la entrada de un nodo en una salida binaria, lo que permite tomar decisiones claras del tipo “sí o no”, “activado o desactivado”, o “verdadero o falso”.

Esta función compara el valor de entrada con un umbral previamente definido, si el valor de entrada es mayor o igual, la salida será 1, de lo contrario la salida será 0, como podemos observar en la Figura 4 y matemáticamente expresarse como se muestra en la ecuación (2).

$$f(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases} \quad (2)$$

Figura 4

Función de paso binario



Nota. - Función de paso binario que depende de un valor umbral con la cual la neurona decide activarse o no. Adaptado de Activation Functions in Neural Networks [Imagen], por Pragati Baheti, 2021, v7 labs (<https://www.v7labs.com/blog/neural-networks-activation-functions>).

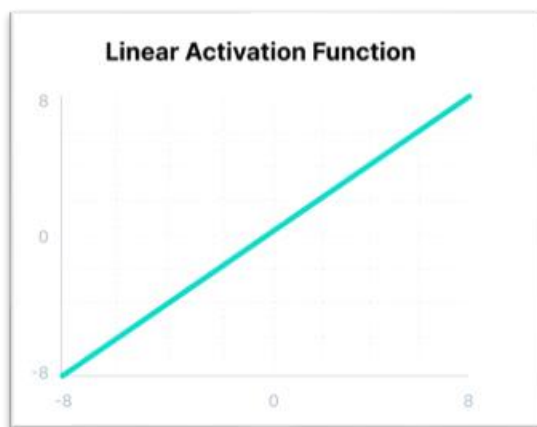
- **Función lineal.** – La función lineal es una de las funciones de activación más simples en redes neuronales. Su comportamiento consiste en devolver la misma salida que la entrada sin aplicar ningún tipo de transformación no lineal. En otras palabras, la activación neuronal es directamente proporcional al valor de entrada. Esto puede observarse en la Figura 5 y se expresa matemáticamente en la Ecuación (3).

Esta función es útil en modelos donde se requiere una salida continua, como en problemas de regresión. Sin embargo, su limitación principal es que no permite introducir no linealidad en la red, por lo que no es adecuada para resolver problemas complejos que requieren aprendizaje de patrones no lineales.

$$f(x) = x \quad (3)$$

Figura 5

Función de activación lineal



Nota. – Función de activación también conocida como no activación o función de identidad.

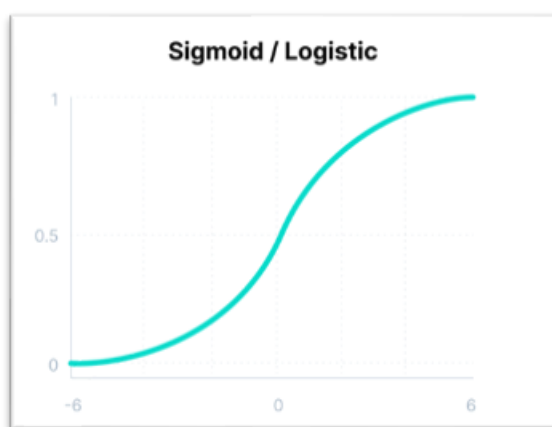
Adaptado de Activation Functions in Neural Networks [Imagen], por Pragati Baheti, 2021, v7 labs (<https://www.v7labs.com/blog/neural-networks-activation-functions>).

- **Función sigmoidea o Logística.** - La función es ampliamente utilizada en redes neuronales, especialmente en modelos donde se requiere una interpretación probabilística de la salida. Esta función presenta una curva en forma de "S" suave, como se muestra en la Figura 6, y su expresión matemática está representada en la Ecuación (4). Su valor de salida se encuentra en el rango de 0 a 1, aproximándose a 1 a medida que la entrada crece, y acercándose a 0 a medida que la entrada decrece. Esta propiedad la hace útil en tareas de clasificación binaria. Sin embargo, tiene desventajas como la saturación en sus extremos y el problema del desvanecimiento del gradiente, lo que puede dificultar el entrenamiento de redes profundas.

$$f(x) = \frac{1}{1 + e^{-x}} \quad (4)$$

Figura 6

Función de activación Sigmoid/Logistic



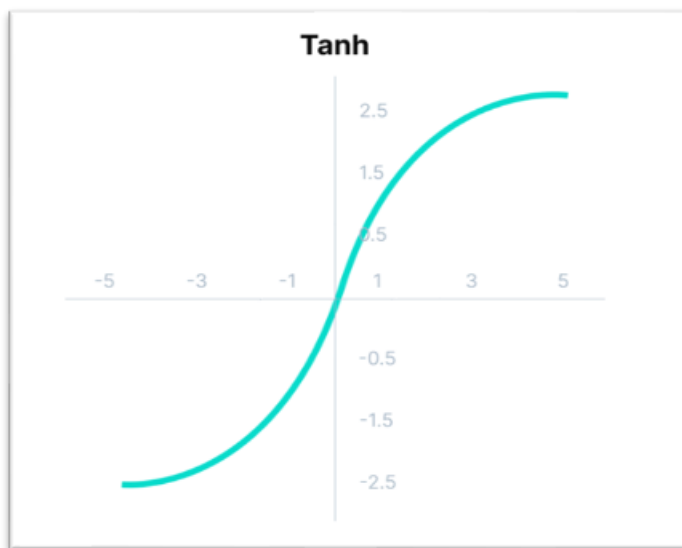
Nota. – Función de activación no lineal sigmoid/logistic. Adaptado de Activation Functions in Neural Networks [Imagen], por Pragati Baheti, 2021, v7 labs (<https://www.v7labs.com/blog/neural-networks-activation-functions>).

- Función de tangente hiperbólica (tanh). - La función de tangente hiperbólica, representada en la Figura 7 y en la Ecuación (5), también tiene forma de “S”, pero a diferencia de la función sigmoide, su rango de salida esta va de -1 a 1. Esta característica le otorga una ventaja significativa al estar centrada en cero, lo que facilita el aprendizaje y la convergencia en modelos entrenados mediante retro propagación, mejorando así el desempeño en comparación con la función sigmoide en algunos escenarios de aprendizaje profundo.

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (5)$$

Figura 7

Función Tanh



Nota. - Funcion Tanh similar a la función de activación sigmoid/logistic con la diferencia en el rango de salida que admite -1 a 1. Adaptado de Activation Functions in Neural Networks [Imagen], por Pragati Baheti, 2021, v7 labs (<https://www.v7labs.com/blog/neural-networks-activation-functions>).

- Función ReLU (Rectified Linear Unit). – es una de las funciones de activación más utilizadas en redes neuronales, especialmente en redes convolucionales. Como se muestra en la Figura 8 y en la Ecuación (6), esta función devuelve el mismo valor de entrada si este es positivo, y 0 si es negativo, introduce no linealidad sin saturación en el rango positivo y permite una retropropagación¹⁵ eficiente gracias a su derivada simple. Además, al no activar todas las neuronas simultáneamente, reduce el riesgo de co-adaptación entre nodos. Sin embargo, puede sufrir del problema de neuronas muertas¹⁶, por lo que se han desarrollado variantes como Leaky ReLU o ReLU Paramétrica para mitigar esta limitación.

$$f(x) = \max(0, x) \quad (6)$$

Figura 8

Función ReLu



Nota. - Adaptado de Activation Functions in Neural Networks [Imagen], por Pragati Baheti, 2021, v7 labs (<https://www.v7labs.com/blog/neural-networks-activation-functions>).

¹⁵ La retropropagación es un método elegante para calcular cómo los cambios en cualquiera de las ponderaciones o sesgos de una red neuronal afectarán a la precisión de las predicciones del modelo.

¹⁶ Neuronas muertas es cuando muchas activaciones son cero

- **Función softmax.** - La función softmax se utiliza principalmente en la capa de salida de redes neuronales destinadas a tareas de clasificación multiclase. Esta función transforma un vector de valores en un vector de probabilidades, donde cada elemento representa la probabilidad¹⁷ de pertenencia a una clase específica. A diferencia de la función sigmoide, que retorna una única probabilidad, la función softmax devuelve una distribución de probabilidad sobre múltiples clases, haciendo que la suma total de todas las salidas sea igual a 1. Su expresión matemática puede observarse en la Ecuación (7). Gracias a esta propiedad, es ampliamente utilizada en combinación con funciones de pérdida como la entropía cruzada para problemas de clasificación supervisada.

$$s(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}} \quad (7)$$

2.5.3 Redes Neuronales Convolucionales (CNN).

Las redes neuronales convolucionales, conocidas como Convolutional Neural Networks (CNN o ConvNets), constituyen un tipo especializado de red neuronal artificial ampliamente utilizado en el campo del deep learning. Estas redes son particularmente efectivas para procesar datos con estructura espacial tridimensional, como imágenes (ancho, alto, canales), y han demostrado un rendimiento superior en tareas como clasificación de imágenes, reconocimiento

¹⁷ Probabilidad cálculo matemático que establece todas las posibilidades que existen de que ocurra un fenómeno en determinadas circunstancias de azar.

de objetos, procesamiento de voz y análisis de audio (IBM International Business Machines Corporation, s.f.).

A diferencia de las redes neuronales tradicionales, las CNN se componen de una arquitectura jerárquica de capas que permite extraer características relevantes de manera automática y progresiva. Sus componentes principales son:

2.5.3.1 Capa Convolutiva (*Convolutional Layer*)

La capa convolutiva constituye el componente central de las redes neuronales convolucionales (CNN), ya que es en esta etapa donde se realiza la mayor parte del procesamiento computacional. Como se muestra en la Figura 9, esta capa emplea filtros o kernels, que son matrices bidimensionales de pesos, los cuales se deslizan sobre la imagen de entrada realizando una operación matemática denominada convolución.

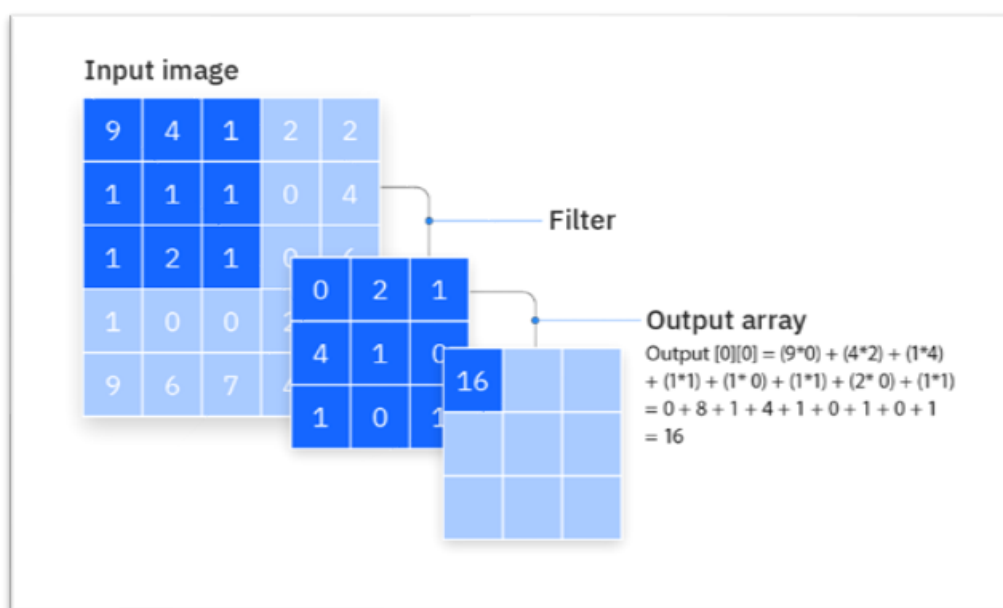
Durante este proceso, el filtro examina pequeñas regiones locales de la imagen, permitiendo detectar patrones o características relevantes como bordes, esquinas, texturas o formas simples. Cada filtro puede aprender a detectar una característica distinta, y el resultado de aplicar dicho filtro genera un nuevo conjunto de datos denominado mapa de activación o feature map. Este mapa representa la activación de ciertas regiones de la imagen original, resaltando aquellas zonas donde la característica buscada está presente.

Después de aplicar cada convolución, es común que la red aplique una función de activación no lineal, como ReLU (Unidad Lineal Rectificada), sobre el mapa de características resultante. Esta operación no lineal permite que el modelo aprenda representaciones más complejas y profundas, contribuyendo a la capacidad de la red para modelar relaciones no

lineales en los datos de entrada. La combinación de convolución y activación permite que las CNN extraigan progresivamente características de bajo nivel (como bordes) en las capas iniciales y características más abstractas (como formas o patrones complejos) en las capas posteriores.

Figura 9

Funcionamiento de una red neuronal convolucional



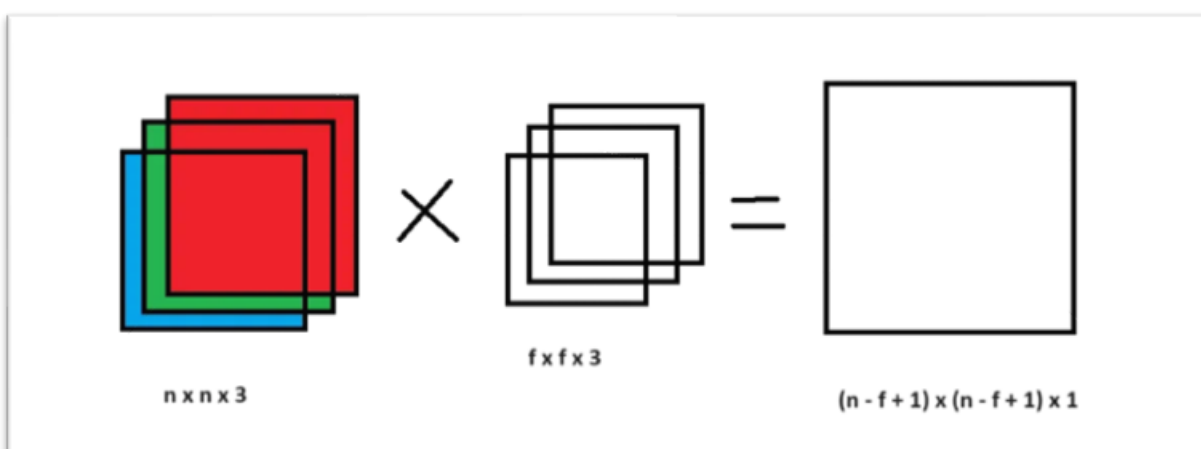
Nota. – Redes neuronales convolucionales se muestra el proceso de convolucion en las diferentes capas y filtros. Adaptado de Convolutional Neuronal Networks, por IBM International Business Machines, IBM (<https://www.ibm.com/think/topics/convolutional-neural-networks>). CC by 2.0

Es importante señalar que en una capa convolucional no se emplea un único kernel, sino múltiples kernels en paralelo. Como se muestra en la Figura 10, esto permite obtener múltiples mapas de características simultáneamente, cada uno capturando un aspecto distinto de la imagen. Es común que el número de estos filtros aumente a medida que se profundiza en la arquitectura de la red, siguiendo una progresión en potencias de dos (por ejemplo: 8, 16, 32, 64, 128, etc.), lo cual contribuye a la capacidad de la red para representar información compleja.

Además, para que la operación de convolución sea válida, el número de canales de entrada del filtro debe coincidir con el número de canales de la imagen de entrada. Por ejemplo, si la imagen de entrada tiene dimensiones $W \times H \times D$ (ancho, alto y profundidad o canales de color como RGB), entonces cada kernel deberá tener dimensiones $F \times F \times D$, donde F representa el tamaño espacial (alto y ancho) del filtro. Esto asegura que la convolución se realice correctamente sobre toda la profundidad de la imagen, produciendo una salida bidimensional por cada filtro. En la Figura 10 se muestra cómo se adapta esta operación en imágenes con múltiples canales, como las de color.

Figura 10

Operación de convolución



Nota. – Operación de convolución en una imagen RGB, donde tanto la profundidad de la imagen de entrada y los kernel tienen que ser las mismas. Adaptación de Detailed understanding about convolution operation in a coloured image with the help of 3D kernels, por Abhishek Jain, 2024, Medium (<https://medium.com/@abhishekjainindore24/detailed-understanding-about-convolution-operation-in-a-coloured-image-with-the-help-of-3d-kernels-16d698645988>).

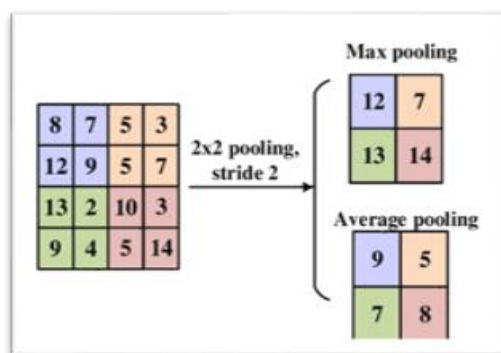
2.5.3.2 Capa De Agrupación O Pooling Layer.

También conocidas como capas de submuestreo o downsampling, permiten reducir progresivamente las dimensiones espaciales de los mapas de características (feature maps) generados por las capas convolucionales. Esta reducción tiene como principal objetivo disminuir el costo computacional, la cantidad de parámetros, y mitigar el riesgo de sobreajuste (overfitting), todo esto sin perder información clave de los datos de entrada.

A diferencia de las capas convolucionales, las capas de pooling no aprenden parámetros durante el entrenamiento, ya que su operación está determinada por funciones matemáticas fijas que se aplican de forma independiente en cada región de la entrada. La operación de pooling se realiza mediante un filtro que se desplaza sobre el mapa de características, dividiendo la imagen en pequeñas regiones y aplicando una función de agregación sobre cada una de ellas. Como se puede observar en la Figura 11.

Figura 11

Capa de agrupación



Nota. – Adaptación capa de agrupación max y avg, por Muhammad Shoaib Ali, 2022, Medium (<https://medium.com/@muhammadshoaibali/flattening-cnn-layers-for-neural-network-694a232eda6a>).

Los tipos de pooling más comunes son el **max pooling** y el **average pooling**:

En el **max pooling**, se selecciona el valor máximo dentro de cada región del filtro. Esta técnica es ampliamente utilizada por su capacidad para conservar las características más destacadas (como bordes o picos de activación), lo cual es especialmente útil para tareas de clasificación y detección.

En el **average pooling**, en cambio, se calcula el valor promedio de cada región. Aunque puede ser útil en ciertos contextos donde se busca suavizar las características, generalmente se considera menos efectivo que el max pooling en la práctica.

La principal ventaja del max pooling es que, además de reducir las dimensiones, permite conservar de manera efectiva las activaciones más importantes, lo que mejora la robustez del modelo frente a pequeñas variaciones o desplazamientos en la imagen. Esta propiedad se conoce como invarianza traslacional, y es clave en la capacidad de generalización de las CNN.

No obstante, se debe tener especial cuidado al seleccionar el tamaño del filtro de pooling y el stride (paso del filtro), ya que una configuración inadecuada puede eliminar demasiada información o no reducir suficientemente el tamaño de los datos, afectando negativamente el rendimiento del modelo. Un filtro de tamaño demasiado grande puede provocar una pérdida excesiva de información relevante, mientras que uno muy pequeño puede mantener un tamaño alto en los datos, generando mayor carga computacional y menor eficiencia.

2.5.3.3 Capa Totalmente Conectada o Fully-Connected Layer (FC).

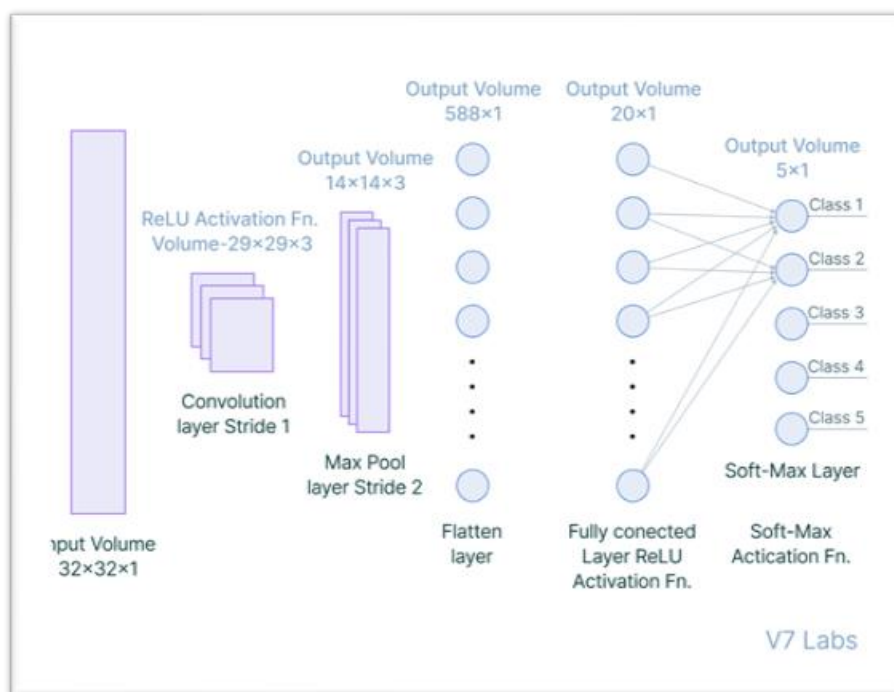
La capa totalmente conectada, también conocida como fully-connected layer o simplemente FC, es una de las etapas finales en la arquitectura de una red neuronal

convolucional (CNN). Su función principal es realizar la tarea de clasificación basándose en las características previamente extraídas por las capas convolucionales y de pooling.

En esta capa, cada nodo o neurona está conectado directamente con todos los nodos de la capa anterior como vemos en la Figura 12, lo que permite una integración completa de la información procesada hasta ese punto. A diferencia de las capas convolucionales, donde las conexiones son locales y limitadas a regiones específicas del mapa de activación, en la FC se utilizan conexiones densas que permiten combinar todas las características relevantes para generar una decisión final.

Figura 12

Capa totalmente conectada y procesos de capas anteriores



Nota. – Adaptación de A Comprehensive Guide to Convolutional Neural Networks, por Pragati Baheti, 2021, v7 labs (<https://www.v7labs.com/blog/convolutional-neural-networks-guide>).

La información que llega a esta capa suele estar ya transformada en un vector unidimensional (flattened vector), producto del proceso de aplanamiento (flattening) que convierte los mapas de activación bidimensionales en un solo arreglo de valores. Estos valores representan un resumen de alto nivel de las características más significativas presentes en la imagen original, como formas, texturas o patrones complejos.

En cuanto a la función de activación, mientras que en las capas anteriores se utiliza comúnmente ReLU (Rectified Linear Unit) para introducir no linealidades, en las capas fully-connected o FC finales es habitual emplear la función softmax, especialmente en tareas de clasificación multiclase. Esta función transforma los valores de salida en probabilidades normalizadas entre 0 y 1, donde cada valor representa la probabilidad de que la entrada pertenezca a una determinada clase. La clase con la probabilidad más alta se considera la predicción final del modelo.

En redes neuronales profundas, puede haber una o varias capas FC entre las capas de extracción de características y la capa de salida. Estas capas adicionales permiten realizar combinaciones más complejas de las características aprendidas y pueden contribuir a mejorar la capacidad predictiva del modelo. Sin embargo, también incrementan el número de parámetros, lo que puede generar un mayor riesgo de sobreajuste si no se cuenta con un volumen de datos suficiente o si no se aplican técnicas de regularización como dropout.

En resumen, la capa FC actúa como un clasificador final que interpreta y combina todas las características extraídas por las etapas previas de la red, y asigna una etiqueta o valor de salida al dato procesado. Es una parte fundamental del proceso de toma de decisiones en una CNN, y su correcta configuración influye directamente en el rendimiento global del modelo.

2.5.3.4 *Capa Softmax o Softmax layer.*

La capa Softmax es una capa de salida comúnmente utilizada en redes neuronales cuando se trata de resolver problemas de clasificación multiclase. Su principal función es transformar un vector de valores reales, que pueden estar en cualquier rango, en un vector de probabilidades normalizadas, en el que cada valor está comprendido entre 0 y 1, y la suma total de todos los valores es igual a 1. Esta propiedad permite interpretar la salida como la probabilidad de que la entrada pertenezca a una clase específica.

Matemáticamente, la función softmax aplica la siguiente operación a cada valor de entrada z_i , dentro de un vector z como se ve en la ecuación (8):

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} \quad (8)$$

Donde n es el número total de clases. Esta fórmula garantiza que los valores más altos en el vector original tendrán una mayor probabilidad, mientras que los valores menores recibirán proporciones correspondientemente más bajas, sin perder la relación relativa entre ellos.

La capa softmax suele colocarse al final de la red, justo después de una o varias capas totalmente conectadas (fully-connected layers). Es en esta capa donde se realiza la decisión final del modelo, ya que el resultado de la función softmax indica cuál es la clase más probable que corresponde a la entrada procesada. En tareas como reconocimiento de imágenes, clasificación de texto o detección de objetos, esta capa es crucial para traducir las representaciones abstractas internas de la red en una salida comprensible y accionable.

Una de las principales ventajas de utilizar softmax es que permite comparar directamente la confianza del modelo respecto a cada clase posible. Por ejemplo, si el modelo asigna una probabilidad del 0.95 a la clase "gato" y 0.05 a "perro", se puede inferir que el modelo está altamente seguro de su predicción. Esta propiedad también es útil en tareas donde se necesita establecer un umbral de decisión o interpretar el grado de certeza en entornos de clasificación con incertidumbre.

En resumen, la capa Softmax cumple un papel fundamental en redes neuronales para clasificación, ya que convierte la salida bruta de la red en una distribución de probabilidad interpretable, facilitando tanto la evaluación del rendimiento como la implementación de decisiones basadas en las predicciones del modelo.

2.5.3.5 *Aplicación De La Redes Neuronales Convolucionales.*

Las redes neuronales convolucionales son una parte muy importante en el Deep learning, ya que esta especialmente diseñada para procesar datos con estructura espacial, en imágenes, señales y secuencias, tiene un uso amplio en diversas áreas:

- Visión por computadora. - análisis de imágenes y videos, como detección medicas en radiografías como modelos de ResNet o DenseNet, reconocimiento de animales como iNaturalist, filtrado de contenido como en las redes sociales, detección de objetos como Yolo, Faster R-CNN son usados en Tesla, Waylo.
- Procesamiento de señales y series temporales, tenemos detecciones médicas, análisis de audio con Conv1D como aplicaciones Siri, Alexa, detección de audio como Shazam e identificación de patrones en series temporales de la bolsa de valores.

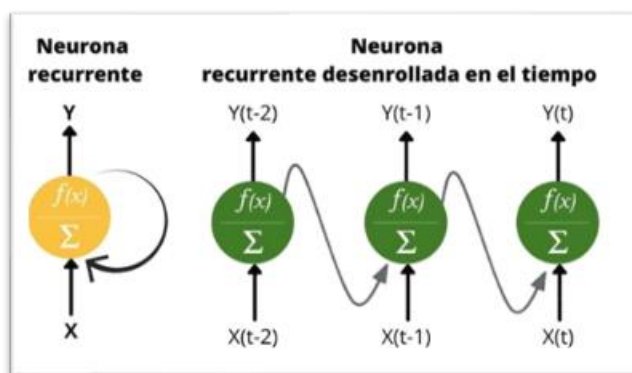
- Procesamiento de lenguaje natural (NPL) aunque los transformer dominan el NPL, las CNN aun como textCNN, en aplicaciones twiter X, fake news, traducción automática CNN + RNN ejemplo Google translate.
- Aplicaciones en dispositivos móviles y Edge computing. – modelos como MobileNet o EfficientNet usadas en apps fotográficas en sus mejoras de imagen y modo retrato usados ejemplo Google Pixel, o realidad aumentada como Snapchat filters, Pokemon Go y IOT internet de las cosas.

2.5.4 Redes Neuronales Recurrentes (RNN).

La red neuronal recurrente o RNN se entrena con datos secuenciales o de series temporales para crear un modelo de machine learning (ML) y/o Deep learning (DL) como LSTM y GRU que puede hacer predicciones o conclusiones secuenciales basándose en entradas secuenciales como se ve en la Figura 13.

Figura 13

Neurona recurrente



Nota. – adaptada de una red neuronal recurrente y como esta se desarrolla en ciclos, por Ruben Cañadas, 2021, abdatum (<https://abdatum.com/tecnologia/redes-neuronales-recurrentes>).

Al igual que las redes neuronales recurrentes utilizan datos de entrenamiento para aprender.

Se distinguen por su memoria, ya que toman información de las entradas anteriores para que estén influyan en la entrada y salida actuales.

Las salidas de las redes neuronales recurrentes como se mencionó dependen de los elementos anteriores dentro de la secuencia, tengamos en mente una frase y cada palabra forma una secuencia en la que el orden importa, se rastrea el contexto manteniendo un estado oculto en cada paso temporal. Se crea un bucle de feedback al pasar el estado oculto de un paso temporal al siguiente. El estado oculto actúa como una memoria que almacena la información sobre entradas anteriores, en cada paso temporal, la red neuronal recurrente procesa la entrada actual junto con ese estado oculto del paso temporal anterior, permitiendo recordar puntos de datos anteriores y utiliza esa información para la salida.

Otra característica distintiva de las redes recurrentes es que comparten parámetros en cada capa de la red, mientras que las redes prealimentadas tienen diferentes pesos en cada nodo las redes neuronales recurrentes comparten el mismo parámetro de peso dentro de cada capa. Estos pesos todavía se ajustan a través de los procesos de retro propagación y descenso de gradiente facilitando el aprendizaje por refuerzo.

Las RNN utilizan algoritmos de propagación directa y retropropagación en el tiempo (BPTT) para determinar los gradientes (o derivadas), lo que difiere ligeramente de la retropropagación tradicional, ya que es específico de los datos de secuencia. Los principios de la BPTT son los mismos que los de la retropropagación tradicional, donde el modelo se entrena a sí

mismo calculando los errores de su capa de salida a su capa de entrada. Estos cálculos nos permiten ajustar y ajustar los parámetros del modelo adecuadamente. BPTT difiere del enfoque tradicional en que BPTT suma errores en cada paso de tiempo mientras que las redes prealimentadas no necesitan sumar errores, ya que no comparten parámetros a través de cada capa.

Una función de activación es una función matemática que se aplica a la salida de cada capa de neuronas de la red para introducir no linealidad y permitir que la red aprenda patrones más complejos en los datos. Sin funciones de activación, la RNN simplemente calcularía transformaciones lineales de la entrada, lo que la haría incapaz de manejar problemas no lineales. La no linealidad es crucial para aprender y modelar patrones complejos, particularmente en tareas como PLN, análisis de series temporales y predicción de datos secuenciales (IBM International Business Machines Corporation, s.f.).

2.5.4.1 Redes Neuronales Recurrentes Estándar.

Las Redes Neuronales Recurrentes estándar son una de las primeras arquitecturas diseñadas para el procesamiento de datos secuenciales. A diferencia de las redes neuronales tradicionales, que consideran cada entrada como independiente, las RNN estándar tienen conexiones que permiten que la información se propague de un paso de tiempo al siguiente. Esto significa que la salida en un instante determinado depende no solo de la entrada actual, sino también del estado oculto (hidden state) anterior, lo que introduce una forma de memoria temporal. Sin embargo, esta estructura básica tiene limitaciones importantes: cuando se trata de secuencias largas, estas redes tienen dificultades para aprender dependencias a largo plazo debido a problemas como el desvanecimiento o explosión del gradiente, lo que afecta

negativamente el rendimiento en tareas complejas como la traducción o el modelado de lenguaje. (IBM International Business Machines Corporation, s.f.).

2.5.4.2 Redes Neuronales Recurrentes Bidireccionales (BRNN).

Las Redes Neuronales Recurrentes Bidireccionales (BRNN) son una extensión de las RNN estándar que abordan algunas de sus limitaciones. La idea principal es procesar la secuencia de datos en ambas direcciones, es decir, hacia adelante (del pasado al futuro) y hacia atrás (del futuro al pasado). Esto permite que la red acceda tanto al contexto anterior como al posterior en cada paso de tiempo, mejorando significativamente la comprensión del significado contextual. Este enfoque es especialmente útil en tareas como el reconocimiento de voz, análisis de sentimiento y procesamiento del lenguaje natural, donde la información futura puede ayudar a entender mejor una palabra o evento anterior. Las BRNN suelen utilizarse en combinación con LSTM o GRU para obtener un rendimiento aún mejor.

2.5.4.3 Memoria A Corto Plazo (LSTM).

Las LSTM (Long Short-Term Memory) son una variante avanzada de las redes neuronales recurrentes (RNN), introducidas por Sepp Hochreiter y Jürgen Schmidhuber en 1997, con el objetivo de superar una de las principales limitaciones de las RNN estándar: el problema del desvanecimiento o explosión del gradiente. Este fenómeno dificulta el aprendizaje de relaciones de dependencia a largo plazo en las secuencias, especialmente cuando estas son muy extensas, como suele ocurrir en el procesamiento de lenguaje natural, análisis de series temporales y datos biométricos.

La innovación clave de las LSTM radica en su estructura interna de celdas de memoria, diseñada para preservar y controlar el flujo de información a través del tiempo. Esta celda de memoria está compuesta por tres compuertas fundamentales:

- La compuerta de entrada regula qué nueva información será almacenada en la celda.
- La compuerta de olvido decide qué información de la celda anterior debe ser descartada.
- La compuerta de salida determina qué parte de la información almacenada será utilizada como salida en el paso de tiempo actual.

Estas compuertas funcionan mediante funciones sigmoides y operaciones de producto punto, permitiendo a la red mantener, actualizar o desechar información de forma selectiva en cada paso temporal. Esta capacidad de control fino es lo que permite a las LSTM modelar dependencias a largo plazo sin que los gradientes se desvanezcan durante el entrenamiento.

Las LSTM son especialmente eficaces en tareas donde el contexto previo tiene un impacto determinante en las predicciones futuras. Por ejemplo:

- En traducción automática, la red debe recordar el sujeto al inicio de una oración para traducir correctamente el verbo al final.
- En reconocimiento de voz, la pronunciación de una sílaba puede depender de sonidos anteriores.
- En análisis de sentimientos, una negación inicial (“no”) puede cambiar completamente el sentido del texto posterior.

- En predicción de series temporales financieras o meteorológicas, el comportamiento pasado es crucial para hacer pronósticos precisos.

Además, las LSTM han demostrado un rendimiento sobresaliente cuando se integran en arquitecturas más complejas, como los modelos Encoder–Decoder con atención, utilizados en traductores automáticos modernos o modelos de resumen automático de textos. También se emplean en la detección de anomalías, generación de texto y en modelos de predicción de eventos futuros en distintas industrias.

En resumen, las LSTM son un pilar fundamental en el desarrollo de redes neuronales que necesitan retener información contextual a través del tiempo, y su diseño estructural continúa siendo una de las soluciones más efectivas y versátiles para el modelado de secuencias en el campo de la inteligencia artificial.

2.5.4.4 Unidades Recurrentes Bloqueadas (GRU).

Las GRU (Gated Recurrent Units) son una versión simplificada y eficiente de las LSTM. Propuestas por Kyunghyun Cho y colegas en 2014, las GRU combinan las funcionalidades de las compuertas de entrada y olvido en una sola compuerta de actualización, lo que reduce el número de parámetros sin comprometer significativamente el rendimiento. Esta arquitectura mantiene muchas de las ventajas de las LSTM, como el manejo de dependencias a largo plazo, pero con menor complejidad computacional y un entrenamiento más rápido. Gracias a estas características, las GRU son una opción popular en aplicaciones con recursos limitados o donde se requiere una respuesta en tiempo real, como en sistemas de recomendación, asistentes virtuales, y análisis de sentimientos.

2.5.4.5 Codificador-Decodificador RNN.

El modelo Encoder-Decoder basado en RNN es una arquitectura central en tareas de secuencia a secuencia (seq2seq), como la traducción automática, resumen de texto, y chatbots. Este modelo está compuesto por dos partes principales:

- El codificador (encoder), que recibe una secuencia de entrada (por ejemplo, una oración en español) y la convierte en un vector de estado interno (también llamado vector de contexto) que resume toda la información relevante.
- El decodificador (decoder), que toma este vector de contexto y genera una secuencia de salida correspondiente (por ejemplo, la traducción en inglés).

Tanto el codificador como el decodificador suelen estar formados por RNN, LSTM o GRU, y a menudo se combinan con mecanismos adicionales como el mecanismo de atención (attention) para mejorar la capacidad de enfoque en partes específicas de la secuencia. Esta arquitectura es clave para aplicaciones donde se necesita transformar una entrada secuencial en otra secuencia de diferente longitud o estructura).

2.5.5 YOLOv5.

YOLO (You Only Look Once) es un algoritmo de detección de objetos en tiempo, es una de las herramientas más eficientes en el campo de la visión por computadora. Su principio fundamental es realizar la detección de objetos mediante una sola pasada a través de una red neuronal convolucional, lo que permite identificar múltiples objetos dentro de una imagen, asignándoles coordenadas de localización (bounding boxes) y un valor de probabilidad o confianza asociado a su clase. Esta característica lo hace sumamente rápido y adecuado para

aplicaciones en tiempo real como videovigilancia, conducción autónoma, sistemas de asistencia y análisis de imágenes médicas, entre otros.

A lo largo de su evolución, YOLO ha tenido varias versiones significativas. En particular, YOLOv5, desarrollado por Glenn Jocher y lanzado en 2020 bajo la empresa Ultralytics, representa una de las versiones más populares debido a su facilidad de uso, eficiencia y precisión como observamos en la Figura 14. A diferencia de sus predecesores, YOLOv5 fue implementado completamente en PyTorch, un framework de aprendizaje profundo ampliamente adoptado por la comunidad científica. El repositorio de YOLOv5 no es una continuación oficial de YOLOv4, pero ha ganado popularidad por su enfoque modular, soporte activo y rendimiento sobresaliente.

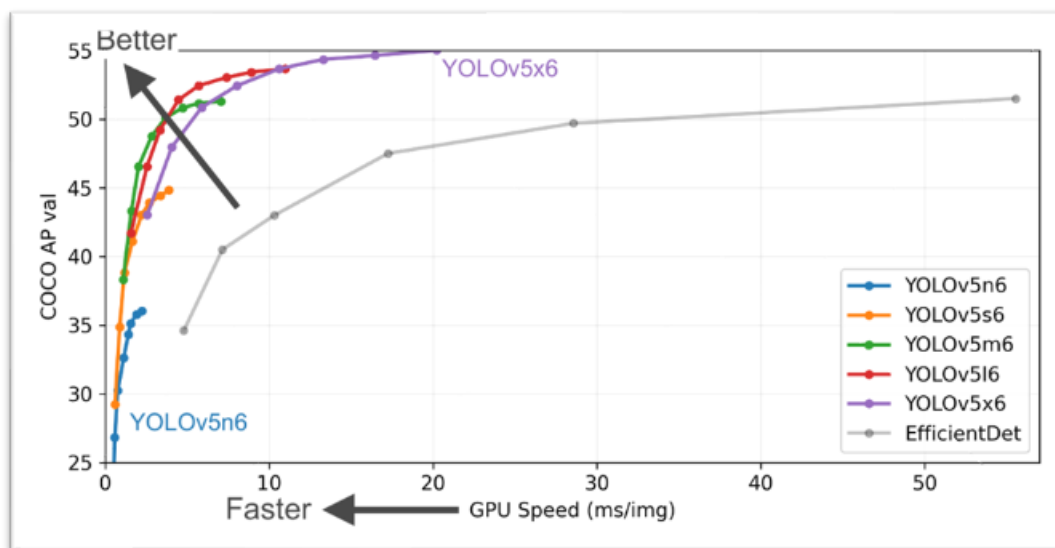
YOLOv5 cuenta con cuatro variantes principales:

- YOLOv5s (small): más rápido y ligero, ideal para dispositivos con recursos limitados.
- YOLOv5m (medium): un equilibrio entre velocidad y precisión.
- YOLOv5l (large): mejor precisión con un mayor coste computacional.
- YOLOv5x (extra-large): el más preciso, pero también el más demandante en términos de procesamiento y tiempo de entrenamiento.

Cada una de estas versiones está diseñada para adaptarse a distintos requerimientos, ofreciendo una progresión en el rendimiento y la precisión según la complejidad de la tarea y los recursos disponibles. En conjunto, estas características convierten a YOLOv5 en una herramienta versátil para la detección de objetos, tanto en imágenes estáticas como en secuencias de video en la detección de objetos en tiempo real.

Figura 14

YoloV5 grafica de rendimiento



Nota. – Adaptación de la gráfica de rendimiento de yoloV5 en sus diferentes versiones, donde COCO AP val denota la precisión media (mAP) sobre la intersección de unin (IoU) y la velocidad de GPU mide el tiempo medio de inferencia por imagen en el conjunto de datos COCO, por Ultralytics, 2020, Ultralytics (<https://github.com/ultralytics/yolov5>).

2.5.6 Tesseract OCR.

Tesseract es un reconocido motor de reconocimiento óptico de caracteres (OCR) que permite la extracción de texto desde imágenes digitales, ya sean escaneos de documentos, fotografías de texto impreso o manuscrito, entre otros. Este sistema es capaz de identificar caracteres individuales, palabras y bloques de texto, facilitando la digitalización y estructuración de información contenida en documentos físicos.

Tesseract fue inicialmente desarrollado por los Hewlett-Packard Laboratories en Bristol, Reino Unido, y en Greeley, Colorado (EE. UU.) entre 1985 y 1994. Posteriormente, en 1996, se

realizaron modificaciones para portarlo a sistemas Windows, y en 1998 se introdujeron cambios importantes en su estructura utilizando C++. En el año 2005, el proyecto fue liberado como código abierto por Hewlett-Packard, y desde 2006 hasta noviembre de 2018, Google asumió su desarrollo, mejorando significativamente su capacidad de reconocimiento y ampliando el soporte a múltiples idiomas y formatos.

Actualmente, Tesseract es considerado uno de los motores OCR de código abierto más robustos, y es ampliamente utilizado en tareas de extracción de datos, digitalización de documentos históricos, análisis de formularios, facturas, recibos y en proyectos que combinan visión por computadora con procesamiento de lenguaje natural (NLP).

2.6 METODOLOGIA MOBILE-D

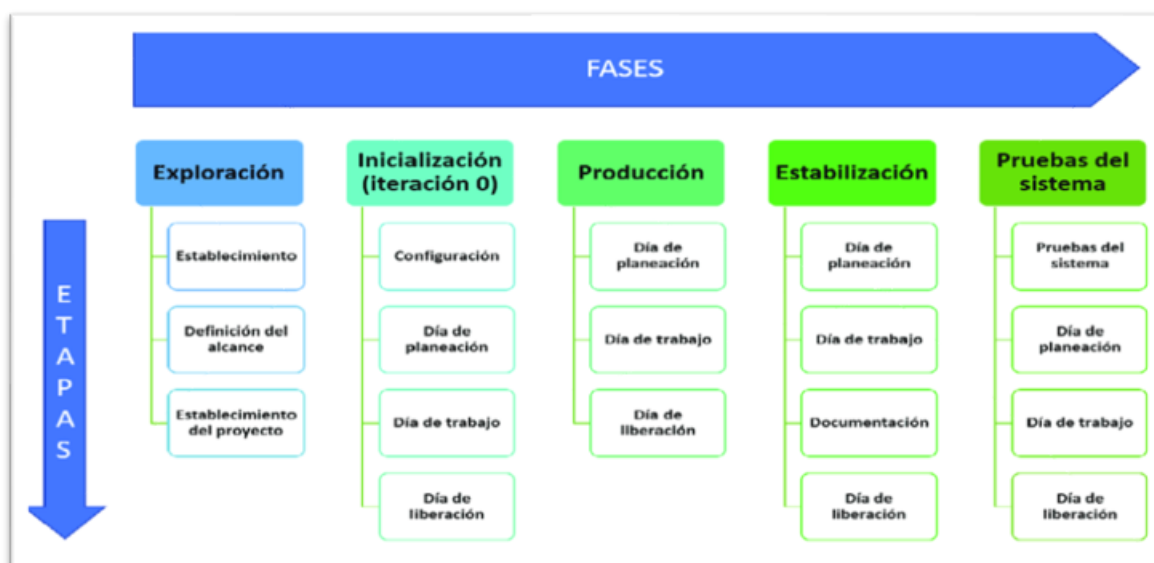
El desarrollo de la aplicación se ha realizado utilizando la metodología Mobile-D, un enfoque ágil especialmente diseñado para la creación de aplicaciones móviles, donde los ciclos de desarrollo son rápidos y se requiere una alta adaptabilidad a los cambios. Esta metodología se basa en la combinación de principios y buenas prácticas provenientes de metodologías ampliamente reconocidas, como Extreme Programming (XP), Crystal Methodologies y Rational Unified Process (RUP).

Mobile-D está especialmente orientada a equipos pequeños y a entornos donde la entrega temprana y continua del producto es prioritaria. Además, promueve una estrecha colaboración entre los desarrolladores y los usuarios, permitiendo validar continuamente las funcionalidades y adaptarlas según las necesidades del proyecto.

Esta metodología se organiza en cinco fases clave, que permiten guiar de forma estructurada y ágil todo el proceso de desarrollo, desde la concepción de la idea hasta la entrega del producto final. Estas fases son: exploración, inicialización, producción, estabilización y pruebas, como se muestra en la Figura 15.

Figura 15

Metodología Mobile-D



Notas. – Adaptación de la metodología Mobile-D, se muestra sus diferentes fases y etapas correspondiente para un mejor entendimiento, por Moramay Ramírez Hernández y Francisco León Pérez, 2023, Researchgate (https://www.researchgate.net/figure/Figura-1-Metodologia-Mobile-D_fig1_368555118).

Fases de la metodología Mobile-D:

- Exploración. - Esta fase inicial tiene como objetivo establecer una visión clara del proyecto. Se definen los conceptos fundamentales, el alcance general, los objetivos del sistema, las funcionalidades deseadas, y se identifican los principales riesgos.

Además, se planifican las siguientes fases del proyecto. Esta etapa resulta clave para alinear al equipo de desarrollo con los requerimientos del cliente y los usuarios finales, asegurando que todos comprendan las expectativas del producto.

- **Inicialización.** - En esta etapa se identifican y organizan todos los recursos técnicos, humanos y de tiempo necesarios para el desarrollo de la aplicación. También se definen las herramientas, frameworks, bibliotecas y entornos que serán utilizados, asegurando una infraestructura sólida y adecuada. Se estructura el equipo de trabajo y se determinan los roles y responsabilidades, creando así un entorno preparado para comenzar el desarrollo iterativo.
- **Producción.** - Esta fase corresponde al núcleo del proceso de desarrollo, donde las funcionalidades del sistema se construyen de forma iterativa e incremental mediante ciclos de planificación-trabajo-liberación. Cada iteración comienza con la definición de requisitos específicos, seguida de la planificación de tareas concretas. Un principio clave en esta etapa es la práctica de Test-Driven Development (TDD), en la cual primero se escriben las pruebas automatizadas que verifican el comportamiento esperado de una funcionalidad, y luego se procede al desarrollo del código que debe cumplir dichas pruebas. Esta práctica mejora significativamente la calidad del software y permite detectar errores de forma temprana.
- **Estabilización.** - Una vez implementadas las funcionalidades principales, se inicia un proceso de integración y refinamiento del sistema. En esta fase se busca garantizar la coherencia y estabilidad del producto mediante pruebas de integración, validación del rendimiento y resolución de defectos. El objetivo es asegurar que todos los

componentes funcionen correctamente en conjunto, y que el sistema como un todo cumpla con los estándares esperados de calidad.

- Pruebas. - Finalmente, se realiza una fase exhaustiva de pruebas donde el producto completamente integrado es evaluado frente a los requisitos funcionales y no funcionales definidos al inicio del proyecto. Esta etapa tiene como finalidad entregar una versión estable, robusta y funcional, eliminando todos los errores o inconsistencias detectadas. El sistema es validado por el cliente o los usuarios finales, asegurando que cumple con sus expectativas antes de su despliegue o lanzamiento.

La metodología Mobile-D, a través de estas fases bien definidas, permite manejar proyectos de desarrollo móvil de forma ordenada, flexible y centrada en la entrega continua de valor. La combinación de iteraciones ágiles, prácticas de calidad y validación constante con los usuarios convierte a Mobile-D en una alternativa eficaz para el desarrollo moderno de aplicaciones móviles.

2.7 TECNOLOGIAS

En este apartado se describen las principales herramientas, plataformas y tecnologías utilizadas durante el desarrollo del presente proyecto. Estas tecnologías abarcan diversos componentes fundamentales para el ciclo completo de implementación, incluyendo entornos de desarrollo integrados (IDE), lenguajes de programación, bibliotecas de software, plataformas de ejecución en la nube, sistemas de control de versiones y modelos de inteligencia artificial.

Son las siguientes:

- **VSCode (Visual Studio Code).** - Editor de código fuente desarrollado por Microsoft para diferentes sistemas operativos: Windows, Linux, MacOS, etc. Incluye soporte para depuración, control integrado de Git, respaldo de sintaxis y refactorización de código, siendo código fuente de licencia MIT, Freeware logo de la aplicación en la Figura 16.

Figura 16

Logo VsCode



Nota. – Microsoft, 2025, VsCode (version 1.100.2) [Software de computadora], Visual Studio CODE (<https://code.visualstudio.com/>).

- **Colab o Colaboraty.** - es un servicio de Google que permite escribir y ejecutar código Python en un navegador, sin necesidad de ser instalado, podemos ver el logo en la Figura 17. Es una plataforma de aprendizaje y educación que ofrece acceso gratuito a recursos de computación con GPU's y TPU's.

Figura 17

Logo Colab



Nota. – Google LLC, 2025, Collaboraty (version 2025-04-09) [Aplicación Web], Google Colab (<https://colab.research.google.com/>).

- **Android Studio.** – es el entorno de desarrollo integrado (IDE) oficial de Google para crear aplicaciones para Android podemos ver el logo en la Figura 18. Es una herramienta que se basa en IntelliJ IDEA.

Figura 18

Logo Android Studio



Nota. – Google LLC, 2025, Android Studio (versión Merkat 2024.3.2) [Software de computadora], Android Studio (<https://developer.android.com/studio?hl=es-419>).

- **Python.** – es un lenguaje de programación de alto nivel, versátil y multiparadigma, interpretado y de código abierto, conocido por su sintaxis clara y legible podemos ver el logo en la Figura 19. Es utilizado en diversas áreas ciencia de datos, aprendizaje automático y automatización.

Figura 19

Logo Python



Nota. – Python Software Foundation, 2025, Python (version 8.3) [Software de computadora] (<https://www.python.org/downloads/>).

- **YoloV5.**- es un modelo de visión artificial para la detección de objetos, es una versión mejorada de los modelos Yolo anteriores y opera a alta velocidad de inferencia, lo que hace eficaz para aplicaciones en tiempo real podemos ver el logo en la Figura 20.

Figura 20

Logo YoloV5

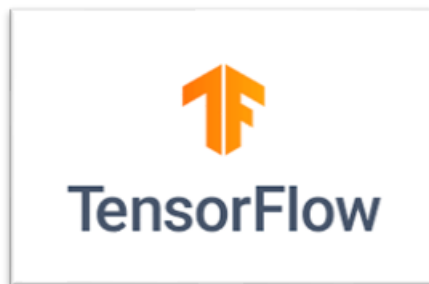


Nota. – Ultralytics, 2025, YoloV5 (version s) [Modelo de visión artificial], Ultralytics (<https://github.com/ultralytics/yolov5/wiki>).

- **Tensor Flow.** - es un sistema de computación numérica de código abierto de GoogleCloud. Lo podemos encontrar en numerosos productos de Google Cloud ya que nos permite desarrollar algoritmos inteligentes podemos ver el logo en la Figura 21.

Figura 21

Logo TensorFlow



Nota. – Google Brain, 2025, Tensorflow (version 2.12.0) [biblioteca de software], google (<https://www.tensorflow.org/>).

- **PyTorch.** -Es una biblioteca de aprendizaje automático de código abierto desarrollada por **Meta AI**. Ofrece una interfaz flexible y eficiente para la creación de redes neuronales profundas, permitiendo definir modelos dinámicos mediante el uso de grafos computacionales imperativos. Se caracteriza por su facilidad de uso, compatibilidad con CUDA para acelerar el procesamiento con GPU, y su integración con herramientas como TorchVision para tareas de visión por computadora. En la Figura 22 se presenta el logo.

Figura 22

PyTorch



Nota. – Meta AI, 2025, *PyTorch* (versión 2.3.0) [Biblioteca de software]. <https://pytorch.org/>.

- **Kotlin.** – Es un lenguaje de programación moderno, multiplataforma y de código abierto, desarrollado por JetBrains Figura 23. Fue creado para ser interoperable con Java, y aunque inicialmente se enfocó en el desarrollo de aplicaciones de escritorio, ha ganado popularidad especialmente en el desarrollo de aplicaciones móviles Android.

Figura 23

Logo Kotlin



Nota. – **JetBrains**, 2025, **Kotlin** (version 1.7.20) [lenguaje de programacion], Android Studio.

- **Github.** – es una plataforma en línea para alojamiento de código que permite a las desarrolladoras almacenar, compartir y colaborar en proyectos de software utilizado el sistema de control de versiones Git, podemos ver el logo en la Figura 24. Es una herramienta esencial para el desarrollo de software, ya que facilita la gestión de versiones, la colaboración entre equipos y la distribución de código fuente.

Figura 24

Logo GitHub



Nota. – Awesome LLC, 2025, Github [Control de versiones colaborativo], (<https://github.com/>).

CAPITULO III

3 MARCO APLICATIVO

3.1 INTRODUCCION

En el presente capítulo se describe el proceso de desarrollo de la aplicación móvil utilizando la metodología Mobile-D, la cual está específicamente orientada al desarrollo ágil de software para dispositivos móviles. Esta metodología propone una estructura organizada en fases que permiten abordar de forma iterativa e incremental el diseño, desarrollo y prueba de aplicaciones móviles. A lo largo de este capítulo se detallan cada una de las fases y etapas que han sido aplicadas en el desarrollo del sistema propuesto, orientado a facilitar el registro contable digital de los comerciantes minoristas a partir de notas de venta manuscritas.

3.2 FASE DE EXPLORACION

En esta fase inicial se identificaron los requisitos preliminares necesarios para el desarrollo del sistema. Se estableció un plan general del proyecto, definiendo el alcance, los objetivos y las herramientas tecnológicas que se utilizarán. Asimismo, se identificaron los actores clave del sistema o grupo de interés (stakeholders), quienes cumplen roles esenciales tanto en la implementación como en el uso de la aplicación móvil.

3.2.1 Establecimiento De Grupo De Interés.

- **Desarrollador.** - Es el responsable del diseño, implementación y mantenimiento del sistema. En este proyecto, el desarrollador cumple el rol de arquitecto de software, encargado de diseñar una aplicación móvil funcional, confiable y eficiente. Su labor

implica la selección y configuración de las herramientas de desarrollo adecuadas (lenguajes de programación, frameworks, motores de inferencia, etc.), así como la integración de tecnologías de inteligencia artificial para el reconocimiento de texto manuscrito. Además, debe garantizar que el producto final cumpla con los requisitos y que se adapte al contexto y capacidades técnicas de los dispositivos móviles disponibles.

- **Comerciante minorista.** - Es el usuario final del sistema y representa al grupo de pequeños comerciantes que realizan ventas a través de notas manuscritas. Se caracteriza por operar con recursos limitados, tanto económicos como tecnológicos, lo que dificulta el acceso a herramientas digitales para la gestión contable. Estos comerciantes suelen registrar sus transacciones de manera manual, lo que conlleva errores, pérdida de información y dificultad para organizar sus finanzas o compartir datos de manera eficiente. El sistema propuesto busca atender esta problemática, facilitando la digitalización automática de sus notas de venta mediante una aplicación móvil sencilla, rápida y accesible.

3.2.2 Requisitos Iniciales.

Diseñar una aplicación móvil ligera y distribuida bajo una licencia GPL¹⁸. Esta aplicación estará orientada a contribuir al proceso de digitalización contable de los comerciantes minoristas bolivianos acogidos al **RTS**¹⁹, mediante el uso de técnicas avanzadas de Deep Learning y reconocimiento de texto manuscrito (Handwritten Text Recognition, HTR), para los

¹⁸ GPL licencia de uso libre (GNU licencia publica general)

¹⁹ RTS siglas de Régimen Tributario Simplificado

contribuyentes acogidos al régimen simplificado (RTS), que permita la digitalización de recibos manuscritos, sirviendo para un control de ventas y ser extendidas como recibos digitales.

3.2.3 Requerimientos Funcionales.

Los requerimientos funcionales establecen las funcionalidades esenciales que la aplicación debe cumplir para alcanzar sus objetivos. Estos son:

- Requerimiento funcional N°1.- La aplicación móvil debe permitir al usuario capturar una imagen de la nota de venta manuscrita utilizando la cámara del dispositivo.
- Requerimiento funcional N°2.- El sistema debe usar un modelo YOLOV5 para detectar automáticamente las áreas clave de la imagen, como la fecha, nombre del cliente, número de serie y detalle.
- Requerimientos funcional N°3.- El sistema debe aplicar un segundo modelo YOLOv5 para identificar múltiples celdas de detalle, incluyendo cantidad, descripción, precio unitario y precio total por línea.
- Requerimientos funcional N°4.- Una red neuronal CRNN debe procesar áreas detectadas para reconocer el texto manuscrito.
- Requerimientos funcional N°5.- El sistema debe permitir la exportación de la nota digital estructurada en un archivo PDF con formato legible.
- Requerimientos funcional N°6.- La aplicación debe almacenar los datos procesados en una base de datos SQLite local en el dispositivo.
- Requerimientos funcional N°7.- La aplicación debe permitir al usuario visualizar las notas almacenadas y ofrecer funcionalidades básicas para el análisis de ventas.

3.2.4 Requerimientos No Funcionales.

Los requerimientos no funcionales definen las características de calidad que debe poseer la aplicación para ser eficiente, accesible y útil en el entorno objetivo.

- Usabilidad: La aplicación debe contar con una interfaz gráfica simple, clara e intuitiva, adaptada a usuarios sin conocimientos técnicos. El flujo de trabajo debe ser directo y comprensible, sin necesidad de capacitación.
- Fiabilidad: La aplicación debe estar optimizada para dispositivos Android de gama media (como el poco F1, poco X3 pro), y debe garantizar un tiempo de procesamiento inferior a 15 segundos por nota, incluyendo análisis de la imagen, reconocimiento de texto y validación.
- Compatibilidad: La aplicación debe funcionar correctamente en versiones Android 8 en adelante y no debe requerir conexión a internet para su uso principal
- Visualización: Durante el proceso de captura, se mostrará una previsualización. Los datos reconocidos (texto y números) serán presentados en una estructura tipo tabla, que refleje el formato original del recibo y que pueda exportarse a formatos compatibles con sistemas contables.
- Idioma: la aplicación móvil tendrá por defecto el idioma castellano.

3.2.5 Definición Del Alcance.

Este proyecto posee alcances y limitaciones bien definidos, en función de los recursos disponibles, el contexto social del usuario objetivo y la viabilidad técnica del sistema.

3.2.5.1 Limitaciones.

- La implementación está orientada únicamente al contexto boliviano, específicamente la paz Bolivia para negociantes en RTS.
- La aplicación está limitada a reconocer textos manuscritos hechos en recibos o notas de ventas.
- La aplicación se limita a escritura clara y legible. Escrituras excesivamente desordenadas o ilegibles pueden no ser reconocidas con precisión.

3.2.5.2 Alcances.

- Diseñar una aplicación móvil basado en Deep Learning y HTR.
- Diseñar e implementar una tecnología innovadora para comerciantes minoristas que pertenecen al régimen simplificado de Bolivia.
- Utilizar los recursos en Deep Learning, CRNN, HTR, para la digitalización de recibos manuscritos.
- Generar recibos digitales.
- Almacenamiento de datos para análisis de ventas.

3.2.6 Establecimiento De Tecnologías.

Para el desarrollo de la aplicación móvil propuesta, se utilizarán tecnologías que permiten el procesamiento de imágenes, el reconocimiento de texto manuscrito y una implementación eficiente en dispositivos de gama media, **Flutter, TensorFlow, PyTorch, YOLOv5, Python, Google Colab, SQLite, Android Studio, GitHub**. Estas tecnologías fueron seleccionadas por

su rendimiento, compatibilidad con dispositivos móviles, disponibilidad de recursos en línea y comunidad activa de desarrollo.

3.3 FASE DE INICIACIÓN

La fase de iniciación tiene como objetivo definir y establecer los elementos fundamentales que guiarán el desarrollo del proyecto. Durante esta etapa se detallará la configuración del ambiente de desarrollo, se seleccionan las versiones de las tecnologías a utilizar. Además, se elaboran el diseño de la aplicación.

Como resultado de esta fase, se obtiene una visión clara del proyecto.

3.3.1 Configuración Del Ambiente De Desarrollo.

Esta actividad es específicamente para el desarrollador de la aplicación móvil ya que tiene como propósito el de instaurar un ambiente idóneo para llevar a cabo el desarrollo, en esta actividad además el desarrollador puede ejecutar pruebas de los requerimientos.

- Tipo de proyecto: aplicación móvil Android
- Tecnologías: YoloV5, CRNN, TFlite, Python, Kotlin

3.3.1.1 Preparación Del Ambiente.

La preparación del ambiente constituye un paso fundamental para garantizar que todas las herramientas y tecnologías necesarias para el desarrollo de la aplicación móvil funcionen de manera adecuada y sin conflictos. Esta etapa implica la instalación, configuración y verificación de compatibilidad de cada componente del ecosistema de desarrollo, con el objetivo de asegurar un entorno funcional, estable y coherente para la implementación del proyecto.

Para ello, se procede a la instalación y validación de las versiones correspondientes de las siguientes herramientas:

- Instalación de Visual Studio Code, descargar de la página oficial, a continuación, el link: <https://code.visualstudio.com/> para Windows 64 bits, instalar extensiones Python.
- Creación de una cuenta Github link a continuación <https://github.com>, se crea un repositorio, el cual se usará para publicar la aplicación y todo el código concerniente al proyecto.
- Para utilizar Google Colab en el proceso de entrenamiento de redes neuronales, es necesario contar con una cuenta de Google. Esta plataforma permite ejecutar código en la nube de manera gratuita, facilitando el desarrollo de modelos de aprendizaje automático sin requerir una infraestructura local. Sin embargo, debido a sus políticas de uso, es importante optimizar al máximo el tiempo disponible, ya que el uso prolongado y continuo puede conllevar a restricciones temporales. En este proyecto se empleará la configuración con GPU Tesla T4, la cual está disponible en la versión gratuita de Colab y ofrece un buen rendimiento para el entrenamiento de modelos de redes neuronales.
- Para el desarrollo del proyecto, es necesario instalar Python en el ordenador. La instalación puede realizarse desde el sitio web oficial: <https://www.python.org/>. En este caso, se utilizará la versión 3.8.0, seleccionada por su alta compatibilidad con diversos entornos de desarrollo, especialmente en proyectos vinculados al ecosistema

Android y herramientas como TensorFlow, TensorFlow Lite y otras bibliotecas relacionadas con inteligencia artificial.

- Para el desarrollo y prueba de la aplicación móvil, se utilizará Android Studio, el entorno de desarrollo oficial proporcionado por Google. Puede descargarse desde el sitio oficial: <https://developer.android.com/>. Esta herramienta permitirá compilar, depurar y ejecutar la aplicación directamente en emuladores o dispositivos físicos. El lenguaje de programación utilizado será Kotlin, debido a su integración nativa con Android y su creciente adopción como estándar para el desarrollo de aplicaciones en esta plataforma.

Tecnologías base de la aplicación:

- Frontend: kotlin
- Backend Local: almacenamiento SQLite
- DL: Tensorflow lite para CRNN, YoloV5
- Librerías: OpenCv, TFLite, YoloV5, TesseractOcr, Numpy, Pandas, Matplotlib, TensorFlow.

Herramientas:

- Entrenamiento: Google Colab, requiere cuenta Google gratuita, requiere conexión a internet, un navegador y una PC local
- Dataset: 1500 imágenes etiquetadas primera detección, segunda detección 6000 imágenes áreas, tercera detección palabras 24000.
- Exportación: TFLite para ejecución en dispositivos móviles.

3.3.2 Planificación De Fases.

En esta etapa se establecen las fases a desarrollar, las iteraciones que se llevarán a cabo y los procesos necesarios para cumplir con cada una de ellas dentro de la metodología Mobile-D.

Tabla 2

Diseño Mobile-D

Fase	Iteración	Descripción	Semana
Exploración	Iteración 0	- Establecimiento del proyecto. - Establecimiento de grupos de interés. - Limitaciones y alcances - Establecimiento de tecnologías.	
Inicialización	Iteración 0	- Análisis y requisitos iniciales. - Selección de herramientas - Planificación de dataset y pruebas.	
Producción	Iteración 1	- Captura de las notas de ventas, para crear los dataset. - Creación de códigos en Python para generar dataset.	
	Iteración 2	- Creación de códigos en Python para tratar las imágenes. - Detección de áreas en las notas ventas con YoloV5, mediante colab.	

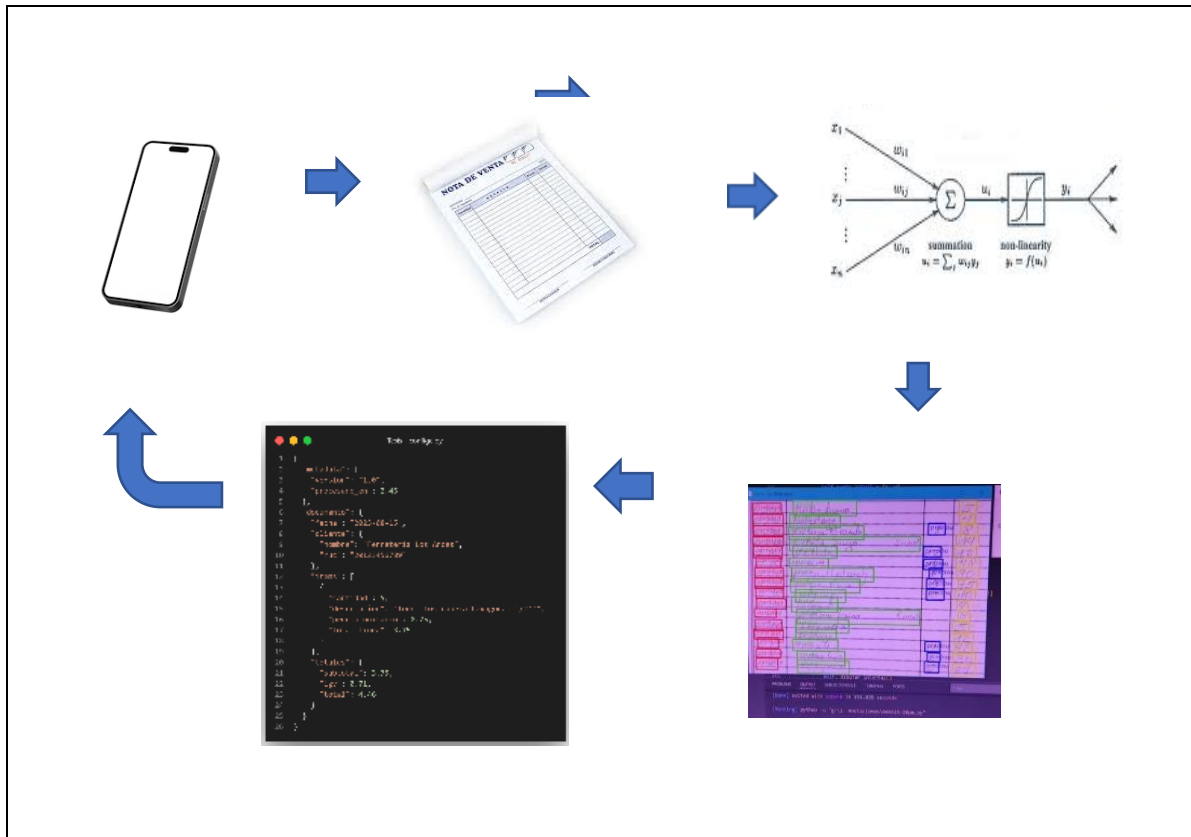
Iteración 3	- Incremento de captura de imágenes para incrementar la confianza en la detección de áreas. - Mejorando el código Python, para etiquetar datos en YoloV5. - Detección de áreas en subsección en nota de venta en el área detalle
Iteración 4	- Códigos Python para generar el dataset para el CRNN - Integración de CRNN para detección y extracción de texto manuscrito por áreas.
Iteración 5	- Corrección semántica haciendo uso de diccionario de palabras. - Estructura de notas digitales mediante la posición detectadas por yoloV5 en la iteración anterior.
Iteración 6	- Conversión y optimización de modelos YoloV5 a TFLite. - Conversión y optimización del modelo CRNN a TFLite.
Iteración 7	- Diseño de aplicación en Kotlin

		- Implementación de TFLite optimizados en código.
		- Manejo de datos tras captura.
Finalización	Iteración 8	- Versión instalable APK.
		- Pruebas de usabilidad.

3.3.3 Diseño De Aplicación.

La aplicación está diseñada con un enfoque MVVM (Model – View- ModelView) y basado en capas, integrando tecnologías de Deep Learning (DL) para el reconocimiento de escritura manuscrita y la digitalización de recibos. El sistema combina componentes de detección de regiones en dos fases la primera área generales de la nota de venta y las segundas celdas del área de detalle, reconocimiento de texto de las celdas detectadas y almacenamiento estructurado de datos.

Podemos apreciar los diferentes procesos simplificados en la Figura 25. La imagen es capturada mediante el móvil Android desde su cámara trasera, esta luego es procesada por la primera capa YoloV5 que detecta las áreas generales, posterior a esa, esta obtiene el área de detalle esta es enviada a otra capa YoloV5 donde esta es subdividida en subsecciones, cantidad, detalle, precioUnit y precioTotal, celda por celda, estas celdas son enviadas a la CRNN para la detección del texto manuscrito y devuelta como datos digitales, este se envía a un diccionario de palabras para la corrección de las palabras, y aprovechamos la detección de YoloV5 de las coordenadas usando promedios se generan las posiciones una vez se determina el las posiciones estas se almacenan en una matriz, para su posterior almacenamiento en SQLite.

Figura 25*Diseño de aplicación*

3.4 FASE DE PRODUCCION

En esta fase se lleva a cabo el desarrollo funcional completo de la aplicación móvil, aplicando todos los requisitos funcionales y no funcionales definidos en las etapas previas.

Durante esta fase, se implementa tanto las funcionalidades esenciales como las complementarias del sistema, entre ellas destacan la captura de notas manuscritas a través de la cámara, la detección automática de secciones clave con YOLOv5, el reconocimiento de texto manuscrito mediante CRNN, el almacenamiento mediante SQLite y generación de reportes y notas de venta exportables.

3.4.1 Iteración 1.

3.4.1.1 Captura De Datos.

En esta etapa se procede a la recolección de la mayor cantidad posible de notas de venta manuscritas como se ve en la Figura 26, que servirán como base para el entrenamiento del modelo de reconocimiento de texto.

Las notas recolectadas serán digitalizadas y etiquetadas adecuadamente para su posterior uso en el proceso de entrenamiento. Este conjunto de datos representa una parte crucial del proyecto, ya que impacta directamente en la capacidad del sistema para generalizar ante nuevas entradas.

Figura 26

Notas de venta, recibos de venta

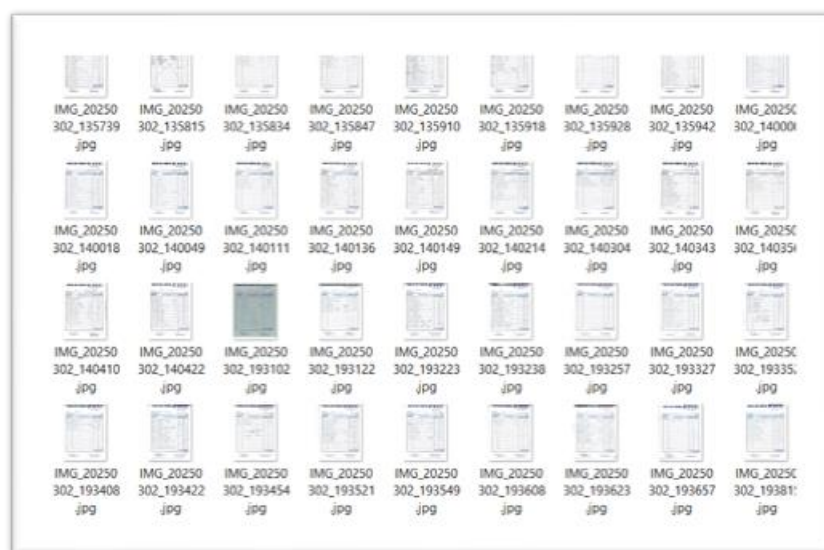


Las imágenes capturadas fueron organizadas de manera sistemática en una carpeta específica, con el propósito de facilitar su procesamiento y posterior etiquetado. Esta organización permite una mejor gestión del dataset durante las fases de entrenamiento, validación y prueba del modelo de Deep Learning.

En la Figura 27 se puede observar una vista general de la estructura de la carpeta que contiene las imágenes recopiladas, la mayoría de los recibos eran las copias, se efectuó una edición con filtros para documentos que añaden mas claridad a letras, con fondos mas claros y se recorto el area de interes, tambien se redimensiono a 960 x 960 para un tamaño ideal para la generación del dataset, se excluyeron aquellas que tenían letras ilegibles a simple vista, y tenían colores de tinta muy difíciles de contrastar con los filtros y los que tenían manchas muy fuertes en gran parte del recibo, tambien se excluyo a los que no tenían las columnas completas o carecían de datos manuscritos.

Figura 27

Notas de venta capturadas



3.4.1.2 Códigos Python Para Tratar Las Capturas.

Se diseña un código que permita capturar las áreas de interes para el entrenamiento de YoloV5 para reducir consumo y potencia de compute como muestra en la Figura 28. Son cuatro áreas de interes, fecha, el numero de serie que llevan impreso en la nota de venta, cliente referido en la nota de venta y el area mas importante de detalla la venta, donde se encuentra la cantidad, nombre del producto, su precio unitario y su precio de cobro, haciendo uso del lenguaje python generamos recuadros que se capturan para generar las áreas los boundingbox para el entrenamiento, las áreas capturadas son las que contiene algun valor, se excluyo las áreas sin informacion.

Figura 28

Notas de venta generación de bundingbox para el dataset

NOTA DE VENTA

LUGAR: **USA** MES: **7** AÑO: **2020**

Nº: **220099**

Señor(es): **MFT - Fabricio**

Por lo siguiente DEBE

Cantidad	DETALLE	P.Unit.	TOTAL
2 bl.	Rucula	7	14
3 K.	Luzma (33un)		1320
5 bs.	Elavados		5
5 bs.	Hararotau		5
5 bs.	Guiriguan		5
2 K.	Haba	7	14
1 bl.	Romero		3
3 K.	Tubilla blanca	7	21
4 K.	Lamati	6	24
4 bs.	Tubillon		4
1 bs.	Flora pueras		22
1 bs.	Ostras		18
3 bs.	Albahaca		3
1 bl.	papa Raja		50
3 un.	Chile		10
1 bl.	q. avasilla		28
5 un.	Pera		10
3 K.	Zanahoria	6	18
1 un.	Apio		2
TOTAL			2693

Controles de etiquetado:

- ☒ Guardar recuadro 1
- ☒ Guardar recuadro 2
- ☒ Guardar recuadro 3
- ☒ Guardar recuadro 4

Anterior Siguiente Guardar Eliminar Cen

Code

El código carga la carpeta de imágenes y crear un archivo csv con las coordenadas como se muestra en la Figura 29, que posteriormente se utilizara para generar áreas a detectar, para el entrenamiento en la detección de áreas, con el respectivo nombre de la imagen.

Figura 29

boundingbox csv, áreas para dataset



Posteriormente a esto se diseño otro script²⁰ en python para generar, organizar el dataset para el entrenamiento de la red neuronal como se aprecia en Figura 31, el script cuenta con dos partes la primera convierte las cordenadas x1, x2, y1, y2, capturadas estas tiene que convertirse a valores utilizables para los boundiboxes como se ve en la ecuacion (9) y la Figura 30, donde se

²⁰ Script es un archivo que contiene instrucciones para ser ejecutadas por un intérprete o motor.

creara txt con las el mismo nombre de la imagen y dentro tendran la clase C, Xcenter, Ycenter, ancho y altura.

(class C, X_{center} , Y_{center} , Width, Height) (9)

Figura 30

Script conversión áreas para boundingbox

```
with open(ruta_txt, 'w') as txtfile:
    for idx, recuadro in enumerate(recuadros):
        # Convertir coordenadas a YOLO
        x1, y1 = recuadro['x1'], recuadro['y1']
        x2, y2 = recuadro['x2'], recuadro['y2']

        x_centro = (x1 + x2) / 2.0
        y_centro = (y1 + y2) / 2.0
        w = x2 - x1
        h = y2 - y1

        # Normalizar
        x_centro /= ancho
        y_centro /= alto
        w /= ancho
        h /= alto

        # Escribir en TXT
        clase_id = class_ids[recuadro['clase']]
        txtfile.write(f"{clase_id} {x_centro:.6f} {y_centro:.6f} {w:.6f} {h:.6f}\n")

        # Guardar región recortada
        region = imagen[y1:y2, x1:x2]
        if region.size == 0:
            continue # Saltar regiones inválidas

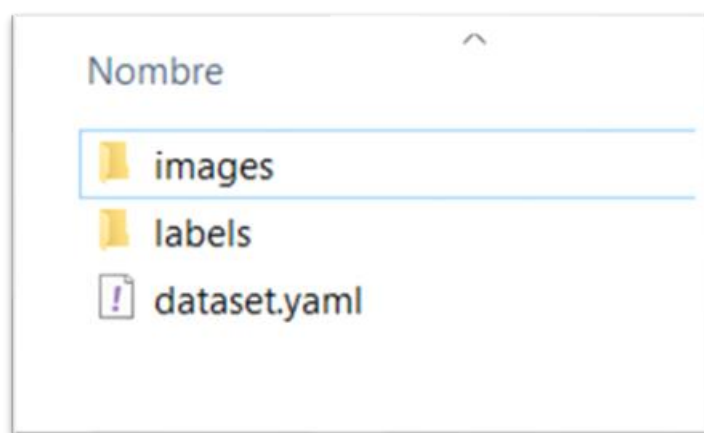
        nombre_region = f"{os.path.splitext(archivo)[0]}_{idx}.jpg"
        ruta_region = os.path.join(regions_dir, recuadro['clase'], nombre_region)
        cv2.imwrite(ruta_region, region)
```

La siguiente parte del scrip ordena el dataset en dos carpetas principales; images y labels, en images estan las imágenes y labels los txt con los boundiboxes para el entrenamiento dentro de cada carpeta se ecuentra organizada tres carpetas: val, train, test, los cuales se uso una regla del 100% de dataset, 75% para train, 15% val, 10, test, para finalizar se crea el archivo con la

extension yaml como se obserba en la Figura 31, la cual tiene la distribucion de las carpetas, tambien la cantidad de clases a entrenar y el nombre de las clases en una vector, este se encontrara con las carpetas images y labels, sera compimida en un archivo *.ZIP para su manejo.

Figura 31

formato de dataset de las notas de ventas



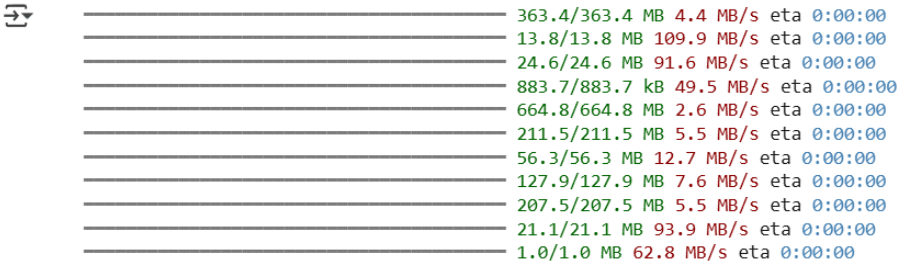
3.4.2 Iteración 2.

3.4.2.1 Entrenamiento De Detección De Áreas Con Yolov5.

Para el entrenamiento del modelo de detección de áreas en notas de venta, se utilizó Google Colab como entorno de trabajo. Esta plataforma, ofrecida por Google, permite acceder gratuitamente a recursos computacionales avanzados, como GPUs Tesla T4, que superan significativamente la capacidad gráfica de un ordenador convencional. Gracias a estas prestaciones, es posible entrenar modelos de aprendizaje profundo con mayor eficiencia y en menor tiempo.

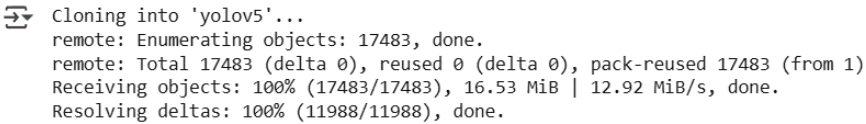
Figura 32*Instalación YOLOV5 y requisitos*

```
[ ] !pip install -qr https://raw.githubusercontent.com/ultralytics/yolov5/master/requirements.txt
```



363.4/363.4 MB	4.4 MB/s	eta 0:00:00
13.8/13.8 MB	109.9 MB/s	eta 0:00:00
24.6/24.6 MB	91.6 MB/s	eta 0:00:00
883.7/883.7 kB	49.5 MB/s	eta 0:00:00
664.8/664.8 MB	2.6 MB/s	eta 0:00:00
211.5/211.5 MB	5.5 MB/s	eta 0:00:00
56.3/56.3 MB	12.7 MB/s	eta 0:00:00
127.9/127.9 MB	7.6 MB/s	eta 0:00:00
207.5/207.5 MB	5.5 MB/s	eta 0:00:00
21.1/21.1 MB	93.9 MB/s	eta 0:00:00
1.0/1.0 MB	62.8 MB/s	eta 0:00:00

```
[ ] !git clone https://github.com/ultralytics/yolov5
```



```
Cloning into 'yolov5'...
remote: Enumerating objects: 17483, done.
remote: Total 17483 (delta 0), reused 0 (delta 0), pack-reused 17483 (from 1)
Receiving objects: 100% (17483/17483), 16.53 MiB | 12.92 MiB/s, done.
Resolving deltas: 100% (11988/11988), done.
```

```
!pip install -qr yolov5/requirements.txt # install dependencies (ignore errors)
```

En esta etapa se configurará el entrenamiento se subió el dataset en colab, se descomprime e instala las dependencias como se ve en la Figura 32, se creará una carpeta yolov5 y entrenamos el modelo con las siguientes configuraciones: yolov5 modelo s, colocamos en epochs 80, batch de 16, el data yaml que se creó en el dataset, esta configuración se determinó después de probar varias posibles llegando está a ser optima en resultados y tiempo.

Una vez entrenado revisamos el entrenamiento se agregó una configuración donde se guarda dos pesos de entrenamiento best y last, donde se guarda el mejor y el ultimo y hacemos pruebas para determinar el valor de detección que conseguimos y si hay Overfitting, podemos observar los valores de entrenamiento en la Figura 33.

Figura 33

Valores de Entrenamiento

```
Validating runs/train/exp2/weights/best.pt...
Fusing layers...
YOLOv5s summary: 157 layers, 7020913 parameters, 0 gradients, 15.8 GFLOPs
```

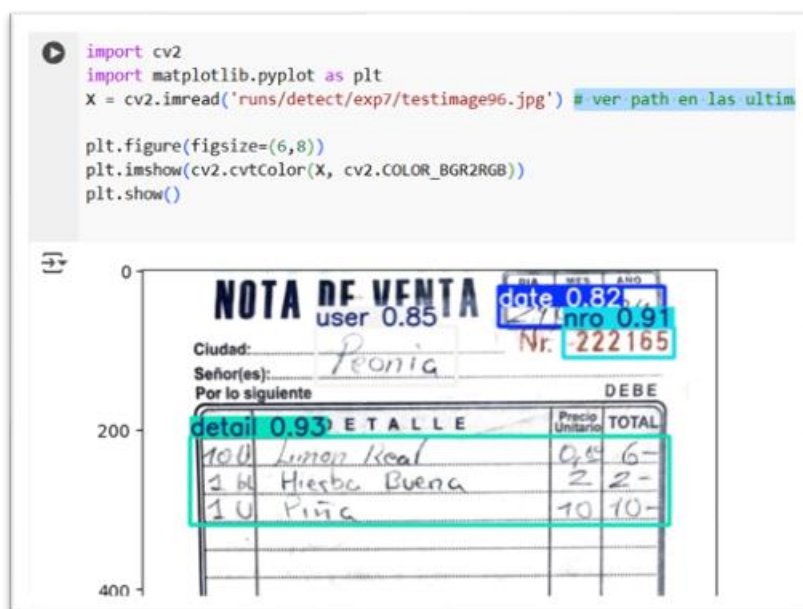
Class	Images	Instances	P	R	mAP50	mAP50-95: 100% 3/3
all	65	252	0.994	0.994	0.993	0.735
date	65	62	0.981	1	0.989	0.672
nro	65	64	0.996	1	0.995	0.714
user	65	63	0.998	0.984	0.995	0.664
detail	65	63	1	0.992	0.995	0.892

Results saved to runs/train/exp2

En esta parte del proyecto se entrenó varias veces hasta llegar a un entrenamiento optimo, como se muestra en la Figura 34, el modelo logra identificar las distintas áreas de interés en una nota de venta, de las cuatro clases: date, user, nro, detail con porcentajes de confianza superior a 80% en todas las clases algunas alcanzando el 90% siendo satisfactorio.

Figura 34

Detección con YoloV5 de nota de venta



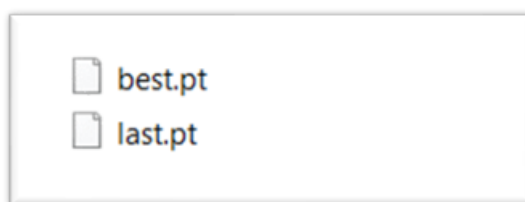
Se aprovecho las imágenes en test que se dejaron la carpeta dataset para hacer las pruebas de detección con los pesos seleccionados como mejor.

La calidad de las detecciones mejora directamente con el tamaño y la diversidad del conjunto de datos utilizado, así como con los ajustes realizados en los hiperparámetros del entrenamiento (como la tasa de aprendizaje, el número de épocas y el tamaño del lote). Si bien los resultados actuales son satisfactorios, se plantea la posibilidad de seguir ampliando el dataset o experimentar con diferentes configuraciones para incrementar aún más la precisión del modelo.

Una vez finalizado el entrenamiento, se exportan los pesos del modelo obtenidos. Estos archivos pueden utilizarse tanto para realizar pruebas locales como para continuar el proceso de refinamiento del modelo. En la Figura 35 se ilustra cómo estos pesos se aplicarán para validar el desempeño del modelo fuera del entorno de entrenamiento.

Figura 35

Pesos de entrenamiento de YoloV5 áreas



3.4.3 Iteración 3.

3.4.3.1 Códigos Python Para Subsecciones Con Yolov5.

En esta iteración aprovecharemos las detecciones del anterior entrenamiento para extraer las áreas detalle para el entrenamiento de esta iteración como vemos en la Figura 36.

Figura 36

Extraer Áreas Detectadas

```
def convertir_yolo_a_coordenadas(x_centro, y_centro, ancho, alto, img_ancho, img_alto):
    x1 = int((x_centro - ancho / 2) * img_ancho)
    y1 = int((y_centro - alto / 2) * img_alto)
    x2 = int((x_centro + ancho / 2) * img_ancho)
    y2 = int((y_centro + alto / 2) * img_alto)
    return x1, y1, x2, y2

def extraer_areas_yolo(ruta_imagen, ruta_etiquetas, carpeta_salida):
    imagen = cv2.imread(ruta_imagen)
    alto, ancho, _ = imagen.shape

    # Leer el archivo de etiquetas en formato YOLO
    with open(ruta_etiquetas, 'r') as archivo:
        lineas = archivo.readlines()

    # Crear la carpeta de salida si no existe
    os.makedirs(carpeta_salida, exist_ok=True)

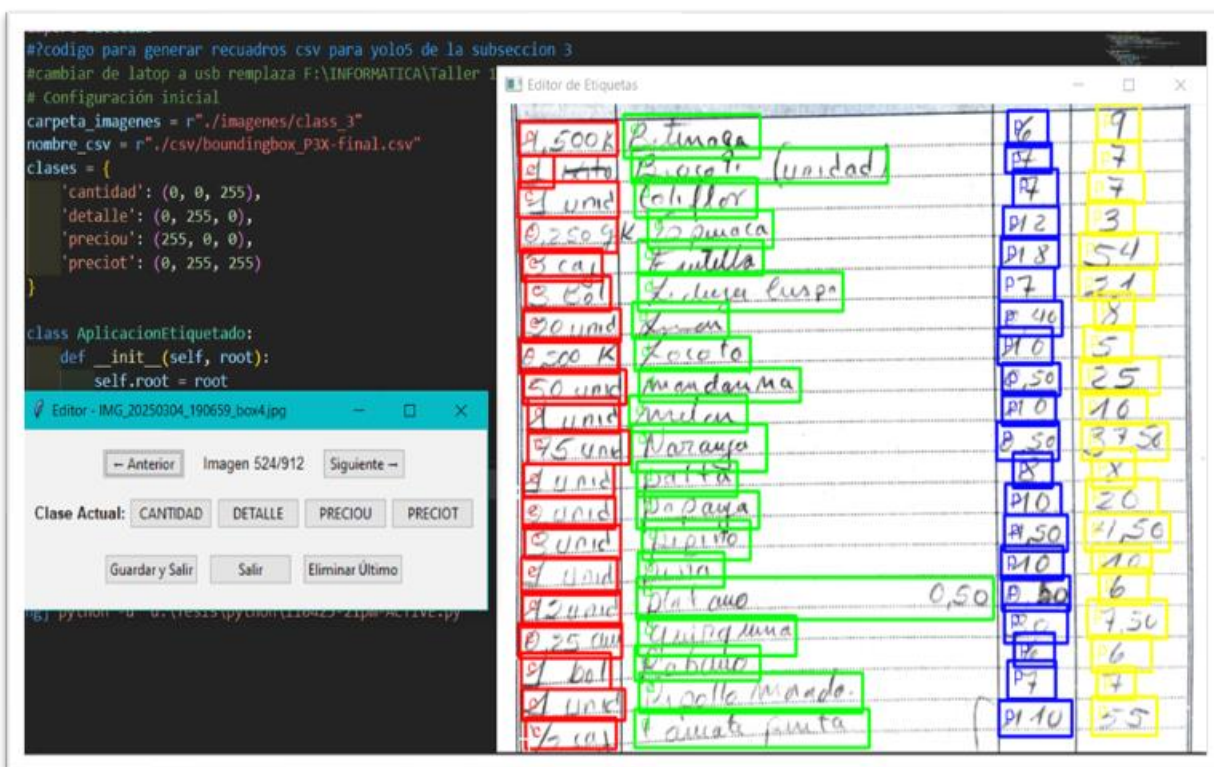
    # Procesar cada línea del archivo de etiquetas
    for i, linea in enumerate(lineas):
        partes = linea.strip().split()
        id_clase = int(partes[0])
        x_centro, y_centro, ancho_caja, alto_caja = map(float, partes[1:])
        x1, y1, x2, y2 = convertir_yolo_a_coordenadas(x_centro, y_centro, ancho_caja, alto_caja, ancho, alto)

        # Recortar el área de la imagen correspondiente a la caja detectada
        recorte = imagen[y1:y2, x1:x2]
        nombre_archivo = f'area_{i}_clase_{id_clase}.jpg'
        cv2.imwrite(os.path.join(carpeta_salida, nombre_archivo), recorte)
```

Se diseñó un script, como en la anterior iteración se generaron bounding boxes para el entrenamiento de YoloV5, para esta operación se utilizará la detección del área 3 que es la parte de detalle en la nota de venta manuscrita como se observa en la Figura 37, a diferencia de la anterior script esta tendrá cuatro clases a detectarse (cantidad, detalle, precioU y precioT), que representan las columnas y tendrá filas que representarán el producto comprado con su cantidad correspondiente precioU y precioT con un mínimo de una fila válida y máximo de 19, máximo observado en los recibos, se prefirió separar por celdas ya que las posiciones de escritura muchas veces sobrepasan las celdas impresas, invadiendo y conflictuando para la detección con varios trazos en toda dirección.

Figura 37

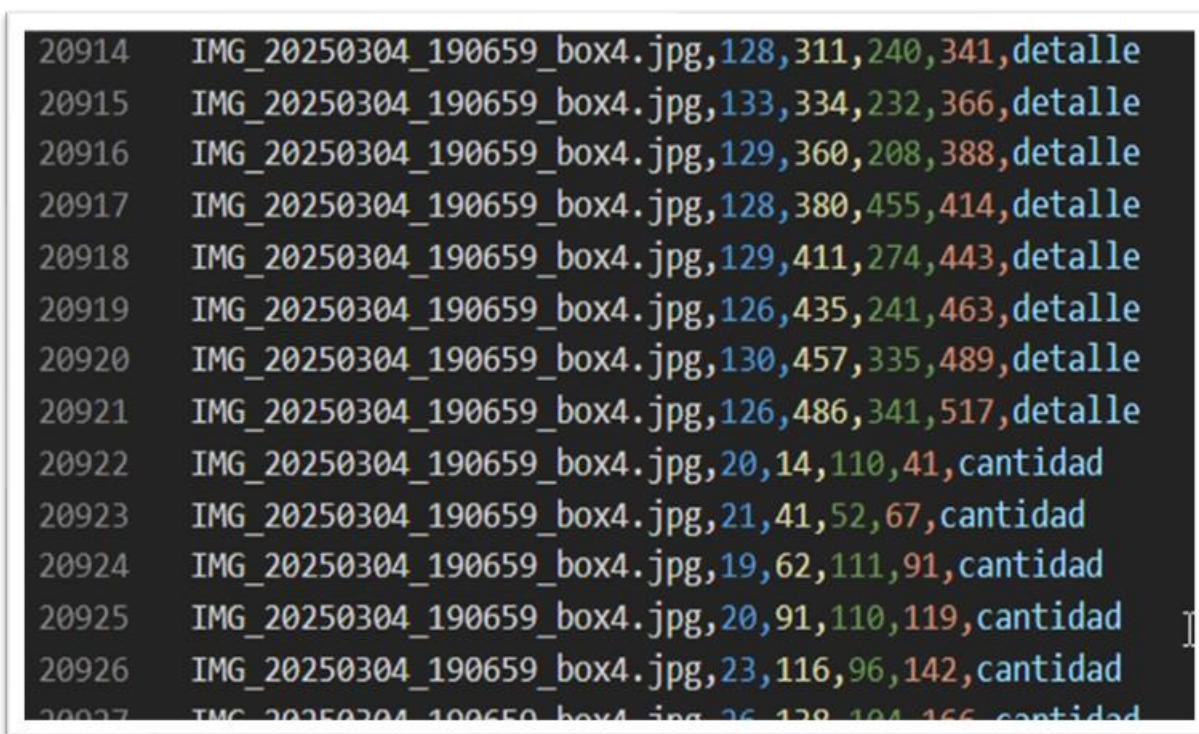
Notas de venta detección de celdas del área detalle



En la Figura 38, se observa las coordenadas que se capturaron para generar áreas x1, x2, y1, y2 que generan el recuadros específicos para el entrenamiento, con el respectivo nombre de la de la clase que corresponde (cantidad, descripcion, precio unitario y precio total), se aprovecho el scrip creado anteriormente adaptandolo a los requerimientos en esta iteracion, se añadio mejoras que permiten multiples recuadros, la capacidad de eliminar la ultima celda con coordenadas generadas y saltar al ultimo procesado en el csv para agilizar el proceso de creacion de boundingboxes, se añadio la posibilidad de utilizar teclado con botenes designados para su uso, ayudando y acelerando la selección de áreas.

Figura 38

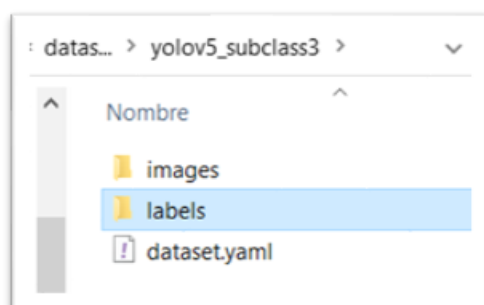
Áreas de detalle en csv capturados



The image shows a screenshot of a CSV file with 13 rows of data. Each row contains an ID, a file path, a bounding box in normalized coordinates (x1, y1, x2, y2), and a label. The labels are either 'detalle' or 'cantidad'. The bounding boxes are color-coded: blue for x1, green for y1, red for x2, and orange for y2.

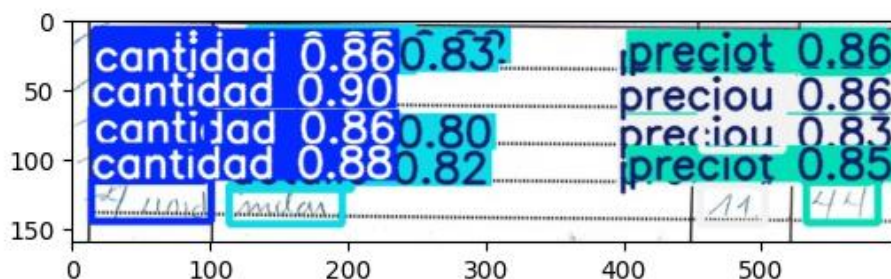
ID	File Path	Bounding Box (x1, y1, x2, y2)	Label
20914	IMG_20250304_190659_box4.jpg	128, 311, 240, 341	detalle
20915	IMG_20250304_190659_box4.jpg	133, 334, 232, 366	detalle
20916	IMG_20250304_190659_box4.jpg	129, 360, 208, 388	detalle
20917	IMG_20250304_190659_box4.jpg	128, 380, 455, 414	detalle
20918	IMG_20250304_190659_box4.jpg	129, 411, 274, 443	detalle
20919	IMG_20250304_190659_box4.jpg	126, 435, 241, 463	detalle
20920	IMG_20250304_190659_box4.jpg	130, 457, 335, 489	detalle
20921	IMG_20250304_190659_box4.jpg	126, 486, 341, 517	detalle
20922	IMG_20250304_190659_box4.jpg	20, 14, 110, 41	cantidad
20923	IMG_20250304_190659_box4.jpg	21, 41, 52, 67	cantidad
20924	IMG_20250304_190659_box4.jpg	19, 62, 111, 91	cantidad
20925	IMG_20250304_190659_box4.jpg	20, 91, 110, 119	cantidad
20926	IMG_20250304_190659_box4.jpg	23, 116, 96, 142	cantidad
20927	IMG_20250304_190659_box4.jpg	26, 138, 104, 166	cantidad

Una vez se realizó el conjunto de dataset, se procede a convertir las coordenadas (x1, x2, y1, y2) a coordenadas aceptadas en el entrenamiento (clase C, Xcenter, Ycenter, ancho y altura), se hace uso de reglas matematicas usadas en la ecuación (9) tenemos un script funcional que nos ayudara con algunas correcciones como el desbordamiento de area ajustandolos a valores dentro de la imagen, se separaron en el mismo formato anterior una carpeta que contenga dos carpetas y un archivo, images, labels y dataset.yaml, para el entrenamiento como se ve en la Figura 39, separados en 3 carpetas: train 75%, val 15%, test 15%, se definió que eran los mejores promedios para el entrenamiento de detección.

Figura 39*Formato de Dataset de Detalle***3.4.3.2 Entrenamiento De Detección En Subáreas.**

Utilizando Colab para el entrenamiento en la detección de áreas con YoloV5 como en la anterior iteración nos ayudaremos de las configuraciones ya hechas con la diferencia de que las epochs serán de 80 y batch de 32 al ser más áreas a detectar se notó una sustancial mejora con algunas configuraciones nos permite lograr resultados óptimos.

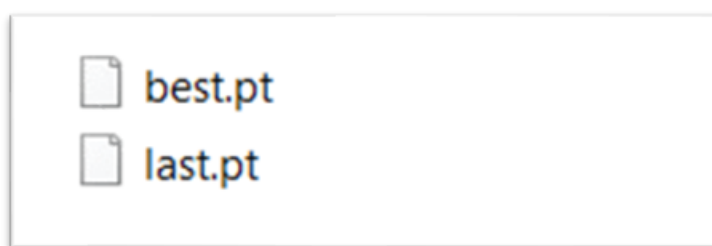
La detección es posible, tenemos una tasa de confianza promedio aceptable a buena con mínimos de 75% a máximos de 90% como se ve en la Figura 40, obteniendo una confianza en la detección aceptable damos por casi finalizada esta iteración.

Figura 40*Detección de YoloV5 celdas detalle*

Una vez finalizado el entrenamiento, se seleccionan los pesos del modelo obtenidos. Estos archivos pueden utilizarse tanto para realizar pruebas locales como para continuar el proceso de entrenamiento del modelo. En la Figura 41, podemos observar estos descargados de Google colab, nos ayudaran a la detección y exportación de las celdas del área detalle con una confianza aceptable.

Figura 41

Pesos de entrenamiento de área detalle



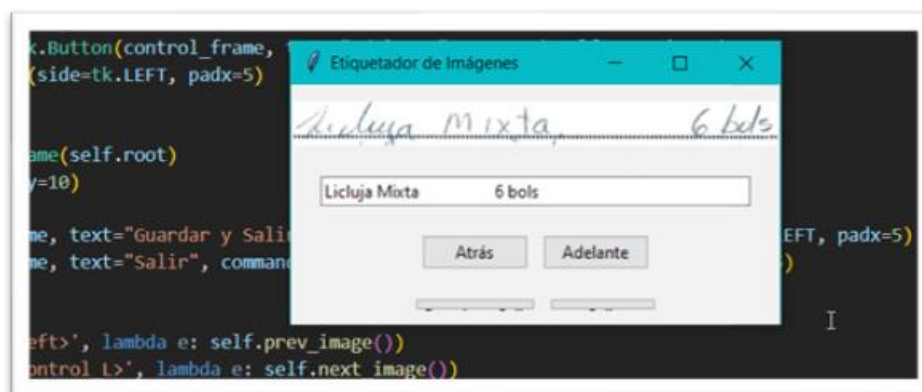
3.4.4 Iteración 4.

3.4.4.1 Dataset Para El Entrenamiento De La CRNN.

En esta iteración, después del entrenamiento en yolov5 se extraen las áreas detectadas para el entrenamiento, se crea un nuevo script con las características que esta cargara la carpeta de las detecciones del área detalle, y se crea un txt donde se almacenara los datos capturados, al ejecutarse este mostrara un entorno grafico como se ve en la Figura 42, donde las ubicaciones serán así la parte superior tendrá la imagen abajo una celda donde colocaremos el valor visualizado con los errores de tipeo y distancias observables.

Figura 42

Captura de datos para el CRNN



Los datos se almacenan de la siguiente forma dentro un Txt: primero el nombre de la imagen, un espacio y luego el valor de la imagen en texto conteniendo espacios visualizados y lo más exacto a lo que se ve como en la figura (Figura 43). Servirán para formar el dataset que se requiere para en entrenamiento del CRNN.

Figura 43

Archivo Txt con datos capturados

```

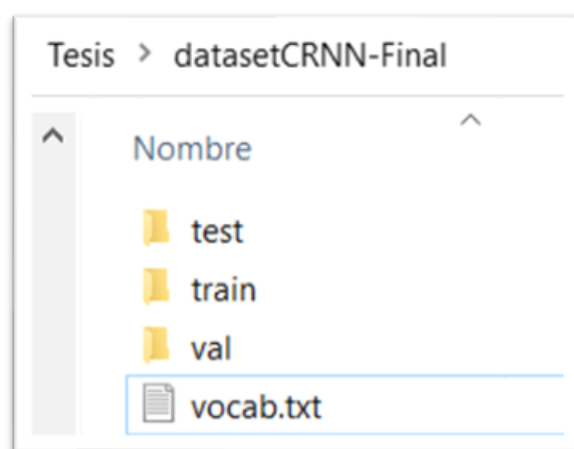
7497 IMG_20250302_131244_box4_17.jpg 1 baij
7498 IMG_20250302_131244_box4_18.jpg 1 bad
7499 IMG_20250302_131244_box4_2.jpg 1 K
7500 IMG_20250302_131244_box4_3.jpg 6 K
7501 IMG_20250302_195034_box4_20.jpg piña 10 unid
7502 IMG_20250302_195034_box4_21.jpg huba buuua
7503 IMG_20250302_195100_box4_13.jpg palta 4 unid
7504 IMG_20250302_195100_box4_14.jpg papa H
7505 IMG_20250302_195100_box4_15.jpg piujl
7506 IMG_20250302_195100_box4_16.jpg papa iwilla
7507 IMG_20250302_195100_box4_17.jpg Narauja 114 unid
7508 IMG_20250302_195100_box4_18.jpg towate
7509 IMG_20250302_195100_box4_19.jpg Locoto
7510 IMG_20250302_195100_box4_20.jpg quuquña
7511 IMG_20250302_195100_box4_21.jpg Cibollin
7512 IMG_20250302_195100_box4_22.jpg ajo
7513 IMG_20250302_195100_box4_23.jpg liclija Morada 1 unid
7514 IMG_20250302_195100_box4_24.jpg Lcluja Siñorta 1 unid

```

En esta parte ya no se requieren coordinas y su transformación a datos para el entrenamiento, pero si requieren un orden y forma como se ve en la Figura 44, se tiene 3 carpetas train con 75% de imágenes para el entrenamiento, val con 15% de imágenes para el entrenamiento y test con el 10%, dentro de cada carpeta se tendrá un Txt que contendrá el valor para el entrenamiento como se ve en la Figura 43 y Vocab.txt de será los caracteres y valores que son: letra mayúsculas, minúsculas, numero, símbolos que nos permitirá indicar los valores que esperamos en el entrenamiento de la CRNN.

Figura 44

Formato de dataset para entrenamiento CRNN



3.4.4.2 Entrenamiento De La CRNN.

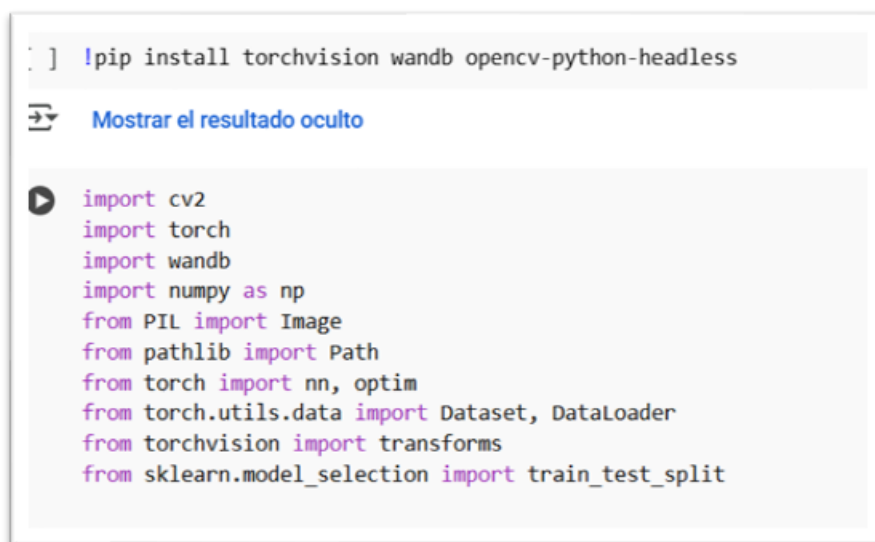
En esta sección se realizará el entrenamiento de un modelo CRNN (Convolutional Recurrent Neural Network) para el reconocimiento de texto manuscrito, específicamente letras y números presentes en celdas que se detectaron previamente. Para ello, se define una clase de configuraciones con los parámetros para el funcionamiento optimo del modelo, como una ruta

del dataset, tamaño de lote, tasa de aprendizaje, dimensiones de las imágenes y número de épocas, muy similar realizados en las iteraciones anteriores.

Como parte inicial subiremos a colab el dataset de entrenamiento, descomprimiendo el archivo Zip²¹, instalaremos las librerías necesarias para poder empezar el entrenamiento como vemos en la Figura 45, donde Torchvision es una librería que complementa Pytorch, es usada porque ayuda al procesamiento de imágenes y transformaciones necesarias para la red neuronal, Wand es una herramienta para monitorear, guardar y visualizar entrenamientos de los modelos y opencv-python-headless versión sin GUI de OpenCV, ideal para servidores o notebooks la usamos para lectura y procesamiento de imágenes.

Figura 45

Instalar Librerías e Importaciones



The image shows a Jupyter Notebook cell with two parts. The top part is a terminal output showing the command `!pip install torchvision wandb opencv-python-headless` being executed. Below the terminal output is a button labeled "Mostrar el resultado oculto". The bottom part of the cell contains Python code for importing various libraries: `import cv2`, `import torch`, `import wandb`, `import numpy as np`, `from PIL import Image`, `from pathlib import Path`, `from torch import nn, optim`, `from torch.utils.data import Dataset, DataLoader`, `from torchvision import transforms`, and `from sklearn.model_selection import train_test_split`.

```
[ ] !pip install torchvision wandb opencv-python-headless
```

Mostrar el resultado oculto

```
import cv2
import torch
import wandb
import numpy as np
from PIL import Image
from pathlib import Path
from torch import nn, optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from sklearn.model_selection import train_test_split
```

²¹ Zip archivo de compresión utilizado por su facilidad de descompresión en colab

En la Figura 46 vemos: el `batch_size` que es el tamaño de grupos de imágenes o lote de grupos de imágenes para el entrenamiento, `lr` (learning rate) que es la tasa de aprendizaje inicial, `epochs` las épocas que son las iteraciones de entrenamiento de lotes, en la imagen vemos los valores que fueron más eficientes en la detección sin sobrecargar el entrenamiento.

Figura 46

Configuración inicial para el entrenamiento CRNN

```
class Config:
    data_dir = "/content/datasetCRNN-Final"
    batch_size = 64
    lr = 0.001
    epochs = 50
    img_height = 64
    img_width = 384
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

El modelo CRNN toma las imágenes que serán de 64 x 384 definidas como tamaño de entrenamiento, se convierten en grises, se recorta y ajusta para tener un tamaño único si fuera necesario, agregando un padding sin deformar la imagen.

Luego devuelve una imagen preprocesada como tensor y su texto codificado en números, listo para entrenar la CRNN.

En la Figura 47, define la arquitectura de la CRNN. En su constructor “`__init__()`”, se establece la secuencia de capas convolucionales para la extracción de características, una red recurrente LSTM bidireccional para el modelado secuencial, y una capa lineal final que proyecta las salidas a un espacio correspondiente al vocabulario definido en `vocab.txt`.

El método `forward`, por su parte, describe el recorrido que sigue una imagen de entrada a través de estas capas, transformándola progresivamente hasta obtener una secuencia de

predicciones. Esta clase es ejecutada continuamente durante el entrenamiento, cada vez que se alimenta un lote de datos al modelo, para clasificar cada parte de la imagen como un carácter.

Figura 47

Class CRNN arquitectura del modelo CRNN

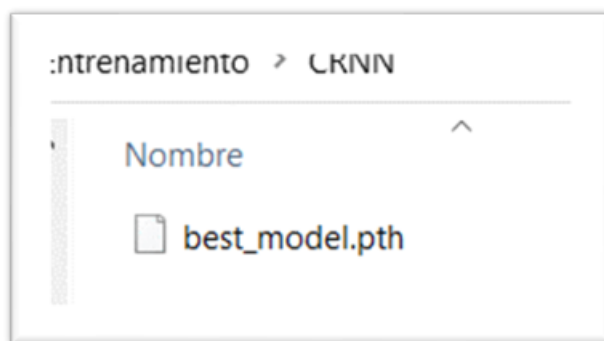
```
class CRNN(nn.Module):
    def __init__(self, vocab_size):
        super().__init__()
        self.cnn = nn.Sequential(
            nn.Conv2d(1, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(128, 256, 3, padding=1), nn.BatchNorm2d(256), nn.ReLU(),
            nn.Conv2d(256, 512, 3, padding=1), nn.BatchNorm2d(512), nn.ReLU(),
            nn.AdaptiveAvgPool2d((1, None))
        )
        self.rnn = nn.LSTM(512, 256, bidirectional=True, num_layers=2)
        self.fc = nn.Linear(512, vocab_size)

    def forward(self, x):
        x = self.cnn(x)
        x = x.squeeze(2).permute(2, 0, 1)
        x, _ = self.rnn(x)
        return self.fc(x)
```

Después del entrenamiento se extraen los pesos de Colab como se ve en la Figura 49 nos servirá para reentrenar o utilizar en la aplicación móvil.

Figura 49

Pesos de entrenamiento de la CRNN



3.4.5 Iteración 5.

3.4.5.1 Corrección Mediante Diccionario De Palabras En Python.

Dado que el modelo de reconocimiento de texto implementado (CRNN, Convolutional Recurrent Neural Network) puede producir resultados con errores ortográficos o semánticos debido a la calidad de las imágenes o la naturaleza confusa de la escritura manuscrita, se incorpora una etapa de corrección semántica posterior al reconocimiento.

Esta etapa consiste en la utilización de un diccionario de palabras previamente definido, implementado en Python, con el objetivo de validar y corregir los términos extraídos por el modelo. Para ello, se emplean algoritmos de similitud léxica, como la distancia de Levenshtein o el método `get_close_matches` de la biblioteca `difflib`, que permiten identificar la palabra más próxima en el diccionario a partir del texto detectado.

Cuando una palabra reconocida no se encuentra explícitamente en el diccionario, el sistema sugiere automáticamente una corrección basada en la palabra más similar y coherente desde el punto de vista semántico. Esta técnica permite no solo mejorar la precisión general del sistema de reconocimiento, sino también garantizar una mayor coherencia y legibilidad en los textos finales generados.

3.4.5.2 Estructura De Notas Digitales Mediante Detección De Posiciones.

Una vez identificados y corregidos los textos mediante técnicas de OCR y corrección semántica, se procede a la estructuración lógica de la información contenida en la nota de venta manuscrita. Esta etapa tiene como objetivo organizar los elementos detectados en un formato estructurado, que permita representar digitalmente la nota de manera coherente y reutilizable.

Para ello, se emplea un modelo YOLOv5 previamente entrenado para detectar regiones clave del documento, tales como la fecha, número de serie, destinatario y el área de detalle de productos o servicios. Posteriormente, sobre la región del detalle detectado, se aplica un segundo modelo YOLOv5 especializado, encargado de identificar las celdas que conforman una tabla de cuatro columnas: cantidad, descripción del producto, precio unitario y precio total.

Una vez obtenidas las posiciones (coordenadas) de cada celda, se realiza un proceso de ordenamiento y asociación espacial, utilizando principalmente la coordenada vertical (eje Y) como criterio principal para agrupar elementos en filas, y la coordenada horizontal (eje X) para asignar los elementos a la columna correspondiente. De esta forma, se reconstruye lógicamente la tabla y se vinculan los textos reconocidos a su ubicación respectiva.

Finalmente, toda la información estructurada —incluyendo los datos generales y el contenido tabular— se convierte una lista de pares, lo cual permite su integración con otros sistemas, almacenamiento en bases de datos o visualización digital. Este formato garantiza una representación fiel y organizada de la nota de venta original.

3.4.6 Iteración 6.

3.4.6.1 Conversión De Modelos YOLOv5 a TFLite para móviles.

Los modelos de detección y reconocimiento utilizados en este proyecto fueron entrenados utilizando Google Colab, ya que esta plataforma ofrece una mayor potencia computacional, especialmente útil para entrenamientos de modelos complejos como YOLOv5 y CRNN. Colab permite el uso de GPUs y TPUs, lo que optimiza significativamente los tiempos de entrenamiento y hace viable trabajar con grandes volúmenes de datos. Una vez completado el entrenamiento de los modelos —YOLOv5 para la detección de regiones clave y CRNN para el reconocimiento de texto, se procedió a su conversión y optimización a TensorFlow Lite (*.tflite), un formato compatible con aplicaciones móviles desarrolladas en Android Studio/ Kotlin.

Dado que los intentos iniciales de conversión mediante ONNX presentaron conflictos de versiones y otras incompatibilidades, se optó por un proceso alternativo directo desde PyTorch a TensorFlow Lite, lo que permitió superar estos inconvenientes de manera eficiente.

Exportación directa a TensorFlow: Para evitar los problemas derivados de la conversión a ONNX, se utilizó una función de exportación directa desde PyTorch a TensorFlow guardado en saved_model como se ve Figura 50. Este enfoque optimiza el flujo de trabajo y garantiza que el

modelo sea compatible con el formato TensorFlow, guardamos el save model para ser convertido.

Figura 50

Conversión Pt a SaveModel de TFLite

```
python export.py --weights /content/best.pt --include tflite --data /content/data/data.yaml --img 960 --include saved_model
```

tfc3_3 (TFC3)	(1, 30, 30, 512)	1,181,184	tf_conv_25[0][0]
tfspfp (TFSPFP)	(1, 30, 30, 512)	656,128	tfc3_3[0][0]
tf_conv_33 (TFConv)	(1, 30, 30, 256)	131,328	tfspfp[0][0]
tf_upsample (TFUpsample)	(1, 60, 60, 256)	0	tf_conv_33[0][0]
tf_concat (TFConcat)	(1, 60, 60, 512)	0	tf_upsample[0][0]... tfc3_2[0][0]
tfc3_4 (TFC3)	(1, 60, 60, 256)	361,216	tf_concat[0][0]
tf_conv_39 (TFConv)	(1, 60, 60, 128)	32,896	tfc3_4[0][0]
tf_upsample_1 (TFUpsample)	(1, 120, 120, 128)	0	tf_conv_39[0][0]
tf_concat_1 (TFConcat)	(1, 120, 120, 256)	0	tf_upsample_1[0]... tfc3_1[0][0]
tfc3_5 (TFC3)	(1, 120, 120, 128)	90,496	tf_concat_1[0][0]

Este comando genera un modelo en formato saved_model de TensorFlow, y simultáneamente crea una versión optimizada del modelo en formato TensorFlow Lite (.tflite).

La exportación a TFLite: una vez generado el modelo en formato save model de TensorFlow, se utiliza el conversor oficial de TensorFlow para obtener el archivo *.tflite como se ve Figura 51, que es compatible con dispositivos móviles. El código utilizado para la conversión en Colab está en la Figura 51, donde se utilizará una configuración float16 que es la más optima ya que float 32 es muy pesada para ordenadores o celulares de más potencia y float16 es muy inestable por que reduce mucho las capas de predicción, siendo float16 el escogido como se ve.

Figura 51

Conversión SaveModel TFLite a TFLite float16

```
#SavedModel a .tflite float16
import tensorflow as tf

saved_model_dir = '/content/best_saved_model'

converter = tf.lite.TFLiteConverter.from_saved_model(saved_model_dir)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.target_spec.supported_types = [tf.float16]

tflite_fp16_model = converter.convert()

# Guardar el modelo
with open('/content/best-fp16.tflite', 'wb') as f:
    f.write(tflite_fp16_model)

print(" Modelo float16 exportado como best-fp16.tflite")
```

WARNING:absl:Importing a function (__inference_pruned_11912) with ops with i
 WARNING:absl:Importing a function (__inference_pruned_11912) with ops with i
 Modelo float16 exportado como best-fp16.tflite

En la Figura 52 tenemos una prueba, junto con su gráfica y confianza como se ve en la Figura 53, se observa una reducción en el nivel de confianza siendo aun aceptable.

Figura 52

Detección con TFLite Float16

```
python detect.py --weights /content/best-fp16.tflite --source /content/data/images/test/testimage169.jpg --data /content/data/data.yaml
```

detect: weights=['/content/best-fp16.tflite'], source=/content/data/images/test/testimage169.jpg, data=/content/data/data.yaml, imsz=[9
 YOLOv5 v7.0-419-gcd44191c Python-3.11.12 torch-2.6.0+cu124 CUDA:0 (Tesla T4, 15095MiB)

2025-06-03 20:18:41.189644: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to register cuFFT factory: Attempting
 WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
 E0000 00:00:1748981921.209823 9670 cuda_dnn.cc:8310] Unable to register cuDNN factory: Attempting to register factory for plugin cuDN
 E0000 00:00:1748981921.216050 9670 cuda_blas.cc:1418] Unable to register cuBLAS factory: Attempting to register factory for plugin cu
 Loading /content/best-fp16.tflite for TensorFlow Lite inference...
 INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
 image 1/1 /content/data/images/test/testimage169.jpg: 960x960 1 date, 1 nro, 1 user, 1 detail, 965.1ms
 Speed: 24.7ms pre-process, 965.1ms inference, 160.7ms NMS per image at shape (1, 3, 960, 960)
 Results saved to runs/detect/resultados_fp16

Figura 53

Visualización de Detección y Confianza con TFLITE

```
from PIL import Image
display(Image.open('runs/detect/resultados_fp16/valimage56.jpg'))
```

NOTA DE VENTA

date 0.87
user 0.75
nro 0.85
Nº 220044

Señor(es): Atix

Por lo siguiente: DEBE

DETALLE		P.Unit.	TOTAL
0.500	ajo	30	15
0.400	albahaca	16	8
0.200	ajo		2
4 K	Carrot	12	48
8 K	Ebolla Blanca	7	56
2 K	Ebolla Roja	7	14
1 bany	Champinano Ostra	20	20
1 bany	Champinano farao	22	22

Este proceso se realiza también con el otro entrenamiento en yoloV5, ya que el proceso es similar logrando una conversión aceptable.

3.4.6.2 Conversión De Modelos CRNN a PyTorch Mobile.

El modelo CRNN, utilizado para el reconocimiento de texto en las notas de venta, fue entrenado originalmente en PyTorch. Al igual que con el modelo YOLOv5, la conversión del modelo CRNN al formato compatible con dispositivos móviles fue esencial para poder implementarlo en la aplicación móvil. Dado que el modelo estaba en formato *.pth, se necesitaba convertirlo a PyTorch Mobile (.pt) para su ejecución en dispositivos con recursos medios.

El proceso de conversión se realizó en varias fases:

a. Adaptación del modelo CRNN a PyTorch

El proceso de adaptación se detectaron problemas en el modelo original, como se ve en la Tabla 3, se observa se reemplazó AdaptiveAvgPool2d por AvgPool2d ya que no soporta correctamente esta función se adaptó para tener un comportamiento y similar. LSTM para ser utilizado correctamente debe usarse proj_size=0 y habilitar SELECT_TF_OPS agregando un soporte extendido para ser funcional.

Tabla 3

Problemas en adaptación CRNN a PT PyTorch

Característica	Incompatibilidad	Solución
AdaptiveAvgPool2d ((1, None))	Dimensiones dinámicas en el pooling	Reemplazar por AvgPool2d(kernel_size=(16, 1))
LSTM bidireccional	Soporte limitado en TFLite	Usar proj_size=0 y SELECT_TF_OPS
Normalización en preproceso	Inconsistencia entre entrenamiento/inferencia	Integrar normalización en el modelo ((x - 0.5) / 0.5)

b. Exportación TorchScript

Una vez determinado los problemas que impiden la conversión a PyTorch procedemos a la conversión como se ve en la Figura 54 fijando una entrada estática de dimensiones ([1 , 1 , 64 , 384]) para tener una compatibilidad en la aplicación móvil.

Figura 54

Configuración de entrada dinámica a una estática

```
> dummy_input = torch.randn(1, 1, 64, 384)
   traced_model = torch.jit.trace(model, dummy_input, check_trace=True)
   traced_model.save("crnn_static.pt")
```

c. Conversión a PyTorch con optimizaciones

Se empleó la herramienta PyTorch para transformar el modelo CRNN previamente entrenado a su versión compatible. Durante esta fase, se especificó una forma de entrada fija ([1, 1, 64, 384]), la cual es representativa de las dimensiones que tendrá la entrada durante la inferencia en el dispositivo.

Debido a que el modelo CRNN incluye componentes como capas LSTM bidireccionales, fue necesario activar operadores no nativos mediante la opción `SELECT_TF_OPS`. Esta configuración permite que la inferencia utilice ciertas operaciones avanzadas disponibles en la versión estándar, asegurando así la compatibilidad completa con la arquitectura del modelo.

Con el fin de reducir el tamaño del modelo y optimizar su ejecución en dispositivos con limitaciones de hardware (memoria y procesamiento), se aplicó cuantización post entrenamiento utilizando la estrategia `tf.lite.Optimize.DEFAULT`. Esta técnica convierte los pesos y activaciones del modelo de precisión flotante (float32) a formatos de menor tamaño como float16, lo que típicamente resulta en una

disminución del tamaño del archivo hasta en un 75%, con un impacto mínimo en la precisión del reconocimiento.

Tras aplicar las configuraciones anteriores, el modelo resultante se revisó y probó como se ve en la Figura 55 con la extensión .pt, el cual puede ser utilizado dentro de una aplicación móvil desarrollada en Kotlin. Este modelo está preparado para realizar predicciones eficientes directamente en el dispositivo, sin necesidad de conexión a internet ni recursos en la nube.

Figura 55

Verificación de conversión

```
with open('/content/datasetCRNN-Final/vocab.txt', 'r', encoding='utf-8') as f:
    vocab = [line.strip() for line in f]
    char_to_idx = {c: i for i, c in enumerate(vocab)}
    idx_to_char = {i: c for i, c in enumerate(vocab)}
    blank_idx = char_to_idx.get('<blank>', len(vocab) - 1)

#Modelo convertido para cel
model = torch.jit.load("crnn_model.pt", map_location="cpu")
model.eval()
```

```
RecursiveScriptModule(
  original_name=CRNN
  (cnn): RecursiveScriptModule(
    original_name=Sequential
    (0): RecursiveScriptModule(original_name=Conv2d)
    (1): RecursiveScriptModule(original_name=ReLU)
    (2): RecursiveScriptModule(original_name=MaxPool2d)
    (3): RecursiveScriptModule(original_name=Conv2d)
    (4): RecursiveScriptModule(original_name=ReLU)
    (5): RecursiveScriptModule(original_name=MaxPool2d)
    (6): RecursiveScriptModule(original_name=Conv2d)
    (7): RecursiveScriptModule(original_name=BatchNorm2d)
    (8): RecursiveScriptModule(original_name=ReLU)
    (9): RecursiveScriptModule(original_name=Conv2d)
    (10): RecursiveScriptModule(original_name=BatchNorm2d)
    (11): RecursiveScriptModule(original_name=ReLU)
    (12): RecursiveScriptModule(original_name=AdaptiveAvgPool2d)
  )
  (rnn): RecursiveScriptModule(original_name=LSTM)
  (fc): RecursiveScriptModule(original_name=Linear)
)
```

3.5 FASE DE ESTABILIZACION

3.5.1 Iteración 7.

En esta iteración se desarrolla una aplicación móvil utilizando Android Studio con Kotlin compose jetpack como lenguaje principal, con el objetivo de integrar el flujo completo de procesamiento de notas de venta manuscritas mediante modelos previamente entrenados y convertidos a TensorFlow Lite. La aplicación se estructura en tres componentes funcionales: interfaz de usuario (frontend), el procesamiento interno de datos tras la captura de la imagen, y presentación de estadísticas mediante una navegación intuitiva y creación de PDF para exportación de notas a clientes. El reconocimiento de texto y el procesamiento semántico se desarrollan completamente en Kotlin, utilizando estructuras de datos, algoritmos de corrección heurística y una base de datos local con SQLite para mejorar el rendimiento y la portabilidad en dispositivos de gama baja.

3.5.1.1 Diseño frontend de la aplicación

La interfaz de la aplicación está diseñada con un enfoque minimalista y funcional, centrado en garantizar una experiencia fluida y adaptable a distintas resoluciones de pantalla. El frontend representa la capa de presentación del sistema y permite que el usuario interactúe con las funcionalidades de captura, reconocimiento y digitalización de notas manuscritas.

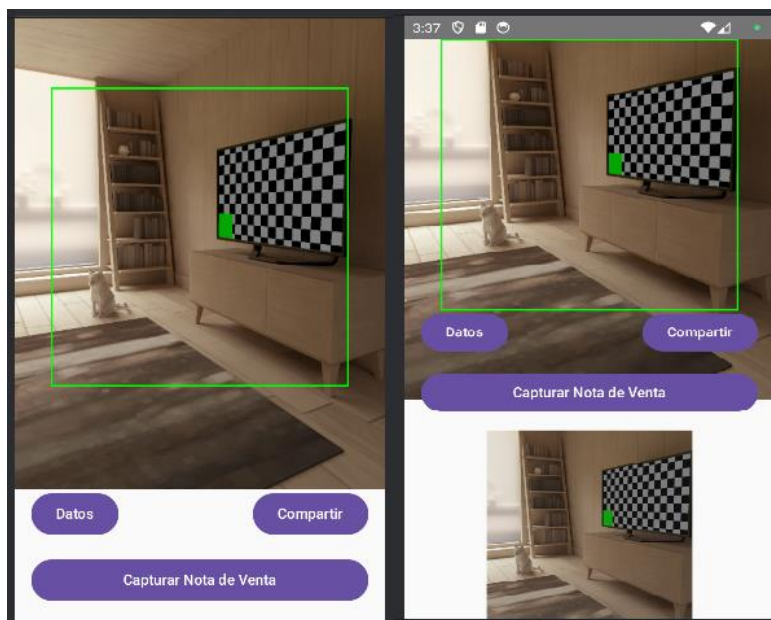
Hace uso de tres pantallas:

Pantalla de Inicio. - en esta se tiene tres partes la parte superior donde se hace uso de CameraX para hacer uso de la cámara trasera del celular para visualizar la nota de venta a capturar como se ve en la Figura 56, esta cuenta con una grilla que representa la mejor área de

toma. En la parte inferior se muestra los botones la centra de captura de la imagen, la izquierda que muestra otra pantalla donde se puede usar datos estadísticos para análisis de ventas, botón derecho que abre una lista de documentos convertidos a pdf para que pueda ser compartido a clientes, los diferentes medios que el usuario vea necesario (WhatsApp, Telegram, Gmail, Messenger, etc.).

Figura 56

Pruebas Frontend Pantalla Inicial



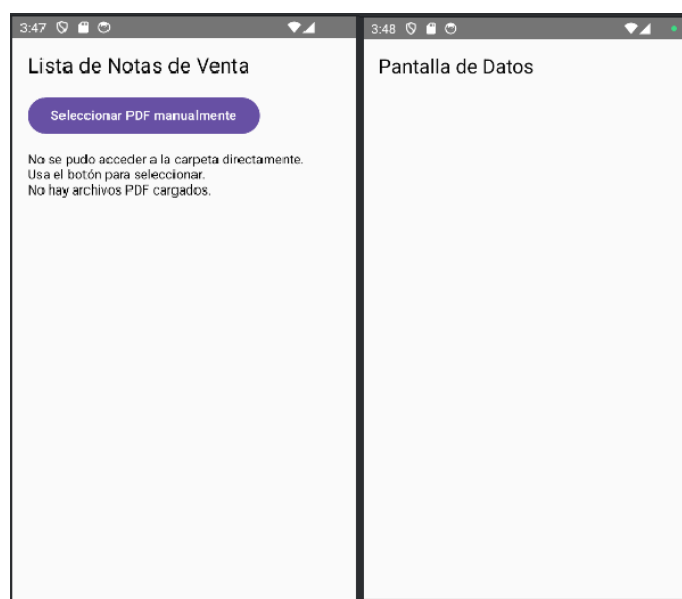
Pantalla de Datos de Venta. - en esta pantalla que se accede desde el botón Datos desde la pantalla inicial, nos ayudara para mostrar valores y análisis de datos básicos de ventas de parte del usuario como se observa en la Figura 57, acá se mostraran datos relacionados a la venta del día, por mes o año, también productos más vendidos.

Pantalla de Exportación. - en esta pantalla que se accede por parte del botón derecho como se puede observar en la Figura 57, esta nos permitirá listar en pdf la nota de venta que se

genere en orden de ultimo creado para poder ser compartida por el usuario al cliente si así lo requiera, esto nos permitirá poder satisfacer la exigencia de notas más claras y legibles para algunos clientes sobre su compra.

Figura 57

Frontend pantallas prueba



3.5.1.2 Implementación Funcional e Integración de Componentes.

Para garantizar una estructura ordenada, escalable y mantenible en el desarrollo de la aplicación móvil, se adoptó el patrón arquitectura MVVM (Model-View-ViewModel). Este patrón permitió separar claramente la lógica de presentación, la interfaz de usuario y la gestión de datos, facilitando el desarrollo modular y el control de cambios.

View. - en la interfaz y uso de la cámara nos permite capturar la imagen que se enviara al ModelView, también en el View recibirá los resultados para ser mostrados en las diferentes pantallas que así lo requieran, como observamos en la Figura 58 dos fragmentos de códigos

donde el primero tenemos el panel de navegación inicializado para usarse y el inferior donde definimos los botones y los procesos que harán este, el botón de captura nos ayudara para conectar con el ModelView y los botones derecho e izquierdo nos enviaran a otras pantallas donde el MV nos ayudara a recibir los datos tratados para ser visualizados por el usuario.

Figura 58

Fragmento View

```
fun AppNavigation() {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = "main") {
        composable("main") { MainScreen(navController = navController) }
        composable("datos") { DatosScreen() }
        composable("compartir") { CompartirScreen() }
    }
}

fun MainScreen(viewModel: MainViewModel = viewModel(), navController: NavController) {
    ) {
        horizontalArrangement = Arrangement.SpaceBetween
    ) {
        Button(onClick = { navController.navigate("datos") }) {
            Text("Datos")
        }
        Button(onClick = { navController.navigate("compartir") }) {
            Text("Compartir")
        }
    }
    //CameraXView(viewModel = viewModel, modifier = Modifier.weight(1f))

    Button(
        onClick = {
            val imageCapture = viewModel.imageCapture ?: return@Button
            val executor = ContextCompat.getMainExecutor(context)
        }
    )
}
```

Model. – Acá se diseñarán todos los módulos independientes donde se realiza un proceso específico:

Tenemos una clase donde se aplican filtros para las imágenes que se reciban y devolverá la otra imagen, ya con los filtros aplicados como se ve en el fragmento de códigos en la Figura 59, son diferentes filtros de iluminación, corrección de color, nitidez y contraste que ayudaran a

tener una imagen más clara que podamos utilizar en las detecciones, muchas veces contaremos con imágenes muy oscuras que nos impedirán la detección eficiente así que la ayuda de estos filtros nos ayudaran a realizar nuestra labor más eficientemente tenemos dos ejemplos de imágenes una con y otra sin estos filtros como se ve en la Figura 60.

Figura 59

Clase para filtros

```
private fun enhanceColorText(bitmap: Bitmap): Bitmap {
    val width = bitmap.width
    val height = bitmap.height
    val config = bitmap.config ?: Bitmap.Config.ARGB_8888
    val output = Bitmap.createBitmap(width, height, config)
    for (y in 0 until height) {
        for (x in 0 until width) {
            val pixel = bitmap.getPixel(x, y)
            val r = Color.red(pixel)
            val g = Color.green(pixel)
            val b = Color.blue(pixel)
            val luminance = 0.299 * r + 0.587 * g + 0.114 * b
            if (luminance < 160) {
                val hsv = FloatArray(3)
                Color.RGBToHSV(r, g, b, hsv)
                hsv[1] = min(1.0f, hsv[1] * 1.5f)
                output.setPixel(x, y, Color.HSVToColor(hsv))
            } else {
                output.setPixel(x, y, pixel)
            }
        }
    }
    return output
}

private fun adjustContrast(bitmap: Bitmap, contrast: Float): Bitmap {
    val width = bitmap.width
    val height = bitmap.height
    val config = bitmap.config ?: Bitmap.Config.ARGB_8888
    val output = Bitmap.createBitmap(width, height, config)

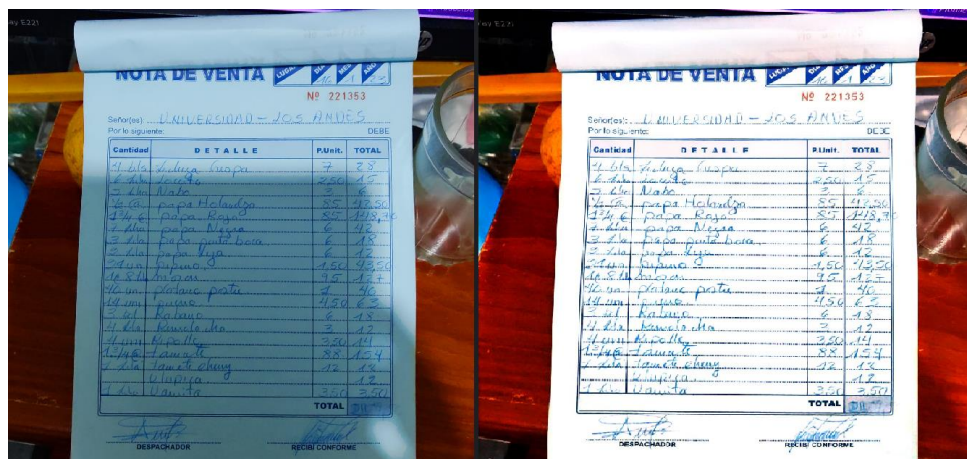
    for (x in 0 until width) {
        for (y in 0 until height) {
            val pixel = bitmap.getPixel(x, y)
            val r = (Color.red(pixel) - 128).toFloat()
            val g = (Color.green(pixel) - 128).toFloat()
            val b = (Color.blue(pixel) - 128).toFloat()

            val newR = (r * contrast + 128).toInt().coerceIn(0, 255)
            val newG = (g * contrast + 128).toInt().coerceIn(0, 255)
            val newB = (b * contrast + 128).toInt().coerceIn(0, 255)

            output.setPixel(x, y, Color.rgb(newR, newG, newB))
        }
    }
    return output
}
```

Figura 60

Izquierda original, Derecha con Filtros



Otra clase esencial para el área de redimensión y añadido de padding si se requieren estas serán utilizadas para el reconocimiento en las redes neuronales como se observa en la Figura 61, también cuenta con las funciones de rotación y una función que ayude a reducir la imagen cercana a solo el área de la nota de venta.

Figura 61

Clase de redimensión y Rotación

```
fun rotateBitmapIfNeeded(bitmap: Bitmap, image: ImageProxy): Bitmap {
    val rotationDegrees = image.imageInfo.rotationDegrees
    if (rotationDegrees == 0) return bitmap // no rotación necesaria

    val matrix = Matrix()
    matrix.postRotate(rotationDegrees.toFloat())
    return Bitmap.createBitmap(bitmap, 0, 0, bitmap.width, bitmap.height, matrix, true)
}

fun cropCenterSquare(bitmap: Bitmap, size: Int): Bitmap {
    val centerX = bitmap.width / 2
    val centerY = bitmap.height / 2

    val halfSize = size / 2
    val left = (centerX - halfSize).coerceAtLeast(0)
    val top = (centerY - halfSize).coerceAtLeast(0)

    val finalSize = size.coerceAtMost(minOf(bitmap.width, bitmap.height))

    return Bitmap.createBitmap(bitmap, left, top, finalSize, finalSize)
}

fun cropUsingOverlayWithLogs(
    bitmap: Bitmap,
    overlayRect: RectF,
    previewWidth: Int,
    previewHeight: Int
```

La clase de detección de áreas donde se detecta las áreas de fecha, num_serie, cliente, detalle, se carga el entrenamiento best-fp16.tflite y con el cual nos ayudara a las detecciones, el código realiza la detección de una imagen utilizando el modelo TensorFlow Lite como vemos en la Figura 62, pero la imagen se verifica si son las dimensiones 960*960, numero de detección es 56700 son el número posible de predicciones que el modelo genera, numClases que es 4,

representa las cuatro clases que reconoce el modelo: user=cliente, date=fecha, nro= num_Serie y detail= detalle_venta, threshold = 0.5f es el nivel de confianza para realizar pruebas se dejó en 0.5 para verificar el mínimo de predicciones, todo esto es utilizado por interpreter que es el que ejecuta el modelo TFLite recibiendo los datos y devolviendo los resultados de la inferencia, se hace la conversión a coordenadas utilizables para dibujar las detecciones.

Figura 62

Clase Detector.kt

```
class Detector(context: Context) {
    private val interpreter: Interpreter
    private val inputSize = 960
    private val numDetections = 56700
    private val numClasses = 4
    private val threshold = 0.5f
    init {
        val modelBuffer = FileUtil.loadMappedFile(context, "best-fp16.tflite")
        interpreter = Interpreter(modelBuffer)
    }
    fun detect(bitmap: Bitmap): Pair<List<DetectionResult>, Bitmap> {
        val input = preprocess(bitmap)
        val output = Array(1) { Array(numDetections) { FloatArray(9) } }
        interpreter.run(input, output)
        val bestDetectionsPerClass = mutableMapOf<Int, DetectionResult>()
        for (i in 0 until numDetections) {
            val objectness = output[0][i][4]
            if (objectness > threshold) {
                val x = output[0][i][0]
                val y = output[0][i][1]
                val w = output[0][i][2]
                val h = output[0][i][3]
                val classScores = output[0][i].slice(5 until 9) // Cambia si hay más clases
                val (classId, classScore) = classScores.withIndex().maxByOrNull { it.value }!!
                if (classScore > threshold) {
                    val left = (x - w / 2) * inputSize
                    val top = (y - h / 2) * inputSize
                    val right = (x + w / 2) * inputSize
                    val bottom = (y + h / 2) * inputSize
                    val rect = RectF(left, top, right, bottom)
                }
            }
        }
    }
}
```

El proceso es similar con detectorD.kt se recibe la imagen se utiliza interpreter que recibe los datos de entrada y devuelve los resultados de la inferencia como se observa en la Figura 63, con la diferencia de los valores estos cambian ya que estas son imágenes de 640*640, el número de detección que realiza es 25200, con esa configuración clave tendremos detecciones correctas.

Figura 63*Clase DetectorD.kt*

```

private const val INPUT_SIZE = 640
private const val NUM_CLASSES = 4 // Tus 4 clases
private const val CONFIDENCE_THRESHOLD = 0.5f // Subir de 0.5 a 0.7
private const val IOU_THRESHOLD = 0.6f // Reducir solapamientos

fun detectarObjetos(bitmap: Bitmap, context: Context): List<Detection> {
    val model = loadModel(context, "best-fp16-v4-class3.tflite")
    val interpreter = Interpreter(model)
    val inputBuffer = preprocess(bitmap)
    val outputBuffer = TensorBuffer.createFixedSize(intArrayOf(1, 25200, NUM_CLASSES + 5), DataType.FLOAT32)
    interpreter.run(inputBuffer, outputBuffer.buffer)
    val detections = postprocess(outputBuffer.floatArray, bitmap)
    detections.forEach {
        println(" ${it.className} (${ "%.1f".format(it.confidence * 100)}%) " +
            "en [ ${it.bbox[0].toInt()}, ${it.bbox[1].toInt()}, ${it.bbox[2].toInt()}, ${it.bbox[3].toInt()}]")
    }
    interpreter.close()
    return detections
}

```

La clase encargada de la predicción CRNN es CRNNModelUtils.kt. esta clase tiene las funciones necesarias para realizar la inferencia utilizando el modelo reentrenado “crnn-model-copy-pt”.

Como se muestra en la Figura 64, el flujo del proceso sigue la siguiente secuencia:

Se importa la librería de PyTorch Mobile para Android y carga el modelo CRNN esto se hace a travez de la función Module.load().

La imagen recibida de las detenciones anteriores es transformada en un tensor compatible con PyTorch mediante el método inputTensor en este proceso se escala, normaliza y convierte en tipo dato.

Una vez preparado el tensor de entrada, se llama método model.forward(inputTensor), el cual genera una salida un tensor con las probabilidades de carácter en cada paso de tiempo.

El resultado es un tensor multidimensional que se convierte a un formato legible utilizando funciones auxiliares como `toTensor()` y el algoritmo de decodificación CTC que traduce las probabilidades en una secuencia de texto reconocida.

Este resultado es retornado como un String por la clase.

Figura 64

Clase CRNNModelUtils.kt

```
fun loadModel(context: Context): Module {
    return Module.load(assetFilePath(context, "crnn_model-copy.pt"))
}

fun predict(context: Context, bitmap: Bitmap, model: Module, vocab: List<String>): String {
    return try {
        val inputTensor = preprocessImageGrayscale(context, bitmap)
        val output = model.forward(IValue.from(inputTensor)).toTensor()
        val outputData = output.dataAsFloatArray
        val shape = output.shape()
        Log.d("Predict", "Output shape: ${shape.joinToString(",")}")
        Log.d("Predict", "Output data length: ${outputData.size}")

        val I = shape[0].toInt()
        val vocabSize = shape[2].toInt()

        if (!vocab.contains("<blank>")) {
            Log.e("Predict", "Vocab no contiene token <blank>")
            return "Error: vocab no contiene <blank>"
        }

        val predictions = mutableListOf<Int>()
        var lastChar = -1
        for (t in 0 until I) {
            var maxIdx = 0
            var maxProb = Float.NEGATIVE_INFINITY
            for (v in 0 until vocabSize) {
                val idx = t * vocabSize + v
                if (outputData[idx] > maxProb) {
                    maxProb = outputData[idx]
                    maxIdx = v
                }
            }
            if (maxIdx != lastChar && maxIdx != vocab.indexOf("<blank>")) {
                predictions.add(maxIdx)
            }
        }
        predictions.joinToString("")
    } catch (e: Exception) {
        Log.e("Predict", "Error: ${e.message}")
        ""
    }
}
```

Otra clase importante es DatabaseHelper.kt esta clase esta encargada de la creación de la base de datos como se ve en la Figura 65, está siendo local utilizando la base de datos SQLite, el cuales ideal para dispositivos móviles por su bajo consumo y operación offline.

El diseño de la base de datos es relacional utilizando las siguientes tablas:

Tabla nota_venta: tabla que almacena la información general de cada recibo de venta incluyendo los campos id_venta, fecha, num_nota, cliente.

Tabla detalle: esta tabla se almacena la información general de detalle con un id_venta como llave foránea y los campos cantidad, descripción, precio_unitario y precio_total y referenciando a id_producto.

Tabla producto: esta se incluyó después de la normalización, su propósito es evitar repeticiones y ayudar en la búsqueda más eficientes por nombre de producto con los campos id_producto, producto.

Esto ayudara a diferentes tipos de consultas complejas.

Además esta otra clase complementaria llamada crud.kt la cual contiene funciones necesarias para realizar operaciones básicas sobre las tablas creadas.

- Inserción de notas de venta y su detalle en las tablas creadas.
- Recuperación de las notas de venta mediante el número de nota.
- Eliminación de nota de venta en conjunto con la tabla detalle relacionado con esta.
- Consultas personalizadas par el reporte y/o análisis de ventas como ser: las ventas por mes, día, esta clase nos permitirá crear distintos tipos de consultas para un análisis básico de ventas.

Figura 65

Clase DatabaseHelper.kt

```
private val dbHelper = DatabaseHelper(context)
fun insertNota(
    idVenta: String,
    date: String,
    cliente: String,
    productDetails: List<ProductDetail>
): Boolean {
    val db = dbHelper.writableDatabase
    db.beginTransaction()
    try {
        // Generar clave única
        val (increment, letter) = getNextIncrementAndLetter(idVenta)
        val fullSerial = "$idVenta-$increment$letter"

        // Insertar nota de venta
        val noteValues = ContentValues().apply {
            put(DatabaseContract.SaleNoteEntry.COLUMN_FULL_SERIAL, fullSerial)
            put(DatabaseContract.SaleNoteEntry.COLUMN_BASE_SERIAL, idVenta)
            put(DatabaseContract.SaleNoteEntry.COLUMN_INCREMENT, increment)
            put(DatabaseContract.SaleNoteEntry.COLUMN_LETTER, letter)
            put(DatabaseContract.SaleNoteEntry.COLUMN_DATE, date)
            put(DatabaseContract.SaleNoteEntry.COLUMN_REFERRED_TO, cliente)
        }
        val noteResult = db.insert(
            DatabaseContract.SaleNoteEntry.TABLE_NAME,
            null,
            noteValues
        )
        if (noteResult == -1L) return false
        for ((index, detail) in productDetails.withIndex()) {
            val rowNumber = index + 1
            val productId = getOrInsertProductId(db, detail.description)
            val detailValues = ContentValues().apply {
                put(DatabaseContract.ProductDetailEntry.COLUMN_SALE_NOTE_SERIAL, fullSerial)
                put(DatabaseContract.ProductDetailEntry.COLUMN_ROW_NUMBER, rowNumber)
                put(DatabaseContract.ProductDetailEntry.COLUMN_QUANTITY, detail.quantity)
                put(DatabaseContract.ProductDetailEntry.COLUMN_ID_PRODUCT, productId)
                put(DatabaseContract.ProductDetailEntry.COLUMN_UNIT_PRICE, detail.unitPrice)
                put(DatabaseContract.ProductDetailEntry.COLUMN_TOTAL, detail.total)
            }
        }
    }
}
```

Son clases más importantes que se requieren para el funcionamiento de la aplicación y utilización de las redes neuronales reentrenadas,

ViewModel. – acá se creará un módulo de control donde la imagen sea recibida sirva como comunicación entre en View y Model, esta imagen será enviada a Model usara la clase SmartImageEnhacer.kt donde se redimensionará, aplicara filtros y se ordenara su posición para retornarla en condiciones para de ser usada, como se ve en la Figura 66, después de obtenerse

esta se envía a la clase `Detector.kt` para utilizar la detección de las áreas de la nota como prueba de integración se dibujó las coordenadas capturadas y extraída el área de interés como se ve en la Figura 67, y esta pasa a `DetectorD.kt` donde del área detalle se detecta las celdas y estas también son extraídas y son enviadas a `CRNNModelUtils.kt` esta recibe las imágenes por celdas y devuelve la predicción en strings dentro una matriz para mantener el orden, para luego ser almacenada en la base de datos SQLite `NotaVenta`, esta podrá ser usada cuando se requiera, el `ModelView` al recibir del `View` la solicitud si es para mostrar los datos para su análisis o se extenderá en formato PDF para ser enviado al cliente como se ve en la Figura 68.

Figura 66

ModelView

```
object : ImageCapture.OnImageCapturedCallback() {
    override fun onCaptureSuccess(image: ImageProxy) {
        val bitmap = CameraUtils.bitmapFromImageProxy(image)
        val corrected = CameraUtils.rotateBitmapIfNeeded(bitmap, image)
        val overlayRect = RectF(0.1f, 0.2f, 0.9f, 0.8f)
        val previewWidth = viewModel.previewWidth
        val previewHeight = viewModel.previewHeight
        val cropped = CameraUtils.cropUsingOverlayWithLogs(corrected, overlayRect, previewWidth, previewHeight)
        val enhancedFull = SmartImageEnhancer.autoEnhanceDocument(cropped)
        CameraUtils.saveBitmapToGallery(context, enhancedFull, "filtroOrg.jpg")
        val resizeds = resizeBitmapToSquare(cropped, 960)
        viewModel.onImageCaptured(resizeds)
        val detector = Detector(context)
        val (detections, bitmapConCajas) = detector.detect(resizeds)
        for (det in detections) {
            Log.d("TFLITE_RESULT", "Clase: ${det.classId}, Score: ${det.score}, Rect: ${det.rect}")
        }
        val classLabels = listOf("date", "nro", "user", "detail")
        val recortes = detector.recortarAreas(resizeds, detections, classLabels)
        for ((nombreClase, recorteBitmap) in recortes) {
            Log.d("RECORTE", "Clase: $nombreClase -> Tamaño: ${recorteBitmap.width}x${recorteBitmap.height}")
        }
        val recortePorNombre = recortes["detail"]
        if (recortePorNombre != null) {
            val detecciones = DetectorD.detectarObjetos(recortePorNombre, context)
            val bitmapConRecuadros = DetectorD.drawDetectionsOnBitmap(recortePorNombre, detecciones)
            viewModel.onImageCaptured(bitmapConRecuadros)
            CameraUtils.saveBitmapToGallery(context, bitmapConRecuadros, "detail")
            val recortePorNombre = recortes["detail"]
        }
    }
}
```

Figura 67

Deteccion y Recorte

Señor(es): Atix - Cocina Nº 221276

Por lo siguiente: DEBE

Cantidad	DETALLE	P.Unit.	TOTAL
4 K	Uva Verde Sin pepa	25	25
8 K	Zanahoria	5	40
3 K	Zugarrin	6	18
—	piñoncho	—	—
TOTAL			73

DESPACHADOR: [Signature] RECIBI CONFORME: [Signature]

Señor(es): Atix - Cocina Nº 221276

Por lo siguiente: DEBE

Cantidad	DETALLE	P.Unit.	TOTAL
4 K	Uva Verde Sin pepa	25	25
8 K	Zanahoria	5	40
3 K	Zugarrin	6	18
—	piñoncho	—	—
TOTAL			73

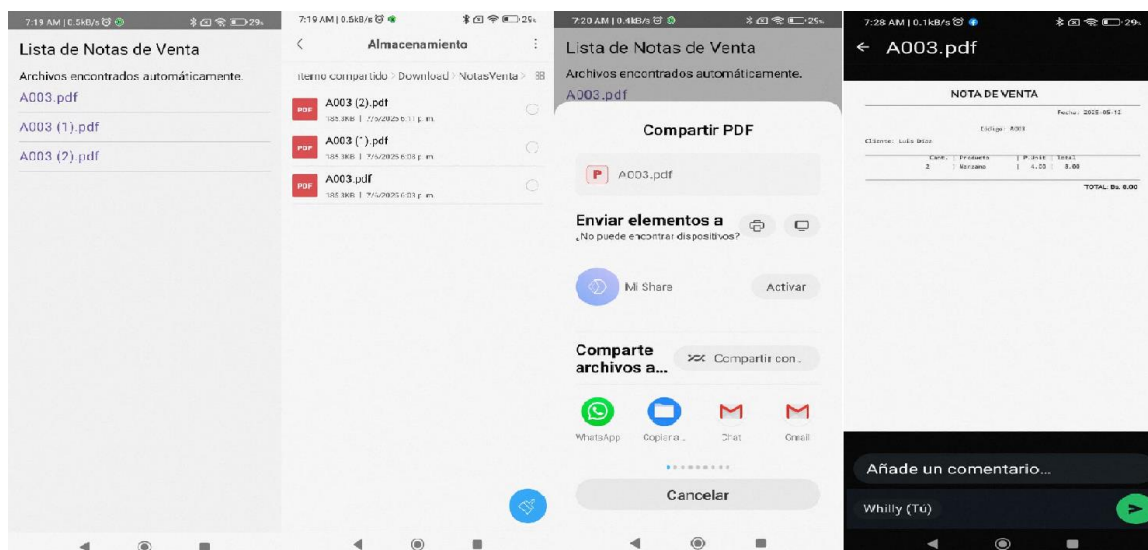
DESPACHADOR: [Signature] RECIBI CONFORME: [Signature]

Atix - Cocina

221276

Figura 68

Captura de aplicación de detección CRNN



3.6 FASE DE PRUEBAS

En esta fase se verifica todas las actividades realizadas en las fases anteriores, asegurando que la aplicación sea funcional, documentada y alineada con los objetivos propuestos.

3.6.1 Validación de Requisitos

Se verifica que la aplicación cumpla con todos los requerimientos funcionales y no funcionales definidos en la fase de exploración.

Tabla 4

Validación de Requisitos

Nº	Descripción	Evidencia / Observación
RF1	Captura de nota manuscrita	Captura implementada y mostrada en la app
RF2	Detección de áreas con YOLOv5	Detección de áreas eficientes
RF3	Detección de Detalles con YOLOv5	Detección de áreas eficientes
RF4	Reconocimiento de texto con CRNN	Reconocimiento de texto
RF5	Exportación en PDF	Posible exportación de notas de venta en PDF
RF6	Almacenamiento Local de Datos	Datos capturados se almacenan en SQLite
RF7	Visualización y Análisis de Datos	Resultados visibles en pantalla
RNF1	Interfaz intuitiva y simple	Uso sin capacitación, diseño en Kotlin
RNF2	Procesamiento menor a 15s	Tiempo medido en dispositivo Poco F1
RNF3	Funciona offline	No requiere internet para funcionar
RNF4	Idioma castellano	Toda la interfaz en español

3.6.2 Pruebas de Funcionamiento

La aplicación de digitalización inteligente de recibos manuscritos, se realizaron pruebas de funcionamiento en dos diferentes dispositivos un Pocophone F1 con Android 9 y un Pocophone X3 Pro con Android 12.

3.6.3 Prueba de Integración

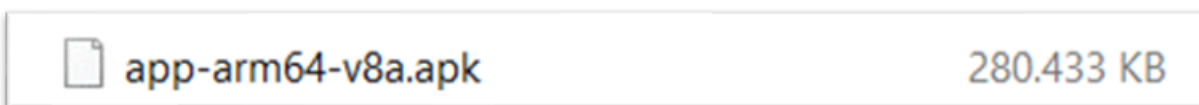
Se validó la correcta integración entre los componentes de detección (YOLOv5), reconocimiento (CRNN), almacenamiento local (SQLite), y la interfaz de usuario. Se realizaron pruebas integrales del flujo completo desde la captura hasta la generación del recibo digital, cumpliendo con todos los requisitos definidos.

3.6.4 Despliegue del instalador de la aplicación

Se generó el archivo .apk, para su uso y distribución como se observa en la Figura 69.

Figura 69

Instalador APK



Nota. – captura de la aplicación móvil.

CAPITULO IV

4 EVALUACION DE RESULTADOS

En este capítulo se centra en validación de la hipótesis planteada en el CAPITULO I proponía que una aplicación móvil basada en Deep Learning y HTR podría permitir a los contribuyentes del RTS digitalizar notas de venta manuscritas con una precisión superior al 70%, generando versiones estructuradas de los mismos, listas para ser almacenadas y analizadas digitalmente. Para ello, se han desarrollado una aplicación que hace uso de la inteligencia artificial, utilizando Deep Learning, Yolov5, CRNN, y se ha aplicado en un conjunto de imágenes de notas de venta manuscritas para evaluar su rendimiento.

4.1 METODOLOGIAS DE EVALUACION

4.1.1 Selección De Estudio.

Se utilizo un conjunto de 78 notas de venta manuscritas, capturadas en distintos entornos, todas las notas presentan escritura legible (según criterio visual). Se usaron notas de venta con diferentes tipos de tinta azul, negro, para observar el contraste con el fondo del papel, se excluyeron notas de venta con letras tipo “doctor” ni escrituras con trazos ilegibles

4.1.2 Descripción De La Aplicación Móvil.

La aplicación móvil ha sido desarrollada utilizando redes neuronales convolucionales, recurrentes y de detección, las cuales permiten identificar y convertir el texto manuscrito presente en los recibos de venta. Para el entrenamiento y validación de los modelos, se utilizó un conjunto de datos compuesto por imágenes de notas de venta manuscritas.

Se filtran en una tabla donde se listan en las siguientes columnas: caracteres, acierto del modelo y el acierto esperado, como se ve en la Tabla 5 .

Tabla 5

Tabla generada atreves de la matriz de confusión.

	Caracteres	Acierto del modelo	Acierto esperado
1	l	316	466
2	i	434	538
3	u	451	554
4	a	1030	1146
5	o	505	583
6	r	281	328
7	B	23	39
8	n	275	320
9	,	437	490
10	c	182	214
11	R	46	63
12	(	69	87
13	e	304	338
14	j	90	109
15	w	10	17
16	s	49	62
17	d	173	195
18	1	831	877
19	b	132	149
20	2	772	810
21	t	224	244
22	g	29	36
23	0	976	1008
24	=	7	10
25	F	24	29
26	3	363	381
27	5	577	599
28	T	16	20
29	4	518	537
30	8	321	336
31	S	37	42
32	k	5	7

33	)	43	48
34	z	44	49
35	m	161	170
36	V	18	21
37	M	35	39
38	h	113	120
39	-	8	10
40	9	153	160
41	q	30	33
42	f	13	15
43	ñ	33	36
44	K	62	66
45	/	4	5
46	y	48	51
47	O	6	7
48	7	249	255
49	E	7	8
50	v	31	33
51	6	424	430
52	A	18	19
53	Z	18	19
54	C	75	77
55	H	23	24
56	P	23	24
57	N	33	34
58	p	209	211
59	L	71	71
60	U	20	20
61	I	15	15
62	.	8	8
63	x	5	5
64	#	3	3
65	D	2	2
66	Q	2	2
67	@	1	1
68	J	1	1
69	X	1	1
70	Y	1	1
71	°	1	1

4.1.3.2 Cálculo de Chi Cuadrado de Bondad de Ajuste.

La prueba de chi cuadrado de bondad de ajuste es una prueba estadística no paramétrica que permite determinar si las frecuencias observadas en una muestra difieren significativamente de las frecuencias esperadas bajo una determinada hipótesis, puede usarse para verificar si las predicciones están distribuidas uniformemente o de acuerdo a una proporción teórica esperada.

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i} \quad (10)$$

Donde:

- O_i : Frecuencia observada en la categoría i.
- E_i : Frecuencia esperada en la categoría i.

Se calcula chi cuadrado como observamos en la Tabla 6. Separamos los cálculos en columnas, en la quinta columna hallamos la diferencia entre los aciertos del modelo y los aciertos esperados, en la siguiente columna elevamos el resultado hallado por la diferencia, posteriormente dividimos por el acierto esperado, una vez concluidos estos cálculos podemos hacer la sumatoria de los valores de la última columna.

Tabla 6

Tabla de cálculos para chi cuadrado.

	Carácter	Acierto del modelo O_i	Acierto esperado E_i	$O_i - E_i$	$(O_i - E_i)^2$	$\frac{(O_i - E_i)^2}{E_i}$
1	l	316	466	-150	22500	48,2832618
2	i	434	538	-104	10816	20,1040892
3	u	451	554	-103	10609	19,1498195
4	a	1030	1146	-116	13456	11,7417103
5	o	505	583	-78	6084	10,4356775
6	r	281	328	-47	2209	6,7347561

7	B	23	39	-16	256	6,56410256
8	n	275	320	-45	2025	6,328125
9	,	437	490	-53	2809	5,73265306
10	c	182	214	-32	1024	4,78504673
11	R	46	63	-17	289	4,58730159
12	(	69	87	-18	324	3,72413793
13	e	304	338	-34	1156	3,42011834
14	j	90	109	-19	361	3,31192661
15	w	10	17	-7	49	2,88235294
16	s	49	62	-13	169	2,72580645
17	d	173	195	-22	484	2,48205128
18	1	831	877	-46	2116	2,41277081
19	b	132	149	-17	289	1,93959732
20	2	772	810	-38	1444	1,78271605
21	t	224	244	-20	400	1,63934426
22	g	29	36	-7	49	1,36111111
23	0	976	1008	-32	1024	1,01587302
24	=	7	10	-3	9	0,9
25	F	24	29	-5	25	0,86206897
26	3	363	381	-18	324	0,8503937
27	5	577	599	-22	484	0,80801336
28	T	16	20	-4	16	0,8
29	4	518	537	-19	361	0,67225326
30	8	321	336	-15	225	0,66964286
31	S	37	42	-5	25	0,5952381
32	k	5	7	-2	4	0,57142857
33	)	43	48	-5	25	0,52083333
34	z	44	49	-5	25	0,51020408
35	m	161	170	-9	81	0,47647059
36	V	18	21	-3	9	0,42857143
37	M	35	39	-4	16	0,41025641
38	h	113	120	-7	49	0,40833333
39	-	8	10	-2	4	0,4
40	9	153	160	-7	49	0,30625
41	q	30	33	-3	9	0,27272727
42	f	13	15	-2	4	0,26666667
43	ñ	33	36	-3	9	0,25
44	K	62	66	-4	16	0,24242424
45	/	4	5	-1	1	0,2
46	y	48	51	-3	9	0,17647059
47	O	6	7	-1	1	0,14285714
48	7	249	255	-6	36	0,14117647
49	E	7	8	-1	1	0,125

50	v	31	33	-2	4	0,12121212
51	6	424	430	-6	36	0,08372093
52	A	18	19	-1	1	0,05263158
53	Z	18	19	-1	1	0,05263158
54	C	75	77	-2	4	0,05194805
55	H	23	24	-1	1	0,04166667
56	P	23	24	-1	1	0,04166667
57	N	33	34	-1	1	0,02941176
58	p	209	211	-2	4	0,01895735
59	L	71	71	0	0	0
60	U	20	20	0	0	0
61	I	15	15	0	0	0
62	.	8	8	0	0	0
63	x	5	5	0	0	0
64	#	3	3	0	0	0
65	D	2	2	0	0	0
66	Q	2	2	0	0	0
67	@	1	1	0	0	0
68	J	1	1	0	0	0
69	X	1	1	0	0	0
70	Y	1	1	0	0	0
71	°	1	1	0	0	0

Se procede a la suma de los valores de la última columna como se ve en la Tabla 6 y la ecuación (11).

$$\chi^2_c = \sum \frac{(O_i - E_i)^2}{E_i} = 185,6454766 \quad (11)$$

El valor resultante es χ^2 calculado es 185,64, ahora se determinará χ^2 crítico, para ello se tomará total de caracteres utilizados en la Tabla 6, que son en total 71 usaremos la ecuación (12), para hallarlo.

$$gl = k - 1 \quad (12)$$

Los cálculos serán:

$$gl = 71 - 1 = 70 \quad (13)$$

Hallamos la significación, en la hipótesis propuesta se optó por el 70% entonces α es 0,3. Una vez obtenido todos los valores buscamos en la tabla de chi cuadrado para determinar χ^2 crítico como vemos en la ecuación (14).

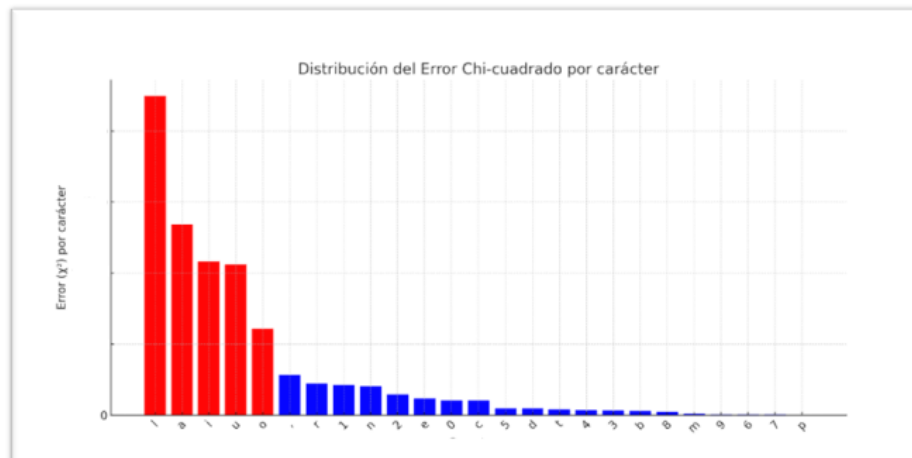
$$\chi_c^2 = 75,6893 \quad (14)$$

Ahora que se tiene $\chi_{critico}^2$ y $\chi_{calculado}^2$ podemos analizar nuestra estadística de prueba, se observa en la ecuación (11) y la ecuación (14), que $\chi_{calculado}^2$ es mayor que $\chi_{calculado}^2$, rechazando la hipótesis nula H_0 .

El valor del estadístico chi-cuadrado obtenido 185,64, supera ampliamente el valor crítico de 75,68, correspondiente a 70 grados de libertad con un nivel de significancia del 30%. Esta diferencia implica que los datos observados no se ajustan a la distribución esperada de manera uniforme como se ve en la Figura 71, lo que sugiere que el rendimiento del modelo varía significativamente entre los diferentes caracteres evaluados.

Figura 71

Gráfico de distribución de error de chi cuadrado por carácter.



4.1.3.3 Cálculo del Coeficiente Kappa de Cohen.

El coeficiente Kappa de Cohen es una medida estadística del grado de concordancia entre dos evaluadores o clasificadores que asignan categorías a los mismos elementos y está definida por la formula en la ecuación (15). A diferencia de la simple precisión, kappa corrige el acuerdo que podría darse por azar.

$$\mathcal{K} = \frac{P_o - P_e}{1 - P_e} \quad (15)$$

Donde:

- P_o : Proporción de acuerdo a lo esperado (OA).
- P_e : Proporción de acuerdo a lo esperado por azar.

Al tener 71 variables en la matriz de confusión no es visualmente apreciable por lo que se usó una tabla extrayendo los total reales, predicciones correctas y total predicciones, para los cálculos de kappa cohen.

Tabla 7

Tabla de la Matriz de Confusión Valores Para Cálculos Kappa Cohen

Caracteres	Total, reales	Predicciones correctas	Total, predicciones
a	1146	1030	1234
0	1008	976	1045
1	877	831	851
2	810	772	796
5	599	577	600
o	583	505	608
u	554	451	532
i	538	434	505
4	537	518	534
,	490	437	489

l	466	316	362
6	430	424	431
3	381	363	378
e	338	304	336
8	336	321	337
r	328	281	313
n	320	275	341
7	255	249	254
t	244	224	249
c	214	182	210
p	211	209	212
d	195	173	207
m	170	161	169
9	160	153	160
b	149	132	152
h	120	113	120
j	109	90	109
(	87	69	87
C	77	75	80
L	71	71	75
K	66	62	74
R	63	46	66
s	62	49	62
y	51	48	60
z	49	44	49
)	48	43	73
S	42	37	43
B	39	23	37
M	39	35	39
g	36	29	34
ñ	36	33	37
N	34	33	34
v	33	31	31
q	33	30	32
F	29	24	28
H	24	23	23
P	24	23	23
V	21	18	22
U	20	20	21
T	20	16	16
Z	19	18	18
A	19	18	18
w	17	10	10

I	15	15	15
f	15	13	14
-	10	8	10
=	10	7	10
E	8	7	8
.	8	8	8
O	7	6	6
k	7	5	5
x	5	5	7
/	5	4	5
#	3	3	3
D	2	2	2
Q	2	2	2
°	1	1	1
X	1	1	1
@	1	1	4
Y	1	1	1
J	1	1	1

Determinamos $P_o = OA$ Overall Accuracy vemos la ecuación (16), es la proporción de acuerdo a lo esperado se calcula como:

$$OA = \frac{\text{numero de predicciones correctas}}{\text{total de predicciones} = N} \quad (16)$$

De la Tabla 7, se utiliza los valores en la columna predicciones correctas y total predicciones, como se ve en la ecuación (17),

$$OA = \frac{\sum_{i=1}^k \text{predicciones correctas}_{i,i}}{\sum_{i=1}^k \sum_{j=1}^k \text{predicciones}_{i,j}} \quad (17)$$

De los cálculos de las sumatorias en OA tenemos el resultado en la ecuación (18):

$$OA = \frac{11519}{12729} = 0,904941472 = P_o \quad (18)$$

Ahora calcularemos P_e la formula se ve en la ecuación (19) que nos ayudara para determinar el resultado para calcular Kappa, donde es una sumatoria de productos donde k es igual a la cantidad de caracteres que son 71, dentro la sumatoria se calculara el total real dividido sobre el total real, multiplicado por, el total predicho dividido por el total real:

$$P_e = \sum_{i=1}^k \left(\frac{\text{total real } i}{N} * \frac{\text{total predicho } i}{N} \right) \quad (19)$$

En la Tabla 8, se observa los resultados para determinar P_e , se calcula $\frac{\text{total real } i}{N}$ en la sexta columna, luego calculamos $\frac{\text{total predicho } i}{N}$ en la séptima columna y hallamos el producto $\frac{\text{total real } i}{N} * \frac{\text{total predicho } i}{N}$ en la octava columna para posterior hacer la sumatoria de los resultados:

Tabla 8

Tabla de cálculos de P_e

#	Caract.	Total, reales	Pred. correctas	Total, pred.	$\frac{\text{total real } i}{N}$	$\frac{\text{total pred. } i}{N}$	$\frac{\text{total real } i}{N} * \frac{\text{total pred. } i}{N}$
1	a	1146	1030	1234	0,09003064	0,09694399	0,00872793
2	o	1008	976	1045	0,07918925	0,082096	0,00650112
3	1	877	831	851	0,06889779	0,06685521	0,00460618
4	2	810	772	796	0,06363422	0,06253437	0,00397933
5	5	599	577	600	0,0470579	0,04713646	0,00221814
6	o	583	505	608	0,04580093	0,04776495	0,00218768
7	u	554	451	532	0,04352266	0,04179433	0,001819
8	i	538	434	505	0,04226569	0,03967319	0,00167681
9	4	537	518	534	0,04218713	0,04195145	0,00176981
10	,	490	437	489	0,03849478	0,03841622	0,00147882
11	l	466	316	362	0,03660932	0,028439	0,00104113
12	6	430	424	431	0,03378113	0,03385969	0,00114382
13	3	381	363	378	0,02993165	0,02969597	0,00088885
14	e	338	304	336	0,02655354	0,02639642	0,00070092
15	8	336	321	337	0,02639642	0,02647498	0,00069884
16	r	328	281	313	0,02576793	0,02458952	0,00063362
17	n	320	275	341	0,02513945	0,02678922	0,00067347

18	7	255	249	254	0,020033	0,01995444	0,00039975
19	t	244	224	249	0,01916883	0,01956163	0,00037497
20	c	214	182	210	0,016812	0,01649776	0,00027736
21	p	211	209	212	0,01657632	0,01665488	0,00027608
22	d	195	173	207	0,01531935	0,01626208	0,00024912
23	m	170	161	169	0,01335533	0,01327677	0,00017732
24	9	160	153	160	0,01256972	0,01256972	0,000158
25	b	149	132	152	0,01170555	0,01194124	0,00013978
26	h	120	113	120	0,00942729	0,00942729	8,8874E-05
27	j	109	90	109	0,00856312	0,00856312	7,3327E-05
28	(	87	69	87	0,00683479	0,00683479	4,6714E-05
29	C	77	75	80	0,00604918	0,00628486	3,8018E-05
30	L	71	71	75	0,00557781	0,00589206	3,2865E-05
31	K	66	62	74	0,00518501	0,0058135	3,0143E-05
32	R	63	46	66	0,00494933	0,00518501	2,5662E-05
33	s	62	49	62	0,00487077	0,00487077	2,3724E-05
34	y	51	48	60	0,0040066	0,00471365	1,8886E-05
35	z	49	44	49	0,00384948	0,00384948	1,4818E-05
36	)	48	43	73	0,00377092	0,00573494	2,1626E-05
37	S	42	37	43	0,00329955	0,00337811	1,1146E-05
38	B	39	23	37	0,00306387	0,00290675	8,9059E-06
39	M	39	35	39	0,00306387	0,00306387	9,3873E-06
40	g	36	29	34	0,00282819	0,00267107	7,5543E-06
41	ñ	36	33	37	0,00282819	0,00290675	8,2208E-06
42	N	34	33	34	0,00267107	0,00267107	7,1346E-06
43	v	33	31	31	0,00259251	0,00243538	6,3137E-06
44	q	33	30	32	0,00259251	0,00251395	6,5174E-06
45	F	29	24	28	0,00227826	0,0021997	5,0115E-06
46	H	24	23	23	0,00188546	0,0018069	3,4068E-06
47	P	24	23	23	0,00188546	0,0018069	3,4068E-06
48	V	21	18	22	0,00164978	0,00172834	2,8514E-06
49	U	20	20	21	0,00157122	0,00164978	2,5922E-06
50	T	20	16	16	0,00157122	0,00125697	1,975E-06
51	Z	19	18	18	0,00149265	0,00141409	2,1108E-06
52	A	19	18	18	0,00149265	0,00141409	2,1108E-06
53	w	17	10	10	0,00133553	0,00078561	1,0492E-06
54	l	15	15	15	0,00117841	0,00117841	1,3887E-06
55	f	15	13	14	0,00117841	0,00109985	1,2961E-06
56	-	10	8	10	0,00078561	0,00078561	6,1718E-07
57	=	10	7	10	0,00078561	0,00078561	6,1718E-07
58	E	8	7	8	0,00062849	0,00062849	3,9499E-07
59	.	8	8	8	0,00062849	0,00062849	3,9499E-07
60	O	7	6	6	0,00054993	0,00047137	2,5922E-07

61	k	7	5	5	0,00054993	0,0003928	2,1601E-07
62	x	5	5	7	0,0003928	0,00054993	2,1601E-07
63	/	5	4	5	0,0003928	0,0003928	1,5429E-07
64	#	3	3	3	0,00023568	0,00023568	5,5546E-08
65	D	2	2	2	0,00015712	0,00015712	2,4687E-08
66	Q	2	2	2	0,00015712	0,00015712	2,4687E-08
67	°	1	1	1	7,8561E-05	7,8561E-05	6,1718E-09
68	X	1	1	1	7,8561E-05	7,8561E-05	6,1718E-09
69	@	1	1	4	7,8561E-05	0,00031424	2,4687E-08
70	Y	1	1	1	7,8561E-05	7,8561E-05	6,1718E-09
71	J	1	1	1	7,8561E-05	7,8561E-05	6,1718E-09

En la ecuación (20), tenemos el resultado de P_e , este resultado nos permitirá calcular Kappa como se ve en la ecuación (21):

$$P_e = \sum_{i=1}^k \left(\frac{\text{total real } i}{N} * \frac{\text{total predicho } i}{N} \right) = 0,343307911 \quad (20)$$

Con los resultados en la ecuación (18) que es P_o y la ecuación (20) que es P_e , podemos determinar el coeficiente Kappa:

$$\mathcal{K} = \frac{P_o - P_e}{1 - P_e} = \frac{0,904941472 - 0,343307911}{1 - 0,343307911} = 0,848693663797 \quad (21)$$

El valor de $\kappa = 0.8486$ indica una muy alta concordancia entre el sistema de digitalización de recibos manuscritos (CRNN) y los datos reales. Como se puede observar en la Tabla 9:

Tabla 9

Interpretación del valor de Kappa

Valor de κ	Interpretación
< 0	Peor que el azar
0.00 – 0.20	Concordancia leve
0.21 – 0.40	Concordancia aceptable

0.41 – 0.60	Concordancia moderada
0.61 – 0.80	Concordancia sustancial
0.81 – 1.00	Concordancia casi perfecta

La aplicación móvil con una concordancia casi perfecta entre 0.81 y 1.00, se rechaza la hipótesis nula (H_0), la aplicación móvil tiene una precisión mejor que el azar y superior a 70%.

CAPITULO V

5 CONCLUSIONES Y RECOMENDACIONES

5.1 Conclusiones

La presente tesis tuvo como objetivo desarrollar una aplicación móvil capaz de digitalizar recibos manuscritos utilizando técnicas de Deep Learning, con el propósito de facilitar la digitalización y análisis de datos básicos de ventas que contribuya así en un mejor control contable para pequeños comerciantes.

La implementación de un modelo basado en Deep Learning y reconocimiento de texto manuscrito (HTR), combinando YOLOv5 para la detección de áreas importantes en la imagen con un modelo CRNN para el reconocimiento de texto manuscrito permite convertir los recibos físicos en un formato digital estructurado. Además, se incorporaron funciones como la exportación a PDF, almacenamiento local en SQLite y la visualización de datos para su análisis, todo esto en una aplicación móvil funcional para dispositivos Android de gama media y que logra funcionar sin conexión a internet.

El sistema desarrollado logro una predicción mayor o igual al 70% en la digitalización de recibos manuscritos, esto demuestra su capacidad para proporcionar una digitalización confiable. Esta herramienta representa un aporte significativo en la inclusión digital de pequeños comerciantes, mostrando que el uso de Deep Learning es viable y útil en entornos laborales y con recursos tecnológicos limitados.

El sistema desarrollado demuestra que es posible aplicar inteligencia artificial en contextos con pocos recursos tecnológicos, acercándose a herramientas modernas para usuarios

sin experiencia técnica, introduciendo herramientas digitales a usuarios sin experiencia técnica y contribuyendo al cierre de la brecha digital en sectores tradicionalmente excluidos de estos avances.

5.2 Recomendaciones

- Se recomienda la ampliación del dataset, para mejorar la precisión y la aplicación sea más robusta, con más notas de venta de diferentes sectores y regiones del país, esto permitirá al modelo generalizar mejor y ser más preciso con diferentes tipos de recibos y escritura.
- Se recomienda realizar ajustes periódicos en los hiperparámetros de los modelos YOLOv5 y CRNN, así también experimentar con arquitecturas más recientes si estas mejoran y se optimizan para dispositivos móviles de recursos medios.
- Se recomienda llevar a cabo más pruebas en distintos dispositivos de gama media, media baja, baja, para evaluar el desempeño de la aplicación, verificar su estabilidad en distintos entornos de iluminación y calidad de imagen,
- Se recomienda ampliar las funcionalidades en la interfaz del usuario, priorizando la usabilidad, simplicidad y accesibilidad para usuarios sin o escasos conocimientos técnicos, para seguir logrando una inclusión digital.
- Una posible recomendación, sería incorporar una sincronización en la nube como respaldo de los datos, por el espacio limitado de los dispositivos móviles.

6 BIBLIOGRAFÍA

- Abhishek, J. (2 de Julio de 2024). *Detailed understanding about convolution operation in a coloured image with the help of 3D kernels*. Obtenido de [https://medium.com/:
https://medium.com/@abhishekjainindore24/detailed-understanding-about-convolution-operation-in-a-coloured-image-with-the-help-of-3d-kernels-16d698645988](https://medium.com/:https://medium.com/@abhishekjainindore24/detailed-understanding-about-convolution-operation-in-a-coloured-image-with-the-help-of-3d-kernels-16d698645988)
- Ali, M. S. (23 de Junio de 2022). *Flattening CNN layers for Neural Network and basic concepts*. Obtenido de [https://medium.com/: https://medium.com/@muhammadshoaibali/flattening-cnn-layers-for-neural-network-694a232eda6a](https://medium.com/:https://medium.com/@muhammadshoaibali/flattening-cnn-layers-for-neural-network-694a232eda6a)
- Amaya Balaguera, Y. D. (2013). Metodologías ágiles en el desarrollo de aplicaciones para dispositivos. Obtenido de <https://dialnet.unirioja.es/descarga/articulo/6041502.pdf>
- Baheti, P. (27 de mayo de 2021). *Activation Functions in Neural Networks [12 Types & Use Cases]*. Obtenido de [https://www.v7labs.com/: https://www.v7labs.com/blog/neural-networks-activation-functions](https://www.v7labs.com/:https://www.v7labs.com/blog/neural-networks-activation-functions)
- Bluche, T. (2015). Deep Neural Networks for Large Vocabulary [tesis de doctorado, Univerisdad Paris-Sud]. *Repositorio Publico*. Obtenido de <https://theses.hal.science/tel-01249405/>
- Boden, M. A. (2017). *Inteligencia artificial*. Turner Publicaciones S.L.
- Cañadas, R. (22 de Noviembre de 2021). *Redes neuronales recurrentes*. Obtenido de [https://abdatum.com/: https://abdatum.com/tecnologia/redes-neuronales-recurrentes](https://abdatum.com/:https://abdatum.com/tecnologia/redes-neuronales-recurrentes)
- Chollet, F. (2021). *DEEP LEARNING with python*. Manning Publications Co.

Cruz, L. S. (2024). *Reconocimiento de texto en manuscritos históricos [Tesis de Mestría, PONTIFICIA UNIVERSIDAD]*. Obtenido de https://tesis.pucp.edu.pe/repositorio/bitstream/handle/20.500.12404/29310/TARAZONA_CRUZ_LUZ_SILVANA_RECONOCIMIENTO_TEXTO.pdf?sequence=1&isAllowed=y

docs.ultralytics.com. (25 de Junio de 2020). *YoloV5*. Obtenido de ultralytics: <https://github.com/ultralytics/yolov5>

futurelab.mx. (25 de Junio de 2019). *Introducción a las Redes Neuronales Pt. I*. Obtenido de futurelab: <https://futurelab.mx/redes%20neuronales/inteligencia%20artificial/2019/06/25/intro-a-redes-neuronales-pt-1/>

García González, L. (2020). *Modelos de atención para la transcripción de textos [Tesis de maestría, Escuela Técnica Superior de Ingeniería]*. Obtenido de <https://biblus.us.es/bibing/proyectos/use/abreproy//fichero/71649%252FTFM-1649+GARC%C3%8DA-GONZ%C3%81LEZ%2C+LUIS.pdf>

García, B. R. (2015). *DISEÑO DE UN AUTÓMATA PARA LA TRADUCCIÓN DEL IDIOMA CHINO AL ESPAÑOL [TESIS DE GRADO, UNIVERSIDAD MAYOR DE SAN ANDRES]*. Obtenido de <https://repositorio.umsa.bo/bitstream/handle/123456789/7660/T.3102.pdf?sequence=1&isAllowed=y>

Hernández Orallo, J., Ramírez Quintana, M., & Ferri Ramírez, C. (2004). *Introducción a la minería de datos*. Pearson Educación.

IBM International Business Machines Corporation. (s.f.). *¿Qué es el deep learning?* Obtenido de International Business Machines Corporation: <https://www.ibm.com/es-es/think/topics/deep-learning>

IBM International Business Machines Corporation. (s.f.). *¿Qué es el reconocimiento óptico de caracteres (OCR)?* Obtenido de International Business Machines Corporation: <https://www.ibm.com/mx-es/think/topics/optical-character-recognition>

IBM International Business Machines Corporation. (s.f.). *¿Qué son las redes neuronales convolucionales?* Obtenido de International Business Machines Corporation: <https://www.ibm.com/think/topics/convolutional-neural-networks>

IBM International Business Machines Corporation. (s.f.). *La historia del OCR*. Obtenido de International Business Machines Corporation: <https://www.ibm.com/es-es/think/topics/optical-character-recognition#:~:text=En%201974%2C%20Ray%20Kurzweil%20fund%C3%B3,impreso%20en%20pr%C3%A1cticamente%20cualquier%20fuente>

IBM International Business Machines Corporation. (s.f.). *Machine learning vs. deep learning vs. redes neuronales*. Obtenido de International Business Machines Corporation: <https://www.ibm.com/es-es/topics/machine-learning>

IBM International Business Machines Corporation. (s.f.). *redes neuronales recurrentes*.

Obtenido de International Business Machines Corporation: <https://www.ibm.com/es-es/think/topics/recurrent-neural-networks>

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). GradientBased Learning Applied to Document. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 7-47.

Leyva, A., Carreño, M., Estrada, I., Sandoval, A., & Ezpinoza, G. (27 de Marzo de 2016). *Desarrollo de una herramienta tipo mLearning utilizando la metodología*. Obtenido de revistas.unab.edu.co:
<https://revistas.unab.edu.co/index.php/rcc/article/download/2620/2226/8279>

Ramires Hernández, M., & León Pérez, F. (Febrero de 2023). *M-learning como herramientas para el apredizaje adaptativo*. Obtenido de researchgate:
https://www.researchgate.net/figure/Figura-1-Metodologia-Mobile-D_fig1_368555118

Strika, L. (26 de julio de 2019). *Convolutional Neural Networks: A python Tutorial Using*. Obtenido de KDnuggets.com: <https://www.kdnuggets.com/2019/07/convolutional-neural-networks-python-tutorial-tensorflow-keras.html>

ANEXOS

Anexo 1: tabla de distribución de Chi-cuadrado

TABLA 3-Distribución Chi Cuadrado χ^2 . (Continuación)

P = Probabilidad de encontrar un valor mayor o igual que el chi cuadrado tabulado, v = Grados de Libertad

v/p	0,001	0,0025	0,005	0,01	0,025	0,05	0,1	0,15	0,2	0,25	0,3	0,35	0,4	0,45	0,5
30	59,7022	56,3325	53,6719	50,8922	46,9792	43,7730	40,2560	37,9902	36,2502	34,7997	33,5302	32,3815	31,3159	30,3073	29,3360
31	61,0980	57,6921	55,0025	52,1914	48,2319	44,9853	41,4217	39,1244	37,3591	35,8871	34,5981	33,4314	32,3486	31,3235	30,3359
32	62,4873	59,0461	56,3280	53,4857	49,4804	46,1942	42,5847	40,2563	38,4663	36,9730	35,6649	34,4804	33,3809	32,3394	31,3359
33	63,8694	60,3953	57,6483	54,7754	50,7251	47,3999	43,7452	41,3861	39,5718	38,0575	36,7307	35,5287	34,4126	33,3551	32,3358
34	65,2471	61,7382	58,9637	56,0609	51,9660	48,6024	44,9032	42,5140	40,6756	39,1408	37,7954	36,5763	35,4438	34,3706	33,3357
35	66,6192	63,0760	60,2746	57,3420	53,2033	49,8018	46,0588	43,6399	41,7780	40,2228	38,8591	37,6231	36,4746	35,3858	34,3356
36	67,9850	64,4097	61,5811	58,6192	54,4373	50,9985	47,2122	44,7641	42,8788	41,3036	39,9220	38,6693	37,5049	36,4008	35,3356
37	69,3476	65,7384	62,8832	59,8926	55,6680	52,1923	48,3634	45,8864	43,9782	42,3833	40,9839	39,7148	38,5348	37,4156	36,3355
38	70,7039	67,0628	64,1812	61,1620	56,8955	53,3835	49,5126	47,0072	45,0763	43,4619	42,0450	40,7597	39,5643	38,4302	37,3354
39	72,0550	68,3830	65,4753	62,4281	58,1201	54,5722	50,6598	48,1263	46,1730	44,5395	43,1053	41,8040	40,5935	39,4446	38,3354
40	73,4029	69,6987	66,7660	63,6908	59,3417	55,7585	51,8050	49,2438	47,2685	45,6160	44,1649	42,8477	41,6222	40,4589	39,3353
45	80,0776	76,2229	73,1660	69,9569	65,4101	61,6562	57,5053	54,8105	52,7288	50,9849	49,4517	48,0584	46,7607	45,5274	44,3351
50	86,6603	82,6637	79,4898	76,1538	71,4202	67,5048	63,1671	60,3460	58,1638	56,3336	54,7228	53,2576	51,8916	50,5923	49,3349
55	93,1671	89,0344	85,7491	82,2920	77,3804	73,3115	68,7962	65,8550	63,5772	61,6650	59,9804	58,4469	57,0160	55,6539	54,3348
60	99,6078	95,3443	91,9518	88,3794	83,2977	79,0820	74,3970	71,3411	68,9721	66,9815	65,2265	63,6277	62,1348	60,7128	59,3347
70	112,3167	107,8079	104,2148	100,4251	95,0231	90,5313	85,5270	82,2553	79,7147	77,5766	75,6893	73,9677	72,3583	70,8236	69,3345
80	124,8389	120,1018	116,3209	112,3288	106,6285	101,8795	96,5782	93,1058	90,4053	88,1303	86,1197	84,2840	82,5663	80,9266	79,3343
90	137,2082	132,2554	128,2987	124,1162	118,1359	113,1452	107,5650	103,9040	101,0537	98,6499	96,5238	94,5809	92,7614	91,0234	89,3342
100	149,4488	144,2925	140,1697	135,8069	129,5613	124,3421	118,4980	114,6588	111,6667	109,1412	106,9058	104,8615	102,9459	101,1149	99,3341
120	173,6184	168,0814	163,6485	158,9500	152,2113	146,5673	140,2326	136,0620	132,8063	130,0546	127,6159	125,3833	123,2890	121,2850	119,3340
140	197,4498	191,5653	186,8465	181,8405	174,6478	168,6130	161,8270	157,3517	153,8537	150,8941	148,2686	145,8629	143,6043	141,4413	139,3339
160	221,0197	214,8081	209,8238	204,5300	196,9152	190,5164	183,3106	178,5517	174,8283	171,6752	168,8759	166,3092	163,8977	161,5868	159,3338
180	244,3723	237,8548	232,6198	227,0563	219,0442	212,3039	204,7036	199,6786	195,7434	192,4086	189,4462	186,7282	184,1732	181,7234	179,3338
200	267,5388	260,7350	255,2638	249,4452	241,0578	233,9942	226,0210	220,7441	216,6088	213,1022	209,9854	207,1244	204,4337	201,8526	199,3337
250	324,8306	317,3609	311,3460	304,9393	295,6885	287,8815	279,0504	273,1944	268,5987	264,6970	261,2253	258,0355	255,0327	252,1497	249,3337
300	381,4239	373,3509	366,8439	359,9064	349,8745	341,3951	331,7885	325,4090	320,3971	316,1383	312,3460	308,8589	305,5741	302,4182	299,3336
500	603,4458	593,3580	585,2060	576,4931	563,8514	553,1269	540,9303	532,8028	526,4014	520,9505	516,0874	511,6081	507,3816	503,3147	499,3335
600	712,7726	701,8322	692,9809	683,5155	669,7690	658,0936	644,8004	635,9329	628,8157	622,9876	617,6713	612,7718	608,1468	603,6942	599,3335

La Paz, 12 de junio de 2025

Señor:
Phd. José María Tapia Baltazar
Director de Carrera
Carrera de Informática
Facultad de Ciencias Puras y Naturales
Universidad Mayor de San Andrés
Presente. -

Ref. Aval de Conformidad de la Tesis de Grado

De mi consideración:

Tengo a bien dirigirme a su persona, para darle a conocer que luego de efectuar el seguimiento a la estructura y contenido de la tesis de grado titulado: **"DIGITALIZACIÓN INTELIGENTE DE RECIBOS MANUSCRITOS PARA PEQUEÑOS COMERCIANTES MEDIANTE UNA APLICACIÓN MÓVIL BASADA EN DEEP LEARNING"** elaborado por el universitario: **WHILLY EDGAR AMORAGA MAMANI con C.I.6720733**, en calidad de tutor, me corresponde **dar mi CONFORMIDAD Y AVAL**, para que el mismo procesa a los trámites correspondientes, de acuerdo a normas y reglamentos universitarios vigentes.

Sin otro particular, me despido de usted con las consideraciones más distinguidas.
Atentamente,



Dr. Juan Marcos Miranda Nina, Ph.D.
TUTOR